A Practical Guide to Control Theory

For Engineering Students in Robot Competitions

Skywalker Liu
ltx2499323877@outlook.com
github.com/liuskywalkerjskd

October 2025

Contents

1 Introduction **4**

2 Digital Signal Processing **5**
2.1 Understanding the Frequency Domain . . . 5
2.2 Laplace Transform and Fourier Transform . . . 5
2.2.1 Laplace Transform . . . 5
2.2.2 Fourier Transform . . . 6
2.2.3 Frequency Spectrum . . . 6
2.3 Frequency-Based Filter Design . . . 8
2.3.1 Low-Pass Filter . . . 8
2.3.2 Band-Pass Filter . . . 9
2.3.3 High-Pass Filter . . . 10
2.3.4 Filter Type Selection and Parameter Tuning . . . 10
2.4 Z-Transform . . . 10

3 System Description **12**
3.1 Mathematical Models of Systems . . . 12
3.1.1 Differential Equations . . . 12
3.1.2 Transfer Functions . . . 12
3.1.3 State-Space Representation . . . 13
3.2 System Analysis via Transfer Functions . . . 14
3.3 System Analysis via State-Space . . . 16
3.4 Motor Modeling: The Most Common Plant . . . 16

4 Classical Control Theory **21**
4.1 The Concept of Feedback Control . . . 21
4.2 System Performance Metrics . . . 22
4.2.1 Stability Margins . . . 22
4.3 Controller Design via Transfer Functions . . . 24
4.4 PID Controller . . . 24
4.4.1 PID Controller – Continuous Form . . . 24
4.4.2 PID Controller – Discrete Form . . . 26
4.4.3 Improved PID Controllers . . . 26
4.5 Advanced PID Improvements . . . 27
4.5.1 Integral Separation . . . 27
4.5.2 Derivative Placement: PID vs PI-D vs I-PD . . . 28
4.5.3 Filtered PID: Frequency Domain Proof . . . 29
4.5.4 Two-Degree-of-Freedom PID (2-DOF) . . . 31
4.5.5 Fuzzy PID . . . 32
4.6 PID Tuning Methods . . . 33
4.6.1 Empirical Tuning . . . 33
4.6.2 Tuning with MATLAB Simulink . . . 33
4.7 System Identification: Figuring Out What Your Plant Actually Does . . . 34
4.7.1 Step Response Method (Time Domain) . . . 34
4.7.2 Frequency Response Method (Bode Identification) . . . 34
4.7.3 Least-Squares Identification (Automated) . . . 35
4.7.4 Practical Sysid Workflow for Competition Robots . . . 35
4.8 Feedforward Control . . . 36
4.9 PID Architectures: Single, Cascaded, and Parallel-Cascaded . . . 37
4.9.1 Architecture 1: Single-Loop PID . . . 37
4.9.2 Architecture 2: Cascaded (Series) PID . . . 37
4.9.3 Architecture 3: Parallel-Cascaded PID (Multi-Axis) . . . 38
4.9.4 Comparison Summary . . . 38
4.9.5 Output Conflict in Parallel PID . . . 40
4.9.6 Food for Thought: Is Inner-Loop K_p the Same as Outer-Loop K_d? . . . 40

5 Discretization and Implementation **41**
5.1 From Continuous to Discrete . . . 41
5.1.1 Zero-Order Hold (ZOH) . . . 41
5.1.2 Tustin (Bilinear) Transform . . . 41
5.1.3 Matched Pole-Zero Method . . . 41
5.2 Fixed-Point vs Floating-Point Arithmetic . . . 42
5.3 Sampling Rate Selection . . . 43
5.4 Implementation on Embedded Systems . . . 44
5.4.1 Timer and Interrupt Configuration . . . 44
5.4.2 Computation Latency and Jitter . . . 44
5.4.3 Anti-Windup in Practice . . . 44

6 Modern Control Theory **47**
6.1 Relationship Between State-Space and Transfer Functions . . . 47
6.2 System Performance Metrics . . . 47
6.2.1 Controllability and Observability . . . 47
6.2.2 Closed-Loop System Stability . . . 49
6.3 Controller Design via State-Space . . . 49
6.3.1 Bellman Equations and Dynamic Programming: The Engine Behind LQR and MPC 49
6.3.2 LQR (Linear Quadratic Regulator) . . . 52
6.3.3 A Brief Primer on Optimization and Convexity . . . 52
6.3.4 MPC (Model Predictive Control) . . . 53
6.3.5 TinyMPC: Riccati-Accelerated MPC for Embedded Systems . . . 55
6.4 Observer Design via State-Space . . . 56
6.4.1 Luenberger Observer . . . 56
6.4.2 The Deep Duality: Controller and Observer as Mirror Images . . . 56
6.4.3 Kalman Filter . . . 57
6.4.4 Extended Kalman Filter (EKF) . . . 58
6.4.5 Quaternion-Based EKF: Avoiding Gimbal Lock . . . 60
6.4.6 Adaptive EKF: Auto-Tuning Q and R Online . . . 61
6.4.7 Luenberger Observer vs. Kalman Filter: A Comprehensive Comparison . . . 63
6.4.8 LQG (Linear Quadratic Gaussian) . . . 64
6.5 Trajectory Generation: Telling Your Motor Where to Go . . . 69
6.5.1 Trapezoidal Velocity Profile . . . 69
6.5.2 S-Curve Profile . . . 69
6.5.3 Bézier Curves and Spline Interpolation . . . 70

7 Appendix: Plug-and-Play C++ Modules **72**
7.1 Low-Pass Filter . . . 72
7.2 PID Controller . . . 73
7.3 Setpoint Ramp (LPF Soft-Start) . . . 74
7.4 Complementary Filter (IMU Tilt) . . . 75
7.5 Lightweight Matrix Library . . . 75
7.6 Kalman Filter . . . 77
7.7 LQR Gain (Offline Computation Helper) . . . 79
7.8 Cascaded PID . . . 80
7.9 Advanced PID (Integral Separation + PI-D + 2-DOF) . . . 81
7.10 EKF for 6-DOF IMU (Pose Estimation) . . . 82
7.11 TinyMPC Solver . . . 85
7.12 Cubic Bézier Trajectory . . . 87
7.13 Module Dependency Map . . . 89

1 Introduction

If you are reading this, you are probably an engineering student who has just joined a robotics competition team, staring at a motor that refuses to spin at the speed you want, or a gimbal that oscillates like it is possessed. Welcome. You are in the right place.

This document is **not** a control theory textbook. Textbooks are great—they are rigorous, comprehensive, and excellent at putting you to sleep at 2 AM. This is a *lecture note*, written by someone who learned most of this the hard way: by breaking robots first, then reading the theory to figure out why.

Our goal here is simple: to give you enough theoretical understanding to **design, tune, and debug controllers** on real hardware. We will cover the math, but we will always ask: "Why does this matter when my robot is on the field?"

Who Is This For?

Undergraduate students in their early and middle years, especially those involved in competitions like RoboMaster, FIRST Robotics, RoboCup, or similar events. We assume you have taken (or are currently taking) calculus and basic linear algebra. If you know what a derivative is and can multiply matrices without crying, you are ready.

How This Note Is Organized

We start with **Digital Signal Processing**—because before you can control anything, you need clean data from your sensors. Then we learn how to **describe systems** mathematically, which is a fancy way of saying "write down what your motor actually does." Next comes **Classical Control Theory**, where PID lives—the bread and butter of competition robotics. After that, we tackle **Discretization and Implementation**, bridging the gap between elegant math and the harsh reality of a microcontroller running at a fixed clock rate. Finally, we visit **Modern Control Theory** for the tools that separate good robots from great ones: LQR, MPC, and Kalman filters.

Let us begin.

2 Digital Signal Processing

You might be wondering: "I signed up for control theory, why are we starting with signal processing?" Great question. Here is the answer: **garbage in, garbage out.**

Your controller is only as good as the data it receives. If your IMU reading is bouncing around like a caffeinated squirrel, no amount of PID tuning will save you. So before we learn to control systems, we learn to clean up their signals.

2.1 Understanding the Frequency Domain

In everyday life, we think about signals in the **time domain**—voltage changes over time, position changes over time. This is natural and intuitive. But it turns out that many problems become *much* easier if we look at signals from a different angle: the **frequency domain**.

The core idea is beautifully simple: *any signal can be decomposed into a sum of sinusoids of different frequencies.* Your noisy gyroscope reading? It is actually a clean low-frequency rotation signal plus a bunch of high-frequency noise signals all added together. If we can identify which frequencies are "signal" and which are "noise," we can surgically remove the noise.

Think of it like an audio equalizer on a music player. The bass, mids, and treble are all mixed together in the time-domain waveform. But the equalizer lets you boost or cut specific frequency ranges independently. That is exactly what we do with filters in control systems.

Key intuition: Slow, meaningful changes in your system (like a chassis turning) happen at low frequencies. Fast, meaningless changes (like electrical noise or vibration) happen at high frequencies. The frequency domain lets us tell them apart.

2.2 Laplace Transform and Fourier Transform

These two transforms are your tickets from the time domain to the frequency domain. They are closely related, but they serve slightly different purposes.

2.2.1 Laplace Transform

The Laplace transform takes a time-domain function $f(t)$ and converts it into a function of the complex variable s:

$$F(s) = \mathcal{L}\{f(t)\} = \int_0^\infty f(t)e^{-st}\,dt \tag{1}$$

where $s = \sigma + j\omega$ is a complex number. The real part σ captures exponential growth or decay, and the imaginary part $j\omega$ captures oscillation.

Why do we care? Because the Laplace transform turns differential equations into algebraic equations. Instead of solving

$$m\ddot{x} + b\dot{x} + kx = F(t),$$

(which requires calculus), we can solve

$$(ms^2 + bs + k)X(s) = F(s),$$

(which is just algebra). Derivatives become multiplication by s. Integration becomes division by s. Life becomes easier.

The most important Laplace transforms you will use daily:

Time domain	**s-domain**	**When you see it**
1 (step input)	$\frac{1}{s}$	Commanding a new setpoint
e^{-at}	$\frac{1}{s+a}$	First-order system response
$\sin(\omega t)$	$\frac{\omega}{s^2+\omega^2}$	Oscillation, vibration
$\dot{f}(t)$	$sF(s) - f(0)$	Derivative in the controller
$\int f(t)\,dt$	$\frac{F(s)}{s}$	Integral in the controller

Pro tip: You do not need to memorize the integral formula. You need to understand that s means "derivative" and $1/s$ means "integral." That single insight will carry you through 80% of control theory.

What is e^{-st} actually doing? The kernel $e^{-st} = e^{-\sigma t}e^{-j\omega t}$ is a "probe signal" that asks two questions simultaneously: (1) $e^{-\sigma t}$: "Does the signal grow or decay at rate σ?" (2) $e^{-j\omega t}$: "How much of frequency ω is present?" By integrating $f(t) \cdot e^{-st}$ over all time, we compute the "correlation" between the signal and every possible exponential-sinusoid. The result $F(s)$ is a map of the entire complex plane, telling us exactly how much of each exponential-oscillatory mode exists in the signal.

Common Laplace transform pairs — worked out:

The **unit step** $u(t) = 1$ for $t \geq 0$:

$$\mathcal{L}\{1\} = \int_0^\infty e^{-st}\,dt = \left[-\frac{e^{-st}}{s}\right]_0^\infty = \frac{1}{s}$$

The **exponential decay** e^{-at}:

$$\mathcal{L}\{e^{-at}\} = \int_0^\infty e^{-at}e^{-st}\,dt = \int_0^\infty e^{-(s+a)t}\,dt = \frac{1}{s+a}$$

This is why a first-order system with pole at $s = -a$ has a time response of e^{-at}—the pole location *is* the decay rate.

The **ramp** $t \cdot u(t)$:

$$\mathcal{L}\{t\} = \frac{1}{s^2}$$

Notice: multiplying by t in time corresponds to differentiating with respect to s and flipping the sign: $\mathcal{L}\{t \cdot f(t)\} = -\frac{d}{ds}F(s)$. Since $\mathcal{L}\{1\} = 1/s$, we get $\mathcal{L}\{t\} = -\frac{d}{ds}\frac{1}{s} = \frac{1}{s^2}$.

2.2.2 Fourier Transform

The Fourier transform is the Laplace transform's well-behaved cousin. It is what you get when you set $s = j\omega$ (i.e., $\sigma = 0$):

$$F(j\omega) = \int_{-\infty}^{\infty} f(t)e^{-j\omega t}\,dt \tag{2}$$

While the Laplace transform works with the full complex plane (and can handle unstable, growing signals), the Fourier transform lives strictly on the imaginary axis. It tells you: "How much of each frequency is present in this signal?"

When to use which:

- **Laplace transform**: For analyzing and designing control systems (transfer functions, stability).
- **Fourier transform**: For analyzing signals (noise, vibration, frequency content).

In practice, for digital systems we often use the **Discrete Fourier Transform (DFT)**, computed efficiently via the **Fast Fourier Transform (FFT)** algorithm. If you ever want to see what frequencies are hiding in your sensor data, just throw it into an FFT. MATLAB's `fft()` or Python's `numpy.fft.fft()` will do the job in one line.

2.2.3 Frequency Spectrum

When you apply the Fourier transform to a signal, you get its **frequency spectrum**—a plot showing the amplitude (and phase) of each frequency component.

Reading a frequency spectrum is like reading an X-ray of your signal:

- A tall peak at 50 Hz? That is probably power line interference.
- A broad hump below 10 Hz? That is likely your actual signal of interest.
- Random fuzz spread across all frequencies? That is white noise.

Practical example: Suppose your gimbal encoder gives a noisy velocity reading. You record 1 second of data, compute the FFT, and see a clean peak at 3 Hz (your gimbal is tracking a target moving at 3 Hz) plus a mess of noise above 50 Hz. Now you know: design a low-pass filter with a cutoff around 20–30 Hz, and your signal will be clean.

The frequency spectrum has two parts:

- **Magnitude spectrum** $|F(j\omega)|$: How strong each frequency component is.
- **Phase spectrum** $\angle F(j\omega)$: The time shift of each frequency component.

For filter design, we mostly care about the magnitude spectrum. Phase becomes important later when we analyze control loop stability (phase margin).

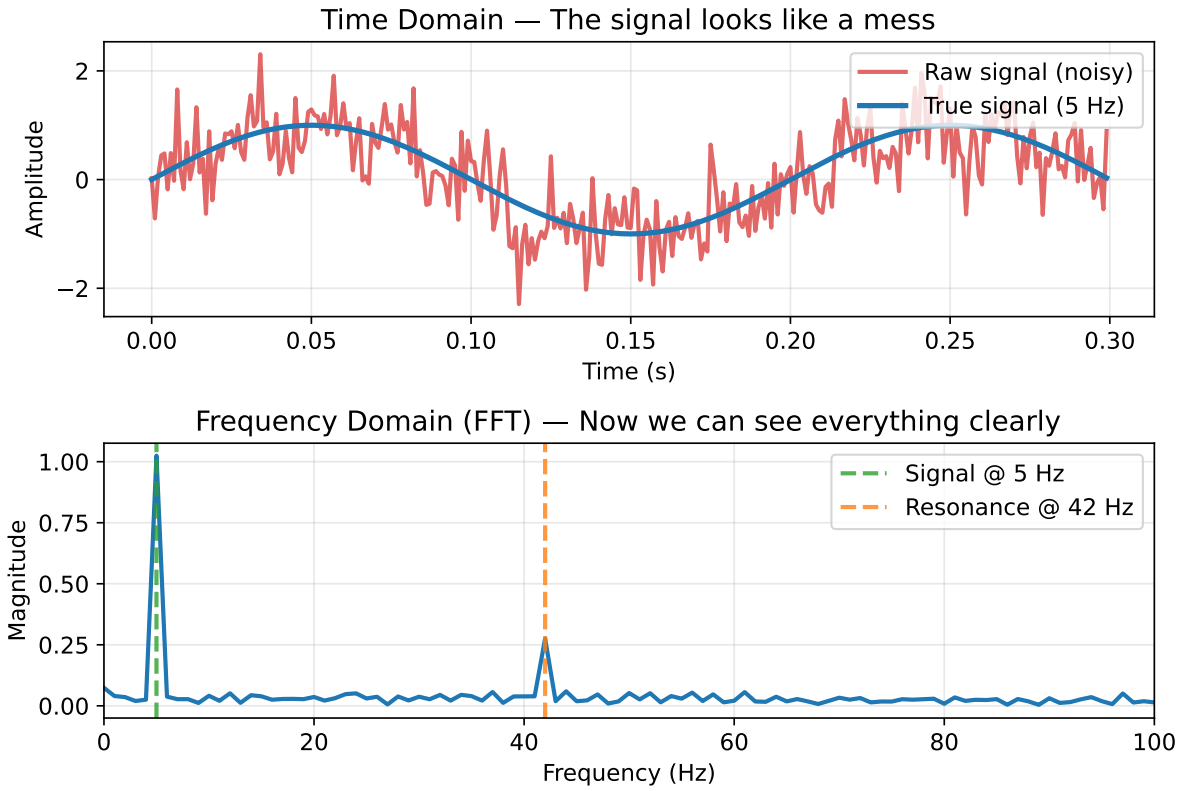

Figure 1: Top: a noisy time-domain signal looks hopeless. Bottom: the FFT instantly reveals a 5 Hz signal and a 42 Hz resonance hiding in the noise.

Case Study: Debugging a Vibrating Chassis with FFT

True story pattern from competition robotics: Your mecanum chassis drives fine at low speed but develops a nasty vibration at high speed. The whole frame buzzes. Everyone suspects a loose screw. You tighten everything. Still vibrates.

The diagnosis: Log the IMU accelerometer data for 5 seconds while driving at the problematic speed. Run an FFT in Python:

```
import numpy as np
freqs = np.fft.rfftfreq(len(data), d=1.0/sample_rate)
spectrum = np.abs(np.fft.rfft(data))
plt.plot(freqs, spectrum)
```

You see a massive spike at exactly 42 Hz. Interesting. You calculate: at high speed, the mecanum wheel rollers contact the ground at…about 42 Hz. The roller contact frequency matches a structural resonance of the chassis frame.

The fix options:

- Add a **notch filter** (band-reject) at 42 Hz in the control loop to prevent the controller from exciting the resonance.
- Add physical damping (rubber mounts) to shift the resonance frequency.
- Stiffen the frame to move the resonance above the operating range.

The lesson: Without the FFT, you would have spent hours tightening screws. With the FFT, the root cause was obvious in 10 seconds. **The frequency domain is a debugging superpower.** Whenever something oscillates and you do not know why, record the data and look at the spectrum.

2.3 Frequency-Based Filter Design

Now that we can see signals in the frequency domain, we can design filters to keep the frequencies we want and reject the ones we do not.

A filter is simply a system that has different gains at different frequencies. We describe this using the filter's **frequency response**—a plot of gain vs. frequency (called a **Bode magnitude plot**).

2.3.1 Low-Pass Filter

The most common filter in robotics. It passes low frequencies and attenuates high frequencies.

First-order low-pass filter:

$$H(s) = \frac{\omega_c}{s + \omega_c} \tag{3}$$

where $\omega_c = 2\pi f_c$ is the cutoff frequency in rad/s. At frequencies below ω_c, the gain is approximately 1 (signal passes through). Above ω_c, the gain rolls off at -20 dB/decade (signal gets attenuated).

In the time domain, this is equivalent to an exponential moving average:

$$y[n] = \alpha \cdot x[n] + (1 - \alpha) \cdot y[n - 1] \tag{4}$$

where $\alpha = \frac{T_s}{T_s + \frac{1}{2\pi f_c}}$ and T_s is the sampling period.

When to use: Smoothing noisy sensor readings (gyroscope, accelerometer, motor current). This is the filter you will use 90% of the time.

Gotcha: A low-pass filter introduces **phase lag**—the output is delayed relative to the input. The more aggressively you filter (lower cutoff), the more lag you introduce. In a fast control loop, this lag can destabilize your system. There is always a trade-off between smooth signals and fast response.

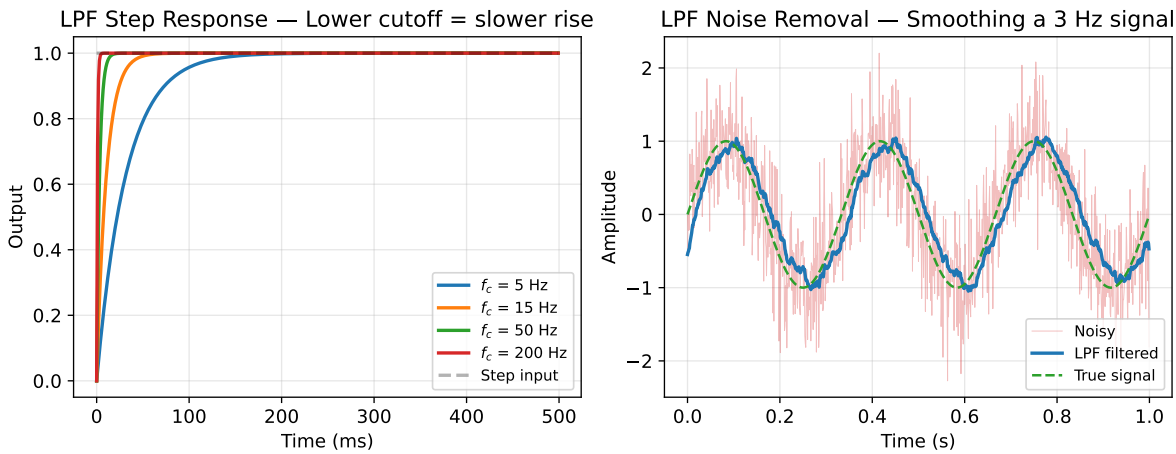

Figure 2: Left: LPF step response — lower cutoff means slower rise but smoother output. Right: filtering a noisy 3 Hz signal — the LPF recovers the true signal while removing high-frequency noise.

Case Study: Why a Low-Pass Filter Gives You "Soft Start" for Free

Suppose you command your motor to jump from 0 to 5000 RPM instantly (a step input). Without any filter, the setpoint changes discontinuously—the motor driver sees a sudden full-voltage command, the gears slam, the chassis lurches, and your teammates spill their coffee.

Now insert a first-order LPF on the setpoint: $r_{\text{filtered}}(s) = \frac{\omega_c}{s+\omega_c} \cdot R(s)$. What happens?

The step input in the s-domain is $R(s) = \frac{5000}{s}$. After the filter:

$$R_{\text{filtered}}(s) = \frac{5000\,\omega_c}{s(s+\omega_c)}$$

Taking the inverse Laplace transform:

$$r_{\text{filtered}}(t) = 5000\left(1 - e^{-\omega_c t}\right)$$

The step has been transformed into a **smooth exponential ramp**. The time constant is $\tau = 1/\omega_c$. After 3τ, the output reaches 95% of the target; after 5τ, it is at 99%.

Why this matters on your robot:

- **Mechanical protection:** Gears and belts have torque limits. A step command can strip teeth or snap belts. The LPF limits the rate of change, keeping forces within safe bounds.
- **Current limiting:** A sudden voltage step causes a huge inrush current ($I = V/R$ when back-EMF is still zero). The LPF ramps up the voltage gradually, keeping current manageable.
- **Tunable aggressiveness:** Want a faster ramp? Increase ω_c (higher cutoff). Want a gentler ramp? Decrease ω_c. One parameter controls the entire behavior.

Comparison with a linear ramp: You could also ramp the setpoint linearly (e.g., increase by 1000 RPM/s). But the LPF is better because it also smooths out *any* discontinuity in the setpoint—not just the initial step, but also sudden changes during operation. It is a "set it and forget it" solution.

The code is trivial:

```
setpoint_filtered += alpha * (setpoint_raw - setpoint_filtered);
```

One line of code. No lookup tables, no state machines. That is the beauty of the LPF as a soft-start mechanism.

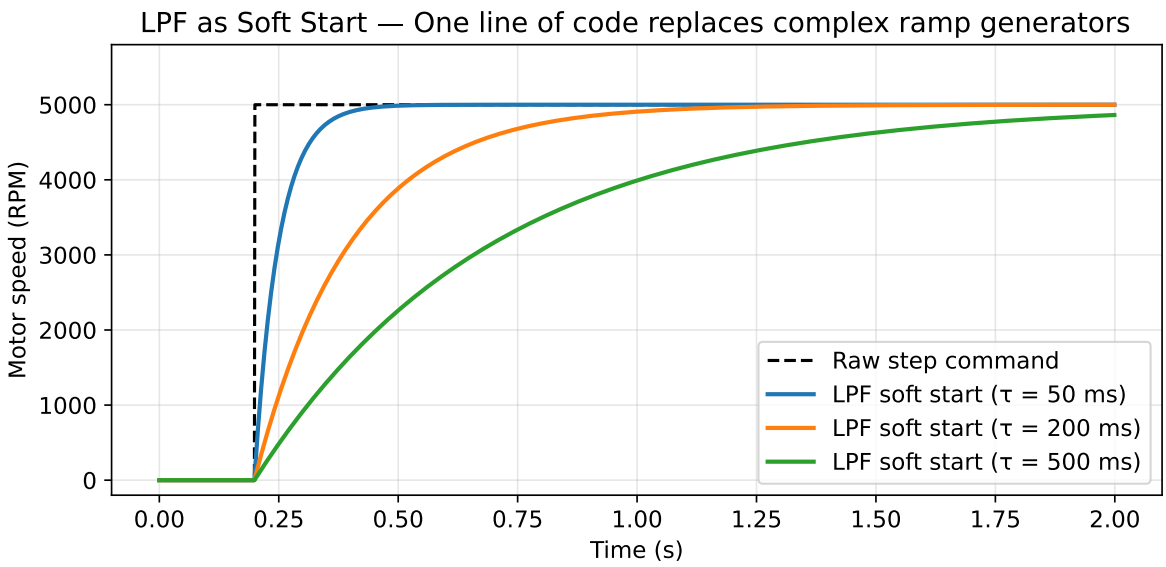

Figure 3: LPF soft start: a single parameter τ controls how aggressively the motor ramps up. The step command becomes a smooth exponential curve.

2.3.2 Band-Pass Filter

Passes a specific range of frequencies and attenuates everything else. Think of it as a low-pass and a high-pass filter combined.

$$H(s) = \frac{\frac{\omega_0}{Q}s}{s^2 + \frac{\omega_0}{Q}s + \omega_0^2} \tag{5}$$

where ω_0 is the center frequency and Q is the quality factor (higher Q = narrower band).

When to use: Extracting a specific frequency component from a signal. For example, if you know your target oscillates at a specific frequency and you want to track it while rejecting everything else.

In robotics, band-pass filters are less common than low-pass filters, but they are useful for vibration analysis and resonance detection.

2.3.3 High-Pass Filter

Passes high frequencies, attenuates low frequencies. It is the opposite of a low-pass filter.

First-order high-pass filter:

$$H(s) = \frac{s}{s + \omega_c} \tag{6}$$

When to use: Removing DC offset or slow drift from a signal. For example, if your accelerometer has a slowly drifting bias, a high-pass filter can remove it while keeping the fast acceleration data intact.

Practical note: A complementary filter (commonly used for IMU sensor fusion) is just a low-pass filter on one sensor and a high-pass filter on another, with their cutoff frequencies matched so the gains add up to 1. The accelerometer gets low-passed (trust it for slow/static orientation) and the gyroscope gets high-passed (trust it for fast rotations). Simple, elegant, and surprisingly effective.

2.3.4 Filter Type Selection and Parameter Tuning

Choosing the right filter comes down to answering three questions:

1. **What frequencies do I want to keep?** This determines the filter type (low-pass, high-pass, band-pass).
2. **Where is the boundary?** This determines the cutoff frequency f_c.
3. **How sharp should the transition be?** This determines the filter order.

Rules of thumb:

- Start with a first-order low-pass filter. It is simple, stable, and good enough for most applications.
- Set the cutoff frequency to 5–10× the bandwidth of your signal of interest. Too low and you lose signal; too high and you keep noise.
- If a first-order filter is not enough, try second-order (Butterworth for flat passband, Chebyshev for sharper rolloff).
- **Never** design a filter without looking at the frequency spectrum of your actual data first. Guessing cutoff frequencies is a recipe for frustration.
- When in doubt, plot the filtered output alongside the raw signal and check if the result makes physical sense.

2.4 Z-Transform

Everything we have discussed so far with the Laplace transform lives in the continuous world. But your microcontroller is digital—it works with samples, not continuous signals. The **Z-transform** is the discrete-time equivalent of the Laplace transform.

The Z-transform of a discrete sequence $x[n]$ is:

$$X(z) = \sum_{n=0}^{\infty} x[n] z^{-n} \tag{7}$$

The key correspondence is:

Continuous (s-domain)	**Discrete (z-domain)**
s (derivative)	$\frac{z-1}{T_s}$ (forward difference)
$\frac{1}{s}$ (integral)	$\frac{T_s}{z-1}$ (accumulator)
Stable if $\mathrm{Re}(s) < 0$	Stable if $\lvert z\rvert < 1$
e^{-aT_s} in s-domain	z^{-1} is one-sample delay

Why do we care? Because when you implement a filter or controller on a microcontroller, you are working in the z-domain, whether you realize it or not. That simple line of code:

```
y = alpha * x + (1 - alpha) * y_prev;
```

is actually a z-domain transfer function:

$$H(z) = \frac{\alpha}{1 - (1-\alpha)z^{-1}}$$

Understanding the Z-transform helps you:

- Convert continuous-time filters/controllers to code (covered in Section 5).
- Analyze the stability of your discrete-time system.
- Understand why your controller might behave differently at different sampling rates.

For now, just remember: z^{-1} **means "one time step ago."** That is the single most important thing about the Z-transform.

Common Z-transform pairs:

Discrete signal	**z-domain**	**When you see it**
$\delta[n]$ (unit impulse)	1	Initial kick
$u[n]$ (unit step)	$\frac{z}{z-1}$	Step command
$a^n u[n]$ (geometric decay)	$\frac{z}{z-a}$	Discrete first-order response
$x[n-1]$ (one-step delay)	$z^{-1}X(z)$	Previous sample in code
$x[n] - x[n-1]$ (first difference)	$(1-z^{-1})X(z)$	Discrete derivative
$\sum_{k=0}^{n} x[k]$ (running sum)	$\frac{1}{1-z^{-1}}X(z)$	Discrete integral

The connection between s and z: The exact mapping is $z = e^{sT_s}$. A continuous pole at $s = -a$ maps to a discrete pole at $z = e^{-aT_s}$. This means:

- A stable continuous pole ($\mathrm{Re}(s) < 0$) maps to $|z| < 1$ (inside the unit circle). Stability is preserved.
- A faster continuous pole (more negative s) maps to a smaller $|z|$ (closer to the origin). Very fast poles map near $z = 0$.
- An oscillatory continuous pole ($s = \sigma \pm j\omega$) maps to $z = e^{\sigma T_s} e^{\pm j\omega T_s}$, which spirals inside (stable) or outside (unstable) the unit circle.

Reading a z-domain transfer function: Any z-domain transfer function can be directly translated to code by replacing z^{-1} with "previous value." For example:

$$H(z) = \frac{b_0 + b_1 z^{-1}}{1 + a_1 z^{-1}}$$

becomes:

```
y = b0 * x + b1 * x_prev - a1 * y_prev;
x_prev = x;  y_prev = y;
```

This is called a **difference equation**, and it is the final form of any digital filter or controller.

3 System Description

Before you can control something, you need to understand what it *does*. A system is anything that takes an input and produces an output. Your DC brushed motor takes voltage and produces angular velocity. Your chassis takes wheel speeds and produces a trajectory. Your gimbal takes motor torques and produces angles.

The art of control engineering begins with writing down a mathematical description of these input-output relationships. This is called **modeling**, and it is simultaneously the most important and most underappreciated step in the process.

A word of wisdom: A mediocre controller on a good model will outperform a perfect controller on a bad model, every single time. Spend time understanding your system.

We describe systems as either **SISO** (Single-Input, Single-Output) or **MIMO** (Multiple-Input, Multiple-Output). A single motor is SISO. A holonomic chassis with four wheels is MIMO. We will start with SISO because it is simpler, and generalize later.

3.1 Mathematical Models of Systems

There are three main ways to describe a system mathematically, each with its own strengths.

3.1.1 Differential Equations

The most fundamental description. We use physics (Newton's laws, Kirchhoff's laws, etc.) to write equations relating inputs, outputs, and their derivatives.

Example: DC brushed motor

A DC brushed motor is the most common actuator in robotics competitions. Here is its simplified model:

Electrical equation (Kirchhoff's voltage law):

$$V = L\frac{di}{dt} + Ri + K_e\omega \tag{8}$$

Mechanical equation (Newton's second law for rotation):

$$J\frac{d\omega}{dt} = K_t i - b\omega - \tau_{\text{load}} \tag{9}$$

where:

- V: input voltage, i: armature current
- L: inductance, R: resistance
- K_e: back-EMF constant, K_t: torque constant
- J: moment of inertia, b: viscous friction
- ω: angular velocity, τ_{load}: load torque

These are two coupled first-order differential equations. In practice, the electrical dynamics ($L\frac{di}{dt}$) are much faster than the mechanical dynamics, so we often simplify by setting $L \approx 0$, giving us:

$$J\frac{d\omega}{dt} = \frac{K_t}{R}(V - K_e\omega) - b\omega - \tau_{\text{load}} \tag{10}$$

This is a first-order ODE relating input voltage V to output speed ω. Now we can analyze and control it.

3.1.2 Transfer Functions

Take the Laplace transform of the differential equation (assuming zero initial conditions), and you get a **transfer function**: the ratio of output to input in the s-domain.

For our simplified DC brushed motor:

$$G(s) = \frac{\Omega(s)}{V(s)} = \frac{K_t/R}{Js + (b + K_t K_e/R)} \tag{11}$$

This can be rewritten in standard first-order form:

$$G(s) = \frac{K}{\tau s + 1} \tag{12}$$

where K is the DC gain (steady-state speed per volt) and τ is the time constant (how fast the motor reaches steady state).

Why transfer functions are great:

- They are compact—one fraction describes the entire system.
- Cascading systems is just multiplication: $G_{\text{total}}(s) = G_1(s) \cdot G_2(s)$.
- Stability analysis is straightforward: check the roots of the denominator (poles).
- Frequency response is obtained by substituting $s = j\omega$.

Why transfer functions are limited:

- They only work for linear, time-invariant (LTI) systems.
- They assume zero initial conditions.
- They are naturally SISO—extending to MIMO requires transfer function matrices, which get unwieldy.
- They hide the internal states of the system.

3.1.3 State-Space Representation

The state-space representation describes a system as a set of first-order differential equations in matrix form:

$$\dot{\mathbf{x}} = A\mathbf{x} + B\mathbf{u} \quad \text{(state equation)} \tag{13}$$

$$\mathbf{y} = C\mathbf{x} + D\mathbf{u} \quad \text{(output equation)} \tag{14}$$

where:

- $\mathbf{x}$: state vector (the internal variables that describe the system's condition)
- $\mathbf{u}$: input vector
- $\mathbf{y}$: output vector
- A: system matrix, B: input matrix, C: output matrix, D: feedthrough matrix

What each matrix physically means:

- A (system/dynamics matrix): Describes how the states evolve on their own, without any input. If you disconnect the motor ($\mathbf{u} = 0$), the system evolves as $\dot{\mathbf{x}} = A\mathbf{x}$. The eigenvalues of A determine the natural behavior: stable decay, oscillation, or divergence. Entry A_{ij} tells you how state j influences the rate of change of state i.
- B (input matrix): Describes how the inputs affect the states. Entry B_{ij} tells you how input j influences the rate of change of state i. If a row of B is all zeros, that state cannot be directly driven by any input—it can only be influenced indirectly through coupling in A.
- C (output matrix): Describes which states you can measure. Entry C_{ij} tells you how much state j contributes to measurement i. If $C = [1\ 0\ \ldots\ 0]$, you are measuring only the first state.
- D (feedthrough matrix): Describes direct input-to-output coupling that bypasses the dynamics. In most physical systems, $D = 0$ because inputs must go through the dynamics before affecting the output. $D \neq 0$ appears when there is a direct algebraic path from input to output (e.g., a resistive voltage divider).

Example: DC brushed motor in state-space

Choose states $\mathbf{x} = \begin{bmatrix} \omega \\ i \end{bmatrix}$ (angular velocity and current):

$$\begin{bmatrix} \dot{\omega} \\ \dot{i} \end{bmatrix} = \begin{bmatrix} -b/J & K_t/J \\ -K_e/L & -R/L \end{bmatrix} \begin{bmatrix} \omega \\ i \end{bmatrix} + \begin{bmatrix} 0 \\ 1/L \end{bmatrix} V \tag{15}$$

$$y = \begin{bmatrix} 1 & 0 \end{bmatrix} \begin{bmatrix} \omega \\ i \end{bmatrix} \tag{16}$$

Why state-space is powerful:

- Handles MIMO systems naturally—just make the matrices bigger.
- Exposes internal states, which we can estimate (observers) and control (state feedback).
- Works seamlessly with modern control techniques (LQR, MPC, Kalman filter).
- Easily extended to nonlinear systems (replace $A\mathbf{x}$ with $f(\mathbf{x})$).

The trade-off: State-space is more powerful but more abstract. Transfer functions give you better physical intuition for SISO systems. Good engineers are fluent in both.

3.2 System Analysis via Transfer Functions

Given a transfer function $G(s)$, we can extract a wealth of information about the system.

Poles and zeros. Write $G(s) = \frac{N(s)}{D(s)}$. The roots of $N(s)$ are the **zeros**; the roots of $D(s)$ are the **poles**.

- **Poles determine stability.** If all poles have negative real parts (i.e., they are in the left half of the complex plane), the system is stable. If any pole has a positive real part, the system is unstable and will blow up. Poles on the imaginary axis mean the system is marginally stable (perpetual oscillation).
- **Poles determine speed.** Poles farther to the left mean faster dynamics. The *dominant poles* (closest to the imaginary axis) determine the overall system speed.
- **Complex poles mean oscillation.** A pair of complex conjugate poles $s = -\sigma \pm j\omega_d$ produces oscillatory behavior. The real part σ determines how fast the oscillation decays; the imaginary part ω_d determines the oscillation frequency.
- **Zeros shape the response.** Zeros can cancel poles, reshape the transient response, or cause non-minimum phase behavior (the system initially moves in the wrong direction).

Quick trick: determining system order by counting integrations. Before diving into the math, here is an intuitive shortcut. The order of a system equals the number of times you must integrate the input variable to arrive at the output variable in a physical sense. For a DC brushed motor controlled by voltage:

- Voltage $\rightarrow$ current (one integration through the inductance $L\frac{di}{dt} = V - \ldots$) $\rightarrow$ that is one integrator.
- Current $\rightarrow$ angular velocity (one integration through the inertia $J\frac{d\omega}{dt} = K_t i - \ldots$) $\rightarrow$ that is a second integrator.

Two integrations from voltage to velocity $\Rightarrow$ second-order system. If we simplify $L \approx 0$ (current responds instantly), we remove one integration, leaving a first-order system. If the output is *position* instead of velocity, add one more integration ($\theta = \int \omega \, dt$), making it third-order (or second-order with the $L \approx 0$ simplification). This counting trick works for any physical system: springs, thermal systems, fluid tanks—count the independent energy storage elements (inductors, capacitors, masses, springs), and you have the system order.

Standard second-order system. Many physical systems can be approximated as:

$$G(s) = \frac{\omega_n^2}{s^2 + 2\zeta\omega_n s + \omega_n^2} \tag{17}$$

where ω_n is the natural frequency and ζ is the damping ratio. The damping ratio tells you everything about the transient response:

ζ	**Behavior**
$\zeta > 1$	Overdamped: slow, no oscillation
$\zeta = 1$	Critically damped: fastest without oscillation
$0 < \zeta < 1$	Underdamped: oscillation with decay
$\zeta = 0$	Undamped: perpetual oscillation
$\zeta < 0$	Unstable: growing oscillation

For most robotics applications, you want ζ between 0.5 and 0.8—fast response with acceptable overshoot.

Useful formulas for the underdamped case ($0 < \zeta < 1$):

The step response is:

$$y(t) = 1 - \frac{e^{-\zeta\omega_n t}}{\sqrt{1-\zeta^2}} \sin(\omega_d t + \phi), \quad \omega_d = \omega_n\sqrt{1-\zeta^2}, \quad \phi = \arccos(\zeta) \tag{18}$$

where ω_d is the **damped natural frequency** — the actual frequency at which the system oscillates (always less than ω_n).

From this, we can derive closed-form expressions for the performance metrics:

$$\text{Percent overshoot:} \quad M_p = e^{-\pi\zeta/\sqrt{1-\zeta^2}} \times 100\% \tag{19}$$

$$\text{Peak time:} \quad t_p = \frac{\pi}{\omega_d} = \frac{\pi}{\omega_n\sqrt{1-\zeta^2}} \tag{20}$$

$$\text{Settling time (2\%):} \quad t_s \approx \frac{4}{\zeta\omega_n} \tag{21}$$

$$\text{Rise time (approx):} \quad t_r \approx \frac{1.8}{\omega_n} \tag{22}$$

What these formulas tell you:

- Overshoot depends *only* on ζ, not on ω_n. A faster system (ω_n higher) overshoots by the same percentage—it just does it faster.
- Settling time depends on the product $\zeta\omega_n$ — this is the real part of the poles. Faster settling requires poles farther left in the complex plane.
- There is a direct trade-off: lower ζ gives faster rise time but more overshoot. Higher ζ gives less overshoot but slower response.
- For $\zeta = 0.707$ ($1/\sqrt{2}$): overshoot $\approx 4.3\%$, which is often cited as the "optimal" damping for the best compromise between speed and smoothness.

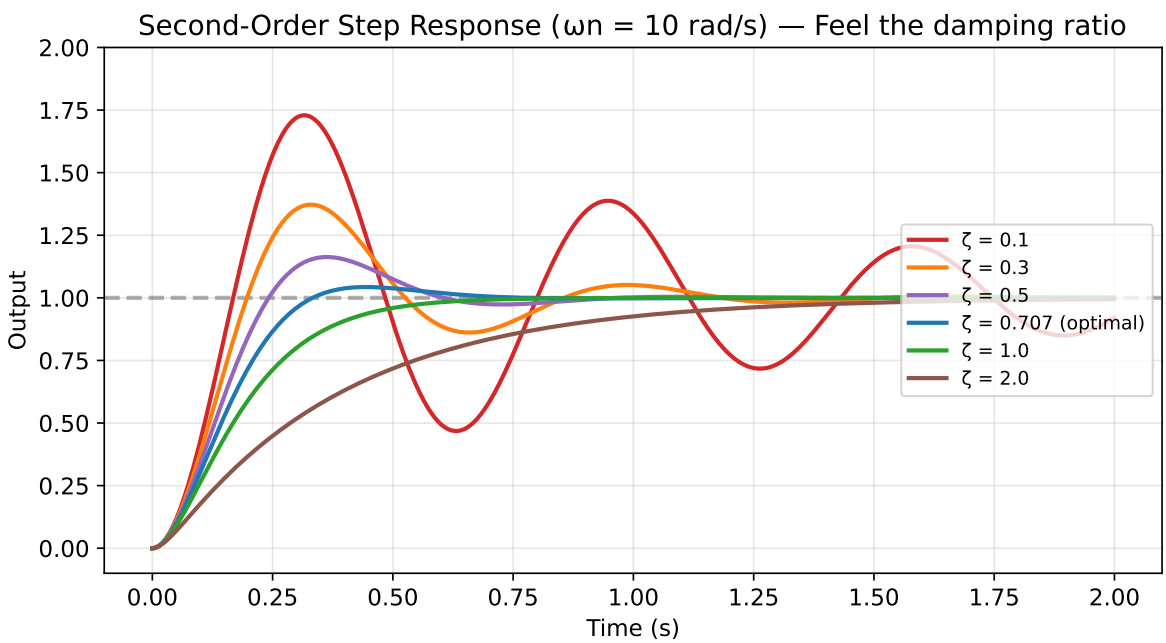

Figure 4: Second-order step response for different damping ratios. Notice how $\zeta = 0.707$ (blue) gives the fastest rise without significant overshoot — this is the "sweet spot" for most control applications.

Case Study: The Spring-Mass-Damper — A System You Can Feel

Before we get more abstract, let us build intuition with the simplest second-order system that you have literally felt with your own body: a car's suspension.

A car hitting a bump is a textbook spring-mass-damper: the car body (mass m) bounces on the suspension spring (stiffness k) while the shock absorber (damping coefficient b) dissipates energy.

$$m\ddot{x} + b\dot{x} + kx = F(t)$$

Transfer function: $G(s) = \frac{1}{ms^2+bs+k} = \frac{1/m}{s^2+\frac{b}{m}s+\frac{k}{m}}$

Natural frequency: $\omega_n = \sqrt{k/m}$. Damping ratio: $\zeta = \frac{b}{2\sqrt{km}}$.

Now feel the math:

- **No shock absorber ($\zeta = 0$):** You hit a bump and bounce forever. This is a trampoline, not a car. Undamped.
- **Worn-out shocks ($\zeta = 0.2$):** You hit a bump and bounce 5–6 times before settling. Passengers feel seasick. Underdamped.
- **Good shocks ($\zeta = 0.5$–0.7):** You hit a bump, bounce once or twice gently, and settle quickly. Comfortable. This is the sweet spot.
- **Off-road truck shocks ($\zeta = 1$):** You hit a bump and slowly return to center with no bounce at all. Safe but stiff. Critically damped.
- **Shocks filled with concrete ($\zeta = 3$):** You hit a bump and it takes forever to return to center. You feel every pebble. Overdamped.

The connection to your robot: Your gimbal, your chassis, your robotic arm—they all have natural frequencies and damping. When your gimbal oscillates after a step command, it is underdamped (ζ too low). When it responds sluggishly, it is overdamped (ζ too high). The PID controller's job (Section 4) is essentially to *change the effective ζ and ω_n* of your system by adding virtual springs (P), virtual dampers (D), and memory (I).

Why ω_n matters: A heavy gimbal with a weak motor has a low ω_n—it physically cannot respond fast. No amount of PID tuning can make a system respond faster than its natural frequency allows. If you need more speed, you need a bigger motor (increase k) or a lighter load (decrease m). This is why hardware design and control design must go hand in hand.

3.3 System Analysis via State-Space

For the state-space model $\dot{\mathbf{x}} = A\mathbf{x} + B\mathbf{u}$, system behavior is determined by the **eigenvalues of** A.

Here is the beautiful connection: *the eigenvalues of A are exactly the poles of the transfer function.* So everything we said about poles applies here too. The eigenvalue analysis just extends naturally to MIMO systems where transfer functions become awkward.

Matrix exponential. The free response (no input) of the state-space system is:

$$\mathbf{x}(t) = e^{At}\mathbf{x}(0) \tag{23}$$

The matrix exponential e^{At} encodes all the dynamics. If you diagonalize A (when possible), you get:

$$e^{At} = P \begin{bmatrix} e^{\lambda_1 t} & & \\ & \ddots & \\ & & e^{\lambda_n t} \end{bmatrix} P^{-1} \tag{24}$$

Each eigenvalue λ_i contributes an exponential mode. Negative real eigenvalues decay. Positive ones grow. Complex ones oscillate. This is the state-space way of saying "poles determine everything."

3.4 Motor Modeling: The Most Common Plant

Every controller needs a plant model. In robotics competitions, 95% of your plants are electric motors. Whether you are driving wheels, spinning flywheels, or controlling a turret, understanding how to model a motor is the single most useful skill you can have before touching any control algorithm.

DC Brushed Motor Model

A **DC brushed motor** has two coupled subsystems: electrical and mechanical.

Electrical dynamics (KVL around the armature loop):

$$V = L\frac{di}{dt} + Ri + K_e\omega \tag{25}$$

Mechanical dynamics (Newton's second law on the rotor):

$$J\frac{d\omega}{dt} = K_t i - b\omega - \tau_{\text{load}} \tag{26}$$

where V is input voltage, i armature current, ω angular velocity, L inductance, R resistance, K_e back-EMF constant, K_t torque constant, J rotor inertia, b viscous friction, and τ_{load} external load torque.

Taking the Laplace transform (with $\tau_{\text{load}} = 0$) and eliminating $I(s)$ yields the **full second-order transfer function**:

$$\frac{\Omega(s)}{V(s)} = \frac{K_t}{JLs^2 + (JR + bL)s + (bR + K_tK_e)} \tag{27}$$

First-order simplification. For most motors used in robotics competitions, the electrical time constant $\tau_e = L/R$ is on the order of milliseconds—far faster than any mechanical response. Setting $L \approx 0$ collapses the model to a much more tractable first-order system:

$$\frac{\Omega(s)}{V(s)} = \frac{K_m}{\tau_m s + 1} \tag{28}$$

where the **motor gain** and **mechanical time constant** are:

$$K_m = \frac{K_t}{bR + K_tK_e}, \qquad \tau_m = \frac{JR}{bR + K_tK_e} \tag{29}$$

This first-order model is what you will use for PID tuning, feedforward design, and most practical work.

Demystifying the Time Constant τ

The time constant τ appears everywhere in control theory, yet many students struggle to develop an intuition for it. Let us fix that.

What τ really means. For any first-order system $G(s) = \frac{K}{\tau s+1}$, the step response is:

$$y(t) = K\left(1 - e^{-t/\tau}\right)$$

At $t = \tau$, the output has reached $1 - e^{-1} \approx 63.2\%$ of its final value. This is where the "63% rule" comes from. But why 63%? Because the first-order system's behavior has a beautiful property: *at every instant, the rate of change is proportional to the remaining distance to the target.* At $t = 0$, the system has full "motivation" and moves the fastest. As it approaches the target, it slows down—like a runner who sprints at the start but slows as the finish line approaches. After one τ, it has covered 63.2% of the gap. After another τ, 63.2% of the *remaining* gap. And so on:

Time	% of final value	Practical meaning
1τ	63.2%	Barely past halfway—still rising noticeably
2τ	86.5%	Getting close, but not there yet
3τ	95.0%	Good enough for most applications
4τ	98.2%	Within 2% error band (settling time!)
5τ	99.3%	Essentially at steady state

A physical analogy: filling a leaky bucket. Imagine filling a bucket that has a hole in the bottom. When the bucket is empty, water flows in fast because there is no pressure pushing it back out. As the bucket fills, the water level creates back-pressure that pushes water out of the hole. The higher the level, the more water leaks out, until eventually inflow equals outflow and the level

stops rising. The time it takes to reach 63% of the equilibrium level is τ—and it depends on both the bucket size (capacitance/inertia) and the hole size (resistance/friction).

This is exactly what happens in a motor: voltage drives current into the winding (inflow), but back-EMF and friction resist motion (outflow). The mechanical time constant τ_m tells you how quickly the motor speed "fills up" to its steady-state value.

Why τ connects to everything:

- **Settling time:** $t_s \approx 4\tau$ (for 2% criterion). If your motor has $\tau_m = 50$ ms, settling takes ∼200 ms. Too slow? You need a bigger motor or a feedback controller.
- **Bandwidth:** The -3 dB bandwidth of a first-order system is $\omega_{bw} = 1/\tau$. A motor with $\tau_m = 50$ ms has a bandwidth of 20 rad/s $\approx$ 3.2 Hz. It physically cannot track signals faster than this without a controller.
- **Pole location:** The pole of $G(s) = K/(\tau s + 1)$ is at $s = -1/\tau$. Smaller τ means the pole is farther left—faster dynamics. This is the transfer-function way of saying "a faster motor has a smaller time constant."
- **Sampling rate:** Your controller's sampling period should be at most $\tau/10$ for adequate control. For $\tau_m = 50$ ms, sample at ≤ 5 ms (200 Hz minimum).

DC gain K is the other half of the story. The time constant tells you *how fast* the system responds; the DC gain tells you *how far* it goes. For a motor, K_m gives the steady-state speed per volt. If $K_m = 10$ rad/s/V, then 1 V eventually produces 10 rad/s. If your application requires 100 rad/s, you need at least 10 V. No controller can change the DC gain of the plant—it can only change how fast you get there.

Electrical vs. mechanical time constant. A motor actually has *two* time constants: $\tau_e = L/R$ (electrical, typically ∼1 ms) and $\tau_m = JR/(bR + K_t K_e)$ (mechanical, typically 20–200 ms). Since $\tau_e \ll \tau_m$, the electrical dynamics "settle" before the mechanical dynamics even get started. This is why we can simplify the second-order model to first-order: by the time you notice the motor moving, the current has already reached its steady-state value. In control theory language, the fast electrical pole is so far left in the s-plane that it has negligible effect on the response—it is not a *dominant pole*.

The dominant pole concept. When a system has multiple time constants, the *slowest* one (largest τ, i.e., the pole closest to the imaginary axis) dominates the response. This is why we can often approximate high-order systems by simple first- or second-order models—the fast poles contribute only brief transients that die out before we even notice them. This is not sloppy engineering; it is justified approximation, and it is the reason first-order models work so well in practice.

BLDC Motors and FOC

Brushless DC (BLDC) motors are three-phase machines, but modern ESCs running **Field Oriented Control (FOC)** handle the three-phase complexity internally. FOC uses the Park and Clarke transforms to project the three-phase currents onto a rotating d-q reference frame, producing an equivalent DC brushed motor in the q-axis. From your controller's perspective, the ESC exposes a *virtual DC brushed motor*—you command a voltage (or current) and observe angular velocity, and the same transfer function applies directly. You do not need to think about phase commutation at all.

Understanding FOC: Why It Matters and How It Works

A BLDC motor has three stator windings arranged 120° apart. To spin the rotor, these windings must be energized in a specific sequence that keeps the magnetic field always "pulling" the rotor forward. The challenge is: the required currents are sinusoidal and their phase depends on the rotor's angular position. Controlling three time-varying, position-dependent sinusoids simultaneously is messy.

The key idea of FOC: transform the problem until it looks like a DC motor.

Step 1: Clarke transform ($abc \to \alpha\beta$). The three phase currents i_a, i_b, i_c are projected onto a fixed two-axis coordinate system (α, β). This eliminates one redundant variable (since $i_a + i_b + i_c = 0$) without losing any information. The result: two sinusoidal signals instead of three.

Step 2: Park transform ($\alpha\beta \to dq$). The two-axis fixed frame is rotated to align with the rotor's magnetic axis, using the measured rotor angle θ_e. In this **rotating reference frame**:

- The q-axis current (i_q) produces torque—exactly like the armature current in a DC brushed

motor.

- The d-axis current (i_d) controls the magnetic field strength. For most applications, i_d is set to zero (no need to weaken or strengthen the field).

After the Park transform, the three time-varying sinusoids have become two *constant* values (i_d and i_q) in steady state. Controlling constants is trivial—just use PI controllers.

Step 3: PI control in dq frame. Two simple PI controllers:

- d-axis PI: keeps $i_d = 0$ (or a reference for field weakening at high speeds).
- q-axis PI: tracks the desired torque current $i_q^* = \tau_{\text{desired}}/K_t$.

Step 4: Inverse transforms. The dq voltage commands are transformed back to three-phase voltages (inverse Park → inverse Clarke), which are applied to the motor windings via PWM.

Why FOC matters for your robot:

- **Precise torque control.** Since i_q directly determines torque, FOC enables accurate torque (and therefore current) control—essential for force-controlled applications and smooth low-speed operation.
- **Maximum efficiency.** By keeping $i_d = 0$, all current goes to producing torque. No energy is wasted on useless field components. This means less heat and longer battery life.
- **Smooth at all speeds.** Unlike simpler commutation methods (trapezoidal/six-step), FOC produces smooth sinusoidal currents with no torque ripple. Your gimbal does not "cog" at low speeds.
- **Simplifies your work.** The ESC handles all the FOC math internally at 10–40 kHz. Your outer controller sees a clean current (or voltage) interface—a virtual DC brushed motor. You only need to design the velocity and position loops.

What you need to provide: FOC requires accurate rotor position feedback. Typically a magnetic encoder (e.g., AS5048) or Hall sensors. Without position feedback, the Park transform cannot be computed, and sensorless FOC (which estimates position from back-EMF) is used instead—common on drone ESCs but less precise for robotics.

State-Space Form

For observer design, LQR, or MPC, you need the full state-space representation. Using $\mathbf{x} = \begin{bmatrix} i & \omega \end{bmatrix}^T$ and $u = V$:

$$\dot{\mathbf{x}} = A\mathbf{x} + Bu, \qquad A = \begin{bmatrix} -R/L & -K_e/L \\ K_t/J & -b/J \end{bmatrix}, \quad B = \begin{bmatrix} 1/L \\ 0 \end{bmatrix} \tag{30}$$

With $L \approx 0$ the current state i becomes an algebraic variable and the state reduces to $\mathbf{x} = [\,\omega\,]$, recovering the first-order model. The full two-state form is what you feed into an LQR or Kalman filter when current sensing is available.

Practical Tip: How to Get Motor Parameters

You rarely have a datasheet with J, b, L. Measure them yourself:

- **Stall test** — apply a known low voltage V_s, hold the shaft still, measure stall current I_s. Then $R = V_s/I_s$.
- **Free-run test** — no load, apply rated voltage V_0, wait for steady state, measure ω_{free}. Then $K_e = (V_0 - I_{\text{free}}R)\,/\,\omega_{\text{free}}$ (SI units: V·s/rad). In SI units, $K_t = K_e$ exactly.
- **Step response** — command a voltage step, log $\omega(t)$. The time to reach 63% of the final value is τ_m.
- **Inertia** — rearrange $\tau_m = JR/(bR + K_tK_e)$ to solve for J, or use a *swing test*: attach a known mass on a known arm, measure angular acceleration α under a known torque, and compute $J = \tau/\alpha$.

Shortcut when using an FOC ESC with a current controller: most modern FOC ESCs close an inner current loop at very high bandwidth. If that loop is active, the current i tracks your command almost instantly. Your effective plant simplifies to *pure mechanical dynamics*:

$$\frac{\Omega(s)}{I_{\text{ref}}(s)} = \frac{K_t}{Js + b}$$

You only need K_t, J, and b—and b is often negligible. This is the plant you will tune your outer velocity loop against.

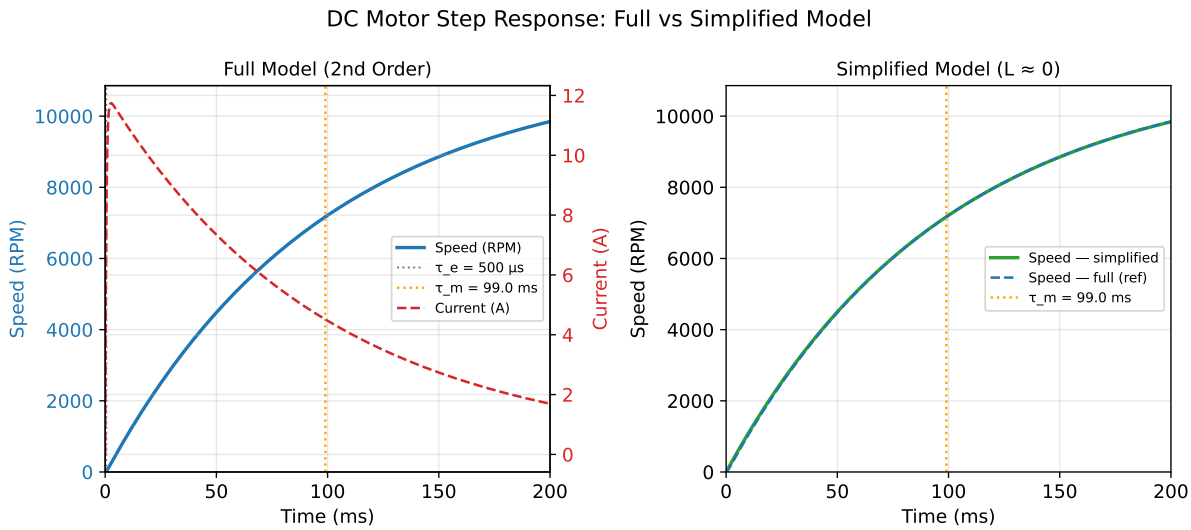

Figure 5: DC brushed motor step response: voltage to angular velocity. Left: full second-order model showing the fast electrical transient ($\tau_e \approx 1$ ms) followed by the slow mechanical rise ($\tau_m \approx 50$ ms). Right: first-order approximation ($L \approx 0$) matches the mechanical response almost perfectly — the electrical pole is too fast to matter.

4 Classical Control Theory

This is the section you have been waiting for. Classical control theory is where we learn to make systems do what we want. And the undisputed champion of classical control—the tool you will use on 95% of your competition robots—is the PID controller.

But before we dive into PID, we need to understand *why* feedback control works in the first place.

4.1 The Concept of Feedback Control

Open-loop control is like cooking without tasting. You follow the recipe exactly, hope for the best, and serve. If the oven temperature is off by 10 degrees, your cake is ruined.

Closed-loop (feedback) control is like cooking while constantly tasting. You taste, adjust the seasoning, taste again, adjust again. Even if the oven is off, you compensate in real time.

Mathematically, a basic feedback loop looks like this:

1. Compute the **error**: $e(t) = r(t) - y(t)$, where r is the desired output (setpoint) and y is the actual output.
2. Feed the error into a **controller** $C(s)$, which produces a control input $u(t)$.
3. The control input drives the **plant** $G(s)$, which produces the output $y(t)$.
4. Measure $y(t)$ and go back to step 1.

The closed-loop transfer function from setpoint R to output Y is:

$$\frac{Y(s)}{R(s)} = \frac{C(s)G(s)}{1 + C(s)G(s)} \tag{31}$$

This is the most important equation in classical control. Stare at it. Love it. Memorize it.

Dissecting the closed-loop transfer function:

Let $L(s) = C(s)G(s)$ be the **loop transfer function** (also called open-loop transfer function). Then:

- **Complementary sensitivity** (setpoint tracking): $T(s) = \frac{L(s)}{1+L(s)}$. This is what we wrote above — it tells you how well the output follows the reference. Ideally $T(s) \approx 1$ at low frequencies (good tracking) and $T(s) \approx 0$ at high frequencies (noise rejection).
- **Sensitivity function**: $S(s) = \frac{1}{1+L(s)}$. This tells you how much a disturbance at the output affects the output. Small S = good disturbance rejection. Note that $T(s) + S(s) = 1$ always — you *cannot* have perfect tracking and perfect disturbance rejection at the same frequency. This is a fundamental limitation of feedback.
- **Input sensitivity**: $C(s) \cdot S(s) = \frac{C(s)}{1+L(s)}$. This tells you how much control effort is needed. If this is large at high frequencies, your actuator will work overtime amplifying noise.

The key insight: At frequencies where $|L(j\omega)| \gg 1$ (high loop gain), $T \approx 1$ and $S \approx 0$ — feedback works well. At frequencies where $|L(j\omega)| \ll 1$ (low loop gain), $T \approx 0$ and $S \approx 1$ — feedback does nothing. The **bandwidth** is the frequency where $|L(j\omega)| = 1$ (the crossover), and it marks the boundary between "feedback is helping" and "feedback is not helping."

Why feedback is powerful:

- **Reduces sensitivity to model uncertainty.** Even if $G(s)$ is not perfectly known, feedback compensates.
- **Rejects disturbances.** External forces (friction, wind, collisions) are automatically corrected.
- **Improves dynamic response.** We can make the system faster or more damped by choosing $C(s)$ appropriately.

The catch: Feedback can also *destabilize* a system. If your controller is too aggressive, the correction overshoots, the error reverses, the correction overshoots again, and you get oscillation or complete instability. This is why we need tools to analyze stability.

Case Study: The Shower Temperature Problem

You have experienced feedback control instability in your own bathroom. Here is how:

You step into the shower. The water is cold. You crank the hot knob (large K_p). Nothing happens for 2 seconds (the pipe delay—this is **transport lag**, a real control theory concept). Impatient, you crank it more. Suddenly, scalding water hits you. You jerk the knob to cold. Two seconds later, ice water. You oscillate between scalding and freezing for 30 seconds until you give up.

What went wrong (in control theory terms):

- The **plant** is the water heater + pipes. It has a significant **time delay** ($\sim$2 seconds).
- You (the controller) used a very high K_p—aggressive corrections.
- Time delay adds phase lag. At the frequency where you are adjusting the knob, the total phase lag exceeds 180°. Your phase margin is negative. The system is **unstable**.

The fix: Turn the knob slowly and **wait**. This is equivalent to reducing K_p so that your bandwidth is well below $1/(2 \times \text{delay})$. You are trading speed for stability—exactly the same trade-off you will face with every control loop on your robot.

Lesson: Systems with time delay are fundamentally harder to control. If your robot has communication latency (e.g., CAN bus delay, vision processing delay), you *must* account for it by reducing controller aggressiveness or using a Smith predictor (an advanced technique that compensates for known delays).

4.2 System Performance Metrics

When we design a controller, we need to define what "good" means. The standard metrics for a step response (sudden change in setpoint) are:

- **Rise time** (t_r): Time to go from 10% to 90% of the final value. Faster is generally better.
- **Overshoot** (M_p): How much the output exceeds the setpoint, expressed as a percentage. Less is generally better.
- **Settling time** (t_s): Time until the output stays within ±2% (or 5%) of the final value.
- **Steady-state error** (e_{ss}): The remaining error after the system has settled. Ideally zero.

These metrics often conflict: faster rise time usually means more overshoot. Control design is about finding the right trade-off for your application. A gimbal tracking a target needs low overshoot (no jittering past the target). A chassis accelerating to top speed can tolerate some overshoot if it means getting there faster.

4.2.1 Stability Margins

Even if a system is stable, we want to know *how stable* it is—how close it is to becoming unstable. This is measured by two numbers:

Gain margin (GM): How much you can increase the loop gain before the system goes unstable. Measured in dB. A gain margin of 6 dB means you can double the gain before instability. Typical target: > 6 dB.

Phase margin (PM): How much additional phase lag the system can tolerate before going unstable. Measured in degrees. A phase margin of 45° means the system can tolerate an additional 45° of lag. Typical target: 30°–60°.

You can read both margins from a **Bode plot** of the open-loop transfer function $C(s)G(s)$.

What is a Bode plot? A Bode plot is two graphs stacked vertically, both with frequency on the horizontal axis (log scale):

- **Top: Magnitude plot.** Shows the gain $|G(j\omega)|$ in decibels (dB) vs. frequency. 0 dB means gain $= 1$ (output equals input). +20 dB means gain $= 10$. -20 dB means gain $= 0.1$. The conversion: $\text{dB} = 20 \log_{10}(|G|)$.
- **Bottom: Phase plot.** Shows the phase angle $\angle G(j\omega)$ in degrees vs. frequency. A phase of $-90°$ means the output lags the input by a quarter cycle.

How to read stability margins from the Bode plot:

- **Phase crossover frequency** (ω_{pc}): where the phase crosses $-180°$. At this frequency, read the magnitude — the distance from the magnitude curve to 0 dB is your **gain margin**. If the magnitude is below 0 dB at ω_{pc}, the system is stable and the gain margin is positive.
- **Gain crossover frequency** (ω_{gc}): where the magnitude crosses 0 dB. At this frequency, read the phase — the distance from the phase curve to $-180°$ is your **phase margin**. If the phase is above $-180°$ at ω_{gc}, the system is stable and the phase margin is positive.

Bode plot sketching shortcuts: For a transfer function $G(s)$:

- Each pole at $s = -a$ contributes -20 dB/decade rolloff starting at $\omega = a$, and $-45°$/decade phase change centered at $\omega = a$.
- Each zero at $s = -a$ contributes $+20$ dB/decade and $+45°$/decade (opposite of a pole).
- An integrator ($1/s$) contributes -20 dB/decade at all frequencies and a constant $-90°$ phase.
- The DC gain sets the vertical offset of the magnitude plot.

With these rules, you can sketch approximate Bode plots by hand in under a minute—invaluable during debugging when you need a quick sanity check.

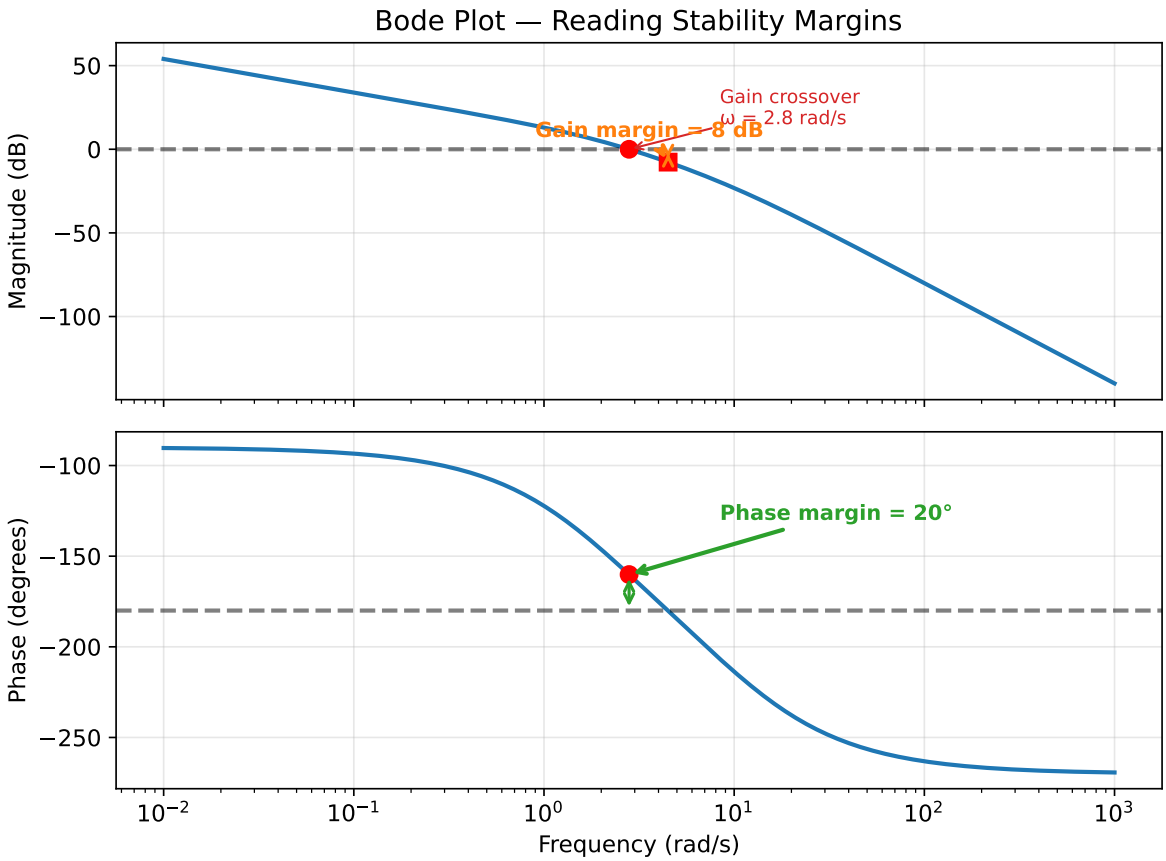

Figure 6: Bode plot of an open-loop system $L(s) = \frac{100}{s(s+2)(s+10)}$. The gain margin (orange) and phase margin (green) are read directly from the plot. Both are positive, so the system is stable.

Rule of thumb: If your system is oscillating, your phase margin is too small. If your system is sluggish, your gain margin is too large (you are being too conservative). A phase margin of around 45° usually gives a good balance.

4.3 Controller Design via Transfer Functions

The game of classical controller design is: given a plant $G(s)$, choose a controller $C(s)$ such that the closed-loop system $\frac{C(s)G(s)}{1+C(s)G(s)}$ meets your performance specifications.

There are many approaches (root locus, frequency response shaping, pole placement), but for competition robotics, the most practical approach is:

1. Start with a PID controller (next section).
2. Tune it using systematic methods.
3. If PID is not enough, add feedforward or switch to modern methods (Section 6).

This might sound unsophisticated, but here is the truth: PID controls an estimated 95% of all industrial control loops in the world. It is simple, robust, and effective. Do not overcomplicate things.

4.4 PID Controller

The PID controller computes the control signal as the sum of three terms based on the error $e(t)$:

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau)d\tau + K_d \frac{de(t)}{dt} \tag{32}$$

Each term has a distinct role:

- **P (Proportional):** "The error is big, so push hard. The error is small, so push gently." It is the most intuitive term—a simple spring-like response.
- **I (Integral):** "The error has been nonzero for a while, so there must be a persistent disturbance. Push harder until it is gone." It eliminates steady-state error but can cause overshoot and windup.
- **D (Derivative):** "The error is changing fast, so the system has momentum. Apply the brakes." It adds damping and reduces overshoot but amplifies noise.

An analogy: Imagine you are driving toward a stop sign.

- P says: "The stop sign is 100 meters away, press the gas." Then: "It is 10 meters away, ease off."
- D says: "You are approaching fast—start braking NOW, not when you are close."
- I says: "You stopped 1 meter past the sign. Roll back slowly until you are exactly there."

4.4.1 PID Controller – Continuous Form

The transfer function of a PID controller is:

$$C(s) = K_p + \frac{K_i}{s} + K_d s = K_p \left(1 + \frac{1}{T_i s} + T_d s\right) \tag{33}$$

where $T_i = K_p/K_i$ is the integral time and $T_d = K_d/K_p$ is the derivative time.
Understanding PID in the frequency domain:
The PID controller's Bode plot reveals why it works:

- **At very low frequencies** ($\omega \to 0$): The K_i/s term dominates. The gain goes to infinity ($+\infty$ dB), which is why I eliminates steady-state error — infinite gain at DC means zero error at DC.
- **At mid frequencies**: K_p dominates. The gain is approximately $20\log_{10}(K_p)$ dB. This is the frequency range where your controller does most of its work.
- **At high frequencies** ($\omega \to \infty$): $K_d s$ dominates. The gain increases with frequency at +20 dB/decade, which is why D amplifies noise — it has increasing gain at the very frequencies where noise lives.

The PID has **two break frequencies**: $\omega_1 = 1/T_i$ (where I hands off to P) and $\omega_2 = 1/T_d$ (where P hands off to D). Between them, the controller behaves like pure proportional control. The Bode magnitude plot looks like a "V" shape (or bathtub) on a log scale: high gain at low frequencies (I), flat in the middle (P), rising at high frequencies (D).

Why the alternative form matters: The form $K_p(1 + \frac{1}{T_i s} + T_d s)$ makes the physical meaning clearer: T_i tells you "how long the integral looks back" and T_d tells you "how far ahead the derivative looks." Larger T_i means slower integral action. Larger T_d means more anticipation (but also more noise sensitivity).

Effect of each gain:

Increase	Rise time	Overshoot	Settling time	SS error
K_p	Decrease	Increase	Small change	Decrease
K_i	Decrease	Increase	Increase	Eliminate
K_d	Small change	Decrease	Decrease	No effect

Practical starting point: Start with P-only control. If there is steady-state error, add I. If there is too much overshoot or oscillation, add D. Do not start with all three at once—you will not know which knob is doing what.

Case Study: Tuning a Gimbal Yaw Motor — Watching P, I, D in Action

Let us walk through tuning a RoboMaster gimbal yaw axis step by step. The motor is a GM6020, controlled via CAN bus, and we want it to track a target angle. The setpoint is a 90° step command.

Step 1: P-only ($K_p = 5.0$, $K_i = 0$, $K_d = 0$)

The gimbal swings toward the target quickly but overshoots by about 15°, oscillates 3–4 times, and settles with a **2° steady-state error**. Why the error? Because as the gimbal approaches the target, the error shrinks, and so does the P output. At some point, the P output equals the friction torque, and the gimbal stops *before* reaching the target. P alone cannot eliminate error caused by constant disturbances (like friction or gravity).

Step 2: Add I ($K_p = 5.0$, $K_i = 0.5$, $K_d = 0$)

The steady-state error vanishes! The integral term slowly accumulates the small residual error and pushes the motor until the error is exactly zero. But now the overshoot is worse (∼20°), and there is a slow oscillation that takes 2 seconds to die out. The integral is "remembering" past errors and adding unnecessary force. This is the classic I-term trade-off: it fixes DC error but worsens transient response.

Step 3: Add D ($K_p = 5.0$, $K_i = 0.5$, $K_d = 2.0$)

The overshoot drops dramatically to about 5°, and the oscillation is almost gone. The derivative term senses the rapid approach and applies a braking torque *before* the gimbal reaches the target. It is like an experienced driver who starts braking well before the intersection, not at the last second.

Step 4: Fine-tune ($K_p = 6.0$, $K_i = 0.3$, $K_d = 3.0$)

We increase K_p slightly for a faster rise time, reduce K_i (we only need it for static friction, not aggressive error correction), and increase K_d for more damping. The result: fast, smooth tracking with less than 3° overshoot and zero steady-state error. The gimbal locks onto the target in under 0.3 seconds.

What happens if you crank each parameter too high?

- K_p too high: The gimbal oscillates violently—it overshoots so hard that the correction overshoots in the other direction. This is the onset of instability.
- K_i too high: Slow, sloshy oscillation. The integral "remembers" too aggressively and keeps overcorrecting. The gimbal wobbles like jelly.
- K_d too high: The motor buzzes and vibrates. The derivative amplifies sensor noise, causing rapid back-and-forth torque commands. Your motor sounds like an angry bee.

The lesson: Each parameter has a "sweet spot." Understanding the *physical mechanism* of each term (spring, memory, damper) helps you diagnose problems instantly instead of blindly twiddling knobs.

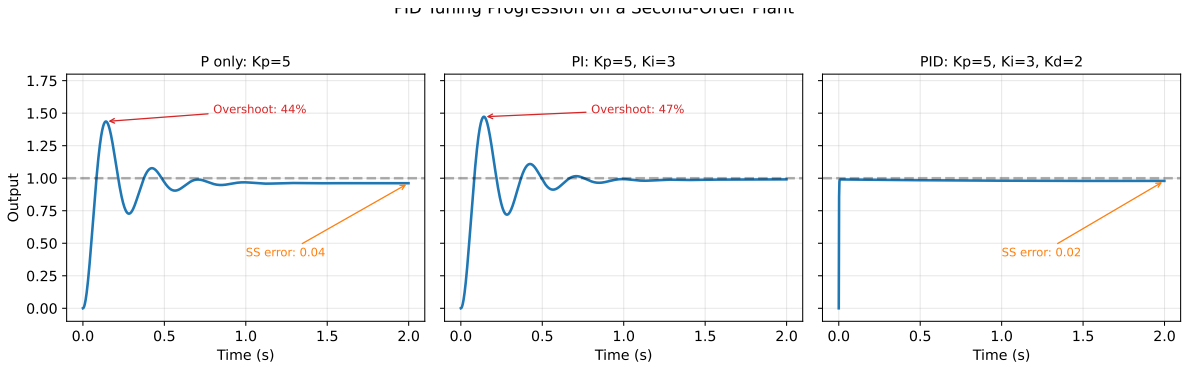

Figure 7: PID tuning progression on a second-order plant. P-only has steady-state error; PI eliminates it but overshoots more; PID damps the overshoot while maintaining zero steady-state error.

4.4.2 PID Controller – Discrete Form

Your microcontroller cannot compute continuous integrals and derivatives. It works with discrete samples at a fixed time step T_s. The discrete PID is:

$$u[n] = K_p\, e[n] + K_i\, T_s \sum_{k=0}^{n} e[k] + K_d \frac{e[n] - e[n-1]}{T_s} \tag{34}$$

Or in a more implementation-friendly **incremental (velocity) form**:

$$\Delta u[n] = K_p(e[n] - e[n-1]) + K_i\, T_s\, e[n] + K_d \frac{e[n] - 2e[n-1] + e[n-2]}{T_s} \tag{35}$$

$$u[n] = u[n-1] + \Delta u[n] \tag{36}$$

The incremental form has a practical advantage: if your controller resets or glitches, it does not produce a sudden large output—it only adjusts by a small increment. This is safer for your hardware.

Pseudocode for a basic discrete PID:

```
error = setpoint - measurement
integral += error * dt
derivative = (error - prev_error) / dt
output = Kp * error + Ki * integral + Kd * derivative
prev_error = error
```

Warning: This naive implementation has three problems that will bite you in practice: integral windup, derivative kick, and noise amplification. Read the next section.

4.4.3 Improved PID Controllers

The textbook PID has several well-known problems. Here are the fixes that every competition robot should use.

1. Integral windup and anti-windup

The problem: When the actuator saturates (motor at max voltage), the integral term keeps accumulating error, growing without bound. When the error finally reverses, the huge integral takes a long time to "unwind," causing massive overshoot.

The fix (clamping):

```
integral += error * dt
integral = clamp(integral, -max_integral, max_integral)
```

Better fix (back-calculation): When the output is saturated, reduce the integral term proportionally:

```
output_raw = Kp * error + Ki * integral + Kd * derivative
output = clamp(output_raw, -max_output, max_output)
integral += (error + (output - output_raw) / Ki) * dt
```

2. Derivative kick

The problem: When the setpoint changes suddenly (step command), the error $e = r - y$ jumps instantaneously. The derivative term sees this as an infinite rate of change and produces a huge spike ("kick") in the output.

The fix: Differentiate the *measurement* instead of the error:

```
derivative = -(measurement - prev_measurement) / dt
```

The measurement changes smoothly (your motor cannot teleport), so there is no kick.

3. Derivative noise amplification

The problem: Differentiation amplifies high-frequency noise. If your measurement is noisy, the derivative term will produce wildly fluctuating outputs.

The fix: Low-pass filter the derivative:

$$D(s) = \frac{K_d s}{1 + \frac{K_d}{N K_p} s} \tag{37}$$

In code, this is a first-order low-pass filter on the derivative term, with N (typically 8–20) controlling the filter cutoff.

4. Setpoint weighting

Sometimes you want the P and D terms to respond differently to setpoint changes vs. disturbances. Setpoint weighting introduces parameters b and c:

$$u = K_p(b \cdot r - y) + K_i \int (r - y)\, dt + K_d \frac{d(c \cdot r - y)}{dt} \tag{38}$$

Setting $b = 1, c = 0$ (derivative on measurement only) is the most common configuration.

4.5 Advanced PID Improvements

The four fixes above are the *minimum* every competition robot needs. But there are deeper structural improvements that can dramatically change performance. Each is analyzed here in the frequency domain and demonstrated with simulation.

4.5.1 Integral Separation

The problem: When the error is large (e.g., right after a step command), the integral term accumulates aggressively. This "pre-charged" integral causes overshoot once the system approaches the setpoint — the integral has "remembered" too much history from the high-error phase.

The fix: Only activate the integral term when the error is smaller than a threshold ε:

$$u_I = \begin{cases} K_i \int e\, dt & \text{if } |e| < \varepsilon \\ 0 & \text{if } |e| \geq \varepsilon \end{cases} \tag{39}$$

Frequency domain interpretation: Integral separation effectively makes the controller **nonlinear** — it behaves like a PD controller (no integral) for large errors and switches to a full PID for small errors. In the frequency domain, this means:

- During large transients: the open-loop gain at DC is finite (PD only), so there is no infinite low-frequency gain. This prevents the integral from "winding up" during the transient.
- Near steady state: the integral activates, providing infinite DC gain and zero steady-state error — exactly what we need.

The controller essentially has **two different Bode plots** depending on the error magnitude: one without the K_i/s pole at low frequencies (PD-like) and one with it (PID). The threshold ε controls when the "switch" happens.

Code:

```
if (fabs(error) < epsilon) {
    integral += Ki * error * dt;
}
// else: integral stays frozen (neither grows nor resets)
```

Choosing ε: Set it to the approximate steady-state error of the PD-only controller. This way, the integral only activates when PD has done most of the work and just needs the final nudge.

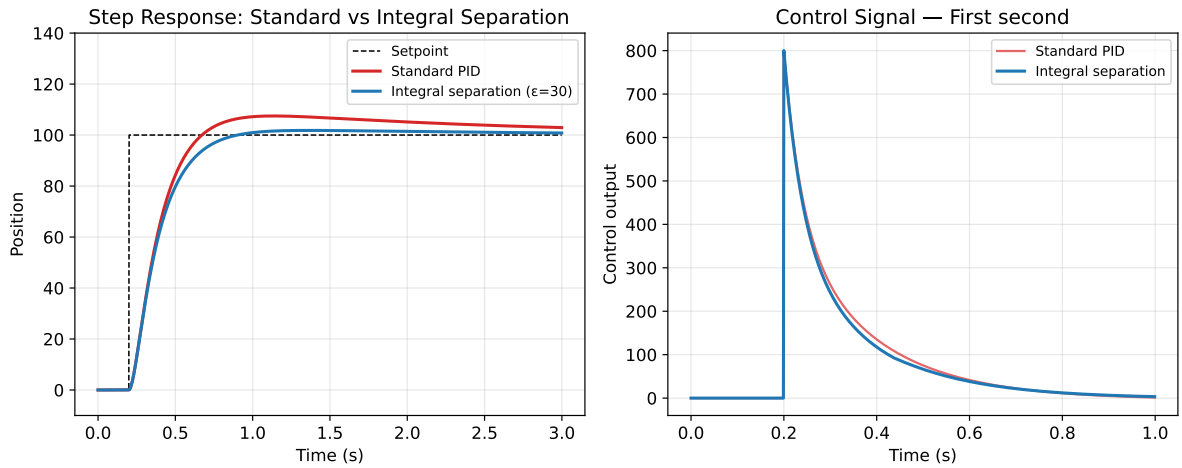

Figure 8: Left: integral separation (blue) reduces overshoot compared to standard PID (red), because the integral does not accumulate during the large initial transient. Right: the control signals during the first second — note how the separated version has a smaller peak.

4.5.2 Derivative Placement: PID vs PI-D vs I-PD

The standard PID applies all three terms to the error $e = r - y$. But there are **three structural variants** that change which signals P and D act on:

Structure	P acts on	I acts on	D acts on
PID	error $(r-y)$	error $(r-y)$	error $(r-y)$
PI-D	error $(r-y)$	error $(r-y)$	measurement $(-y)$
I-PD	measurement $(-y)$	error $(r-y)$	measurement $(-y)$

Frequency domain analysis: All three structures have the *same* closed-loop disturbance rejection transfer function (from disturbance to output). They differ only in the *setpoint-to-output* transfer function:

- **PID:** $\frac{Y}{R} = \frac{(K_d s^2 + K_p s + K_i)G(s)}{s + (K_d s^2 + K_p s + K_i)G(s)}$ — the numerator has a $K_d s^2$ term, which creates a "derivative kick" in the step response (numerator zeros amplify the step's high-frequency content).
- **PI-D:** $\frac{Y}{R} = \frac{(K_p s + K_i)G(s)}{s + (K_d s^2 + K_p s + K_i)G(s)}$ — the $K_d s^2$ is removed from the setpoint path, eliminating the derivative kick. The denominator (stability, disturbance rejection) is unchanged.
- **I-PD:** $\frac{Y}{R} = \frac{K_i \cdot G(s)}{s + (K_d s^2 + K_p s + K_i)G(s)}$ — both $K_d s^2$ and $K_p s$ are removed from the setpoint path. The step response is the smoothest (no overshoot from P or D) but also the slowest.

The key insight: These structures let you independently tune how the system responds to *setpoint changes* (via the numerator) vs. how it rejects *disturbances* (via the denominator). Same denominator = same stability, same disturbance rejection. Different numerator = different setpoint tracking behavior.

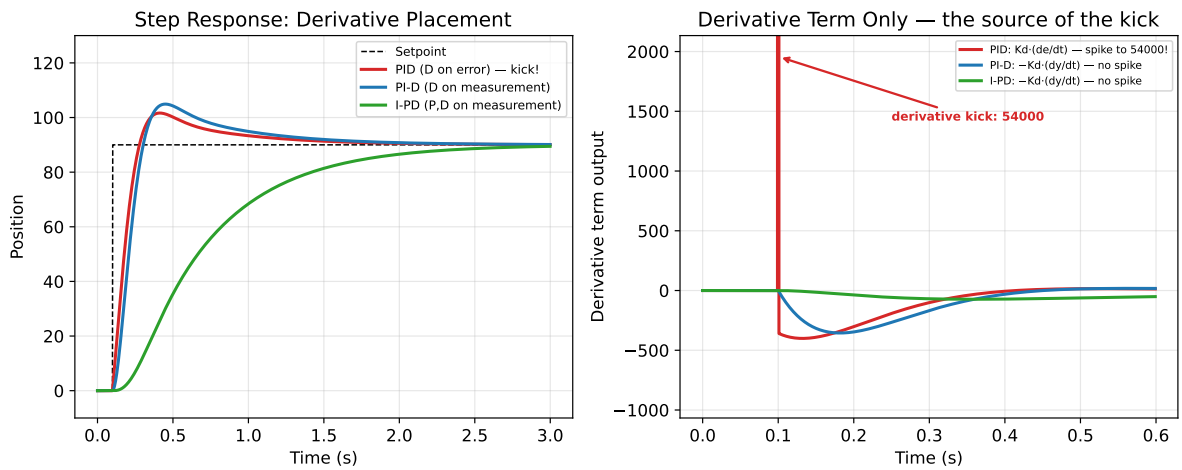

Figure 9: Left: PID (red) shows a derivative kick spike and overshoot. PI-D (blue) eliminates the kick. I-PD (green) is the smoothest but slowest. Right: the control signal reveals the violent kick in standard PID — this spike is what strips gears.

4.5.3 Filtered PID: Frequency Domain Proof

The ideal PID controller $C(s) = K_p + K_i/s + K_d s$ has a fundamental problem: the $K_d s$ term has **unbounded gain at high frequencies**. On a Bode plot, the magnitude of $K_d s$ rises at +20 dB/decade forever. This means:

1. Sensor noise (which lives at high frequencies) gets amplified infinitely.
2. The controller demands infinitely fast actuator response, which is physically impossible.
3. In discrete time, the derivative term aliases high-frequency noise into the control band.

The fix — filtered derivative:

$$C_{\text{filtered}}(s) = K_p + \frac{K_i}{s} + \frac{K_d N s}{s + N} \tag{40}$$

The term $\frac{Ns}{s+N}$ is a first-order high-pass filter with cutoff at $\omega = N$ rad/s. Below N, it behaves like a pure derivative ($\approx s$). Above N, it saturates to a constant gain N. On the Bode plot:

- Below $\omega = N$: the filtered PID is identical to the ideal PID. Same performance.
- Above $\omega = N$: the magnitude **flattens** instead of rising forever. Maximum derivative gain is $K_d \cdot N$. Noise amplification is bounded.

Choosing N: Typical values are $N = 8$–20. The rule: N should be large enough that the derivative still works effectively at the controller bandwidth, but small enough to attenuate noise. Specifically: $N > 5 \times \omega_{\text{bandwidth}}$ (so the filter does not eat into useful derivative action) and $N < \omega_{\text{noise}}/3$ (so noise is attenuated).

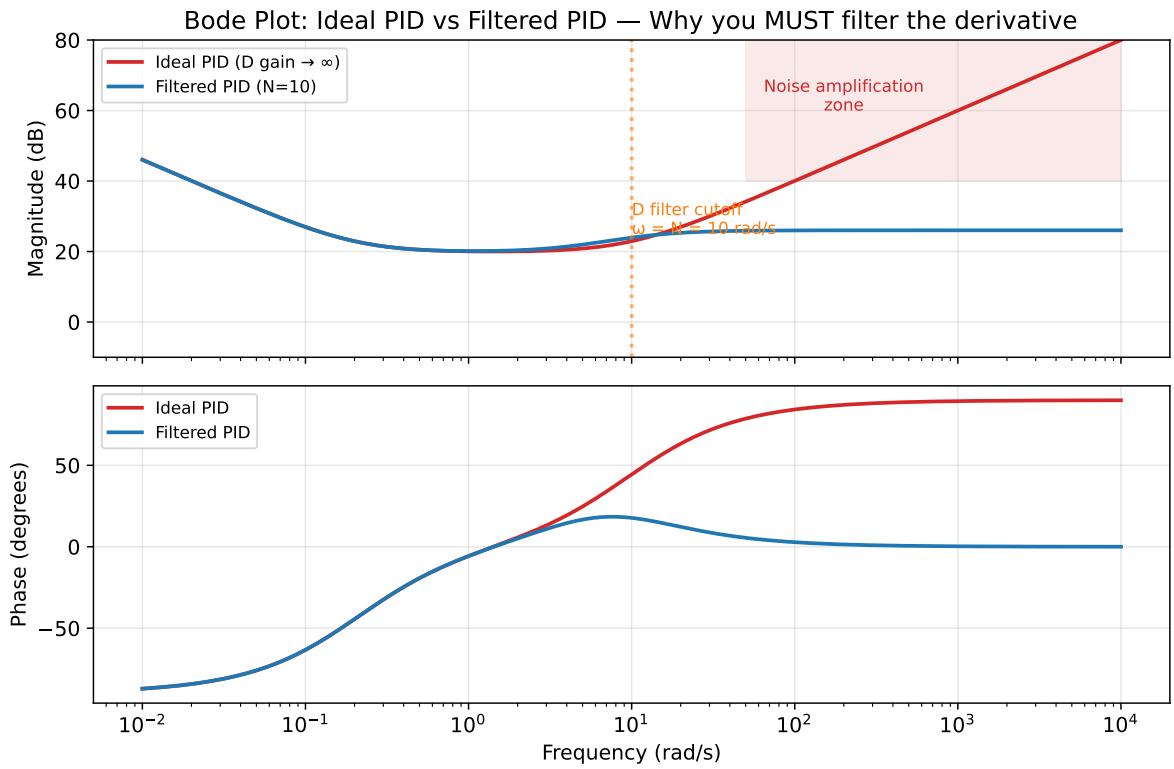

Figure 10: Bode plot: ideal PID (red) has gain rising forever at high frequencies — it will amplify every bit of sensor noise. Filtered PID (blue) flattens above $\omega = N$, bounding the noise amplification. Below the filter cutoff, both are identical.

Output-stage LPF: An alternative approach is to low-pass filter the *entire* PID output before sending it to the actuator:

$$U(s) = \frac{\omega_f}{s + \omega_f} \cdot C(s) \cdot E(s) \tag{41}$$

This is simpler to implement (one LPF after the PID) but it also filters the P and I terms, which can slow down the response. The derivative-only filter is preferred because it targets the problem precisely.

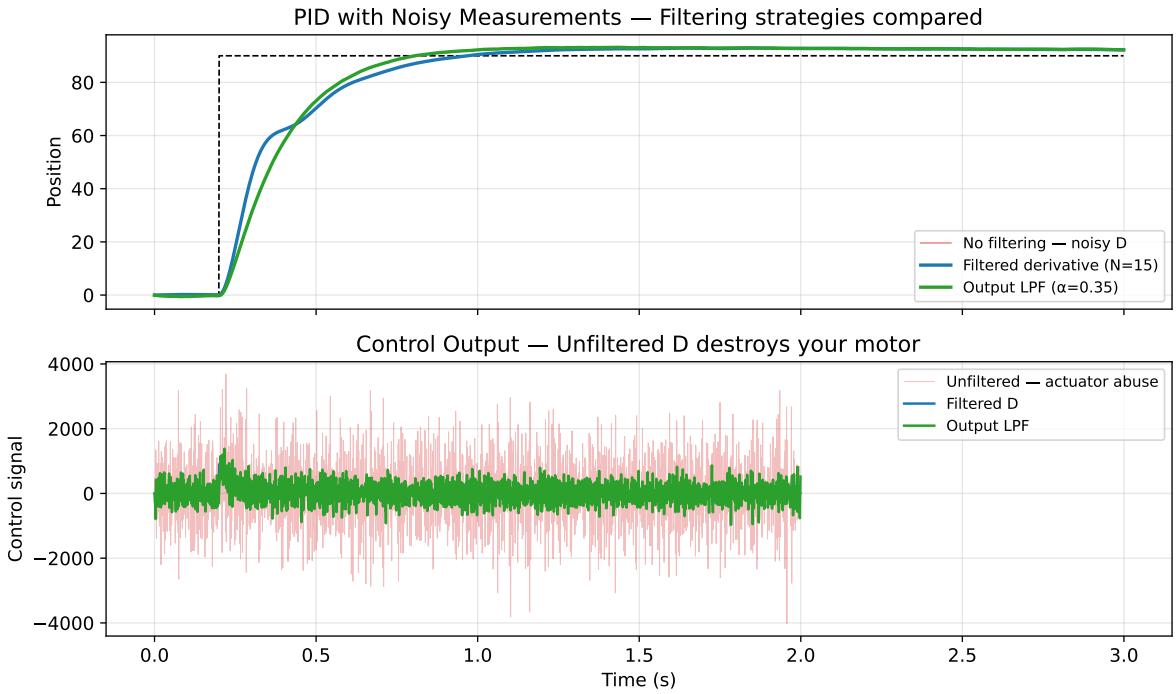

Figure 11: Top: position response with noisy measurements. Unfiltered PID (red) oscillates due to noise-amplified derivative. Filtered derivative (blue) and output LPF (green) both smooth the response. Bottom: the unfiltered control signal is violent — it would destroy a real motor.

4.5.4 Two-Degree-of-Freedom PID (2-DOF)

Standard PID has **one degree of freedom**: you choose K_p, K_i, K_d to meet your specifications. But there is a fundamental conflict: gains that give good *setpoint tracking* (fast, low overshoot for step commands) are different from gains that give good *disturbance rejection* (fast return after a bump).

The 2-DOF PID resolves this by adding setpoint weighting parameters b (on P) and c (on D):

$$u = K_p(b \cdot r - y) + K_i \int (r - y)\, dt + K_d \frac{d(c \cdot r - y)}{dt} \tag{42}$$

Frequency domain analysis: The closed-loop transfer functions become:

$$\frac{Y}{R} = \frac{(bK_p + cK_d s + K_i/s) \cdot G(s)}{1 + C(s)G(s)} \quad \text{(setpoint tracking — tuned by } b,\ c\text{)} \tag{43}$$

$$\frac{Y}{D} = \frac{G(s)}{1 + C(s)G(s)} \quad \text{(disturbance rejection — independent of } b,\ c\text{!)} \tag{44}$$

The magic: b and c *only* affect the setpoint transfer function. The disturbance rejection is controlled entirely by K_p, K_i, K_d. You can now:

1. Tune K_p, K_i, K_d for aggressive disturbance rejection.
2. Separately reduce b (typically 0.5–0.8) to soften the setpoint response and reduce overshoot.
3. Set $c = 0$ to eliminate derivative kick (this is the PI-D structure).

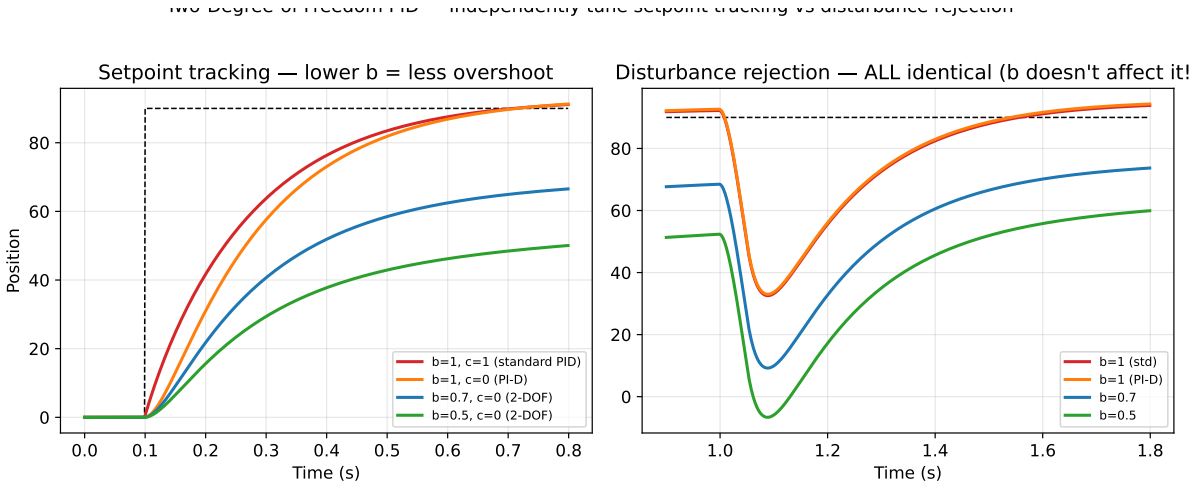

Figure 12: Left: different b values control setpoint overshoot independently. Right: disturbance rejection at $t = 1.0$s is **identical** for all b values — proving that 2-DOF truly decouples the two objectives.

4.5.5 Fuzzy PID

Motivation. A standard PID controller uses fixed gains K_p, K_i, K_d. But the *optimal* gains depend on the operating conditions: when the error is large, an aggressive response is needed; when the error is small, fine control is more appropriate. Fuzzy PID adapts K_p, K_i, K_d in real time based on the current error e and its rate of change $\dot{e}$.

How it works.

1. **Fuzzification.** Define fuzzy sets for the error e and its derivative $\dot{e}$. A common choice uses five linguistic levels: NB (Negative Big), NS (Negative Small), ZO (Zero), PS (Positive Small), PB (Positive Big). Membership functions (typically triangular or trapezoidal) map the crisp values of e and $\dot{e}$ into degrees of membership in each fuzzy set.

2. **Rule table.** A rule table (e.g., 5×5 or 7×7) maps every combination of $(e, \dot{e})$ to adjustments $(\Delta K_p, \Delta K_i, \Delta K_d)$. Rules are written in plain language, for example:

   - "If e is PB and $\dot{e}$ is PB $\Rightarrow$ increase K_p strongly, decrease K_i (to avoid windup), increase K_d."
   - "If e is ZO and $\dot{e}$ is NS $\Rightarrow$ keep K_p moderate, enable K_i for steady-state accuracy, reduce K_d."

3. **Defuzzification.** The fired rules are aggregated (commonly by centre-of-gravity) to produce crisp values of ΔK_p, ΔK_i, ΔK_d.

4. **Gain update.** The actual PID gains are:

$$K_p = K_{p0} + \Delta K_p, \qquad K_i = K_{i0} + \Delta K_i, \qquad K_d = K_{d0} + \Delta K_d$$

   where K_{p0}, K_{i0}, K_{d0} are the baseline gains.

Example rule table for ΔK_p (simplified 3×3).

$e \setminus \dot{e}$	N	Z	P
N	PB	PS	ZO
Z	PS	ZO	NS
P	ZO	NS	NB

Reading: when the error is Negative and getting more negative ($\dot{e} = \text{N}$), increase K_p strongly (PB) to drive the system back; when both are near Zero, no adjustment is needed (ZO).

Practical assessment.

- **Advantage:** Fuzzy PID adapts to changing conditions—load changes, nonlinear operating regions—without requiring an explicit system model.

- **Disadvantage:** The rule table design is largely empirical. In effect, you trade "tuning 3 gains" for "tuning a table of rules," which can be harder to validate rigorously.
- **When to use:** In competition robotics, a well-tuned fixed PID or 2-DOF PID usually suffices. Fuzzy PID is worth considering when the system operates across very different regimes (e.g., a chassis that must handle both empty and fully loaded conditions).
- **Compared to gain scheduling:** Gain scheduling manually switches between pre-tuned PID parameter sets at discrete operating points, which can cause transients at the switching boundaries. Fuzzy PID provides smoother transitions because the membership functions naturally interpolate between rules.

4.6 PID Tuning Methods

You have designed a PID controller. Now comes the eternal question: what values of K_p, K_i, K_d should you use?

4.6.1 Empirical Tuning

The most common method in competitions, because it requires no math and works surprisingly well.

Ziegler-Nichols ultimate gain method:

1. Set $K_i = 0$ and $K_d = 0$.
2. Gradually increase K_p until the system oscillates with constant amplitude. This is the **ultimate gain** K_u, and the oscillation period is T_u.
3. Use the following table to set your PID gains:

Controller	K_p	K_i	K_d
P	$0.5K_u$	—	—
PI	$0.45K_u$	$0.54K_u/T_u$	—
PID	$0.6K_u$	$1.2K_u/T_u$	$0.075K_uT_u$

Warning: Ziegler-Nichols gives aggressive tuning with about 25% overshoot. For many competition applications, you will want to reduce K_p and K_i from these values.

Manual tuning (the practical way):

1. Start with K_p only. Increase until the response is fast but slightly oscillatory.
2. Add K_d to damp the oscillation. Increase until the oscillation is gone.
3. If there is steady-state error, add a small K_i. Increase slowly—too much causes slow oscillation.
4. Fine-tune. Test with different setpoints and disturbances.

Competition tip: Always tune with the *actual load* on the robot, not with the motor running freely. A gimbal with a camera mounted behaves very differently from a bare motor on a test bench.

4.6.2 Tuning with MATLAB Simulink

If you have access to MATLAB, you can model your system and use the **PID Tuner** app or **Simulink Control Design** to find optimal gains automatically.

Basic workflow:

1. Create a Simulink model of your plant (motor + load).
2. Add a PID Controller block.
3. Use the "Tune" button in the PID block to open the PID Tuner.
4. Adjust the response time and robustness sliders.
5. Export the gains to your embedded code.

This is excellent for getting a starting point before fine-tuning on hardware. The key challenge is getting an accurate plant model—if the model is wrong, the tuning will be wrong.

4.7 System Identification: Figuring Out What Your Plant Actually Does

So far, we have assumed that you *know* your plant model — the transfer function $G(s)$ or the state-space matrices A, B, C, D. But where do these come from? You have two options:

1. **First-principles modeling:** Write down the physics equations (Newton's laws, Kirchhoff's laws, etc.) and derive the model analytically. This is what we did for the DC brushed motor in Section 3. It works well when the physics is simple and well-understood.

2. **System identification (sysid):** Apply known inputs to the real system, measure the outputs, and *fit* a mathematical model to the input-output data. This is what you do when the physics is too complex, unknown, or when you want to capture effects that first-principles models miss (nonlinear friction, cable stiffness, gear backlash).

In practice, you almost always use **both**: first-principles to get the model structure ("it is a second-order system with a delay"), then sysid to get the actual parameter values (K, τ, ζ, etc.).

4.7.1 Step Response Method (Time Domain)

The simplest sysid technique. Apply a step input and record the output.

For a first-order system $G(s) = \frac{K}{\tau s+1}$:

1. Apply a step of amplitude A to the motor voltage.

2. Record the velocity response $y(t)$.

3. Read off:
   - K = (final steady-state value) / A = DC gain
   - τ = time to reach 63.2% of the final value = time constant

For a second-order system $G(s) = \frac{K\omega_n^2}{s^2+2\zeta\omega_n s+\omega_n^2}$:

1. Apply a step and record.

2. Read off:
   - K = (final value) / (step amplitude)
   - M_p = percent overshoot $\Rightarrow \zeta = \frac{-\ln(M_p/100)}{\sqrt{\pi^2+\ln^2(M_p/100)}}$
   - t_p = time to first peak $\Rightarrow \omega_n = \frac{\pi}{t_p\sqrt{1-\zeta^2}}$

Practical tips:

- Log data at the highest rate your sensor supports (1 kHz+ for motors).

- Run the test 3–5 times and average to reduce noise.

- Test with different step amplitudes to check linearity. If the response shape changes, the system is nonlinear.

- Record both the rising step (0 → V) and the falling step (V → 0). If they differ, you have asymmetric dynamics (e.g., different friction in forward and reverse).

4.7.2 Frequency Response Method (Bode Identification)

Apply sinusoidal inputs at different frequencies and measure the gain and phase shift at each frequency. This directly constructs the Bode plot of the system.

Procedure:

1. Apply $u(t) = A\sin(\omega t)$ to the system.

2. Wait for transients to die out (several periods).

3. Measure the steady-state output amplitude B and phase lag ϕ.

4. Compute gain: $|G(j\omega)| = B/A$ and phase: $\angle G(j\omega) = \phi$.

5. Repeat for ω from $0.1\times$ to $10\times$ the expected bandwidth.

Why this is powerful: The Bode plot directly tells you the system order (count the rolloff slopes), the resonance frequencies (peaks in the magnitude plot), and the time delays (linear phase decrease). You can then fit a transfer function to match the measured Bode plot.

Faster alternative — chirp signal: Instead of testing one frequency at a time, apply a **chirp** (frequency sweep) that ramps from low to high frequency over a few seconds. Then compute the FFT of both input and output, and divide: $G(j\omega) = \frac{Y(j\omega)}{U(j\omega)}$. One experiment gives you the entire Bode plot.

4.7.3 Least-Squares Identification (Automated)

For digital systems, sysid can be fully automated. Given input-output data $\{u[k], y[k]\}$, fit a discrete transfer function:

$$y[k] = -a_1 y[k-1] - \cdots - a_n y[k-n] + b_0 u[k-1] + \cdots + b_m u[k-m-1] \tag{45}$$

This is a **linear regression** problem. Stack all samples into a matrix equation $\mathbf{y} = \Phi\boldsymbol{\theta}$ and solve by least squares: $\boldsymbol{\theta} = (\Phi^T\Phi)^{-1}\Phi^T\mathbf{y}$.

In Python (3 lines):

```
from scipy.signal import dlsim
from scipy.optimize import curve_fit
# Or use the dedicated library:
import control
sys_id = control.tf(num, den, dt)  # fit to data

# Even simpler with MATLAB:
% data = iddata(y, u, Ts);
% sys = tfest(data, 2);  % fit 2nd-order TF
% bode(sys);  % inspect the result
```

MATLAB's System Identification Toolbox is the gold standard for this. The function `tfest(data, n)` fits an n-th order transfer function to recorded data automatically. Start with $n = 1$, increase until the fit is good (typically $n = 2$ is enough for motors).

4.7.4 Practical Sysid Workflow for Competition Robots

1. **Open-loop step test:** Disable all controllers. Send a known voltage step to the motor. Log encoder/IMU at maximum rate.
2. **Quick estimate:** Eyeball the step response for K and τ (first-order) or K, ζ, ω_n (second-order).
3. **Validate:** Simulate the identified model with the same step input. Overlay the simulation on the real data. If they match, you are done.
4. **Refine if needed:** If the first-order model is not accurate enough, try second-order. If there is a visible delay, add a time delay term $e^{-\tau_d s}$.
5. **Feed into controller design:** Use the identified model for PID tuning (MATLAB PID Tuner), LQR design (A, B matrices), or feedforward computation.

Common gotchas:

- **Friction:** Static friction (stiction) makes the system nonlinear at low speeds. Your linear model will be inaccurate near zero velocity. Consider identifying separate models for "moving" and "starting."
- **Backlash:** Gear backlash creates a dead zone. The system does not respond to small inputs until the gears engage. This shows up as a flat region in the step response.
- **Saturation:** If your step amplitude drives the motor into saturation, the identified model will be wrong. Use a step amplitude within the linear operating range (typically 30–70% of max voltage).
- **Coupling:** On a multi-axis system (gimbal, chassis), moving one axis can excite another. Identify each axis separately with the others locked if possible.

Why sysid matters: A controller designed for a model with 20% parameter error will have 20% worse performance than one designed for the true model. Five minutes of data logging + sysid can save hours of manual PID tuning. This is especially true for LQR and MPC, which depend heavily on accurate A and B matrices.

4.8 Feedforward Control

Feedback is reactive—it waits for an error to occur, then corrects. Feedforward is **proactive**—it predicts what the system needs and provides it immediately.

Example: You want your gimbal to track a target moving at constant velocity. With pure feedback, the controller is always chasing the target, resulting in a tracking error. With feedforward, you calculate the motor voltage needed to maintain that velocity and add it to the feedback output:

$$u(t) = u_{\text{feedforward}}(t) + u_{\text{feedback}}(t) \tag{46}$$

The feedforward term handles the "expected" part of the motion; feedback handles the "unexpected" part (disturbances, model errors).

Gravity compensation is a common feedforward term for robotic arms and gimbals:

$$u_{\text{ff}} = \tau_{\text{gravity}}(\theta) = mgl\sin(\theta) \tag{47}$$

Without this term, your feedback controller wastes effort fighting gravity instead of tracking the reference.

The golden rule of feedforward: Feedforward relies on the model, so it is only as good as the model. Feedback does not rely on the model, so it is always robust. **Use feedforward to improve performance, but always keep feedback as your safety net.**

Case Study: Cascaded PID on a Gimbal — Why Two Loops Beat One

You want your gimbal to point at a specific angle. The naive approach: one PID loop from angle error directly to motor voltage. This works, but it is mediocre. Here is why, and what the pros do instead.

The problem with single-loop control:

The transfer function from voltage to angle is second-order (motor dynamics + integration from velocity to position). A single PID must handle both the fast electrical/current dynamics and the slow mechanical dynamics simultaneously. It is like asking one person to drive the car AND navigate at the same time—doable but stressful.

The cascaded (nested) loop solution:

Use two PID controllers in series:

1. **Inner loop (velocity):** Runs at 1 kHz. Input: desired velocity. Output: motor voltage command. This loop is fast and rejects disturbances (friction, load changes) quickly.
2. **Outer loop (position):** Runs at 200 Hz. Input: desired angle. Output: desired velocity (which becomes the inner loop's setpoint). This loop handles the slower position tracking.

Why cascaded is better:

- Each loop handles one "layer" of dynamics. Simpler to tune—tune inner loop first (ignoring outer loop), then tune outer loop.
- The inner loop acts as a "shield"—it rejects disturbances before the outer loop even sees them. If someone bumps your gimbal, the velocity loop corrects within milliseconds.
- Limits are enforced at each level: the outer loop outputs a velocity command that is naturally bounded, preventing sudden large torques.

Adding feedforward to the mix:

Now add a velocity feedforward from the trajectory planner:

$$\omega_{\text{cmd}} = \underbrace{K_p(\theta_{\text{ref}} - \theta)}_{\text{outer PID}} + \underbrace{\dot{\theta}_{\text{ref}}}_{\text{feedforward}}$$

The feedforward gives the inner loop the velocity it needs to follow the reference trajectory. The outer PID only corrects small tracking errors. Result: smooth, accurate tracking even during fast motion. This is the architecture used by nearly every competitive RoboMaster gimbal.

Rule of thumb for cascaded loops:

- Inner loop should be 5–10× faster than outer loop.
- Tune from inside out: inner first, then outer.
- Each loop should look like a simple first-order system to the loop above it.

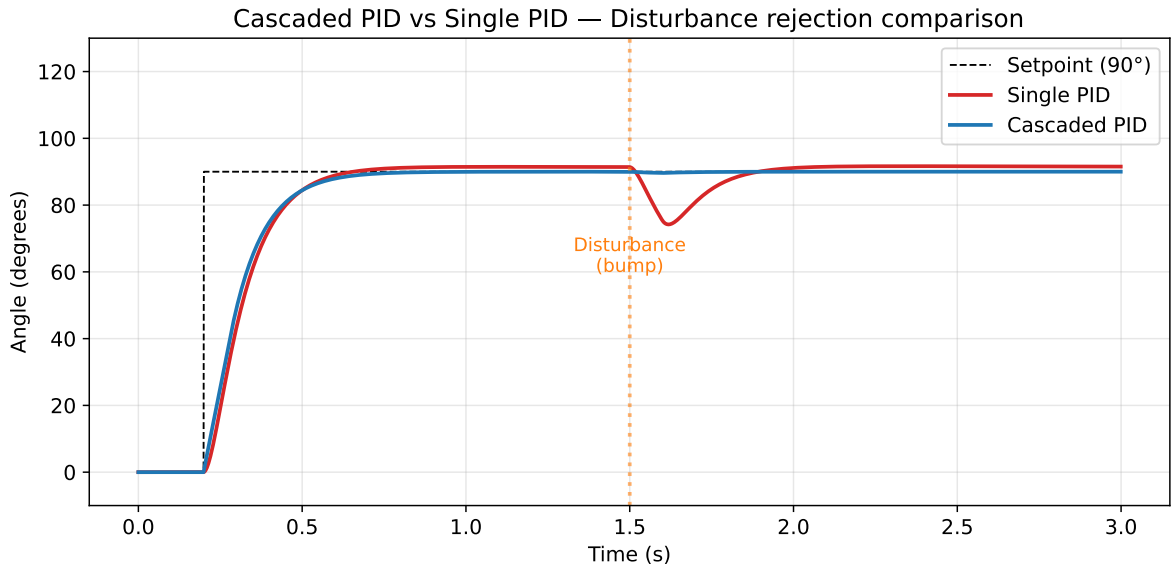

Figure 13: Cascaded PID (blue) vs single PID (red) on a gimbal motor. Both track the 90° setpoint, but when a disturbance hits at $t = 1.5$s, the cascaded architecture recovers much faster because the inner velocity loop absorbs the shock.

4.9 PID Architectures: Single, Cascaded, and Parallel-Cascaded

The case study above demonstrated why cascaded PID outperforms a single-loop PID on a single axis. In practice, competition robots require coordinated multi-axis control (e.g., gimbal yaw + pitch, chassis X + Y + rotation). This section systematically compares three PID architectures and provides ready-to-use code for each.

4.9.1 Architecture 1: Single-Loop PID

The simplest architecture: one PID controller per axis, from position error directly to motor voltage.

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau)\, d\tau + K_d \frac{de(t)}{dt}, \qquad e(t) = r(t) - y(t)$$

Advantages: Simple to implement, only one set of gains to tune per axis, minimal computational cost.

Limitations: The single controller must simultaneously handle fast dynamics (electrical/current) and slow dynamics (mechanical/position). This coupling makes tuning difficult—increasing gains for faster tracking often triggers oscillation. Disturbance rejection is slow because the controller only reacts after the disturbance propagates through the entire plant to the position output.

4.9.2 Architecture 2: Cascaded (Series) PID

Two nested loops per axis: the outer loop generates a velocity (or current) setpoint for the inner loop.

$$\omega_{\text{cmd}} = \text{PID}_{\text{outer}}\big(\theta_{\text{ref}} - \theta\big), \qquad u = \text{PID}_{\text{inner}}\big(\omega_{\text{cmd}} - \omega\big)$$

Advantages:

- Each loop handles a single dynamic layer, making tuning straightforward (inner first, then outer).
- The inner loop rejects disturbances before they reach the outer loop—disturbance rejection is dramatically faster.

- Natural saturation enforcement: the outer loop outputs a bounded velocity command, preventing sudden large torques.

Design rules:

- The inner loop bandwidth should be 5–10× that of the outer loop.
- From the outer loop's perspective, the inner loop should behave like a simple first-order system (i.e., well-tuned, fast, no overshoot).
- Tune from inside out: stabilize and optimize the inner loop with the outer loop disabled, then close the outer loop.

4.9.3 Architecture 3: Parallel-Cascaded PID (Multi-Axis)

On a multi-axis system (e.g., a gimbal with yaw and pitch), each axis runs its own cascaded PID controller *in parallel*. The axes share sensor data and can optionally include cross-axis coupling compensation.

Why coupling matters: On a gimbal, pitching the camera shifts the center of mass, creating a torque disturbance on the yaw axis. Without compensation, the yaw controller sees this as an unexpected disturbance and reacts slowly. With a cross-coupling feedforward term, the yaw controller anticipates the pitch motion and compensates proactively:

$$u_{\text{yaw}} = \text{CascadedPID}_{\text{yaw}}(\cdot) + k_{\text{cross}} \cdot \dot{\theta}_{\text{pitch}}$$

```
// Parallel-cascaded PID with cross-coupling compensation
struct GimbalController {
    CascadedPID yaw;
    CascadedPID pitch;
    float cross_yaw_from_pitch = 0.0f;  // coupling gain

    struct Output { float yaw_voltage; float pitch_voltage; };

    Output update(float yaw_ref, float yaw_pos, float yaw_vel,
                  float pitch_ref, float pitch_pos, float pitch_vel,
                  float yaw_vel_ff = 0, float pitch_vel_ff = 0) {
        // Cross-coupling compensation: pitch rate disturbs yaw
        float cross_comp = cross_yaw_from_pitch * pitch_vel;

        float u_pitch = pitch.update(pitch_ref, pitch_pos,
                                     pitch_vel, pitch_vel_ff);
        float u_yaw   = yaw.update(yaw_ref, yaw_pos,
                                   yaw_vel, yaw_vel_ff)
                      + cross_comp;
        return {u_yaw, u_pitch};
    }

    void reset() { yaw.reset(); pitch.reset(); }
};
```

4.9.4 Comparison Summary

	Single PID	**Cascaded PID**	**Parallel-Cascaded**
Loops per axis	1	2	2
Tuning difficulty	Medium	Low (layered)	Low (layered)
Disturbance rejection	Slow	Fast	Fast
Cross-axis coupling	Not handled	Not handled	Compensated
Code complexity	Minimal	Moderate	Moderate
Typical use case	Simple actuators	Single-axis gimbal	Multi-axis gimbal

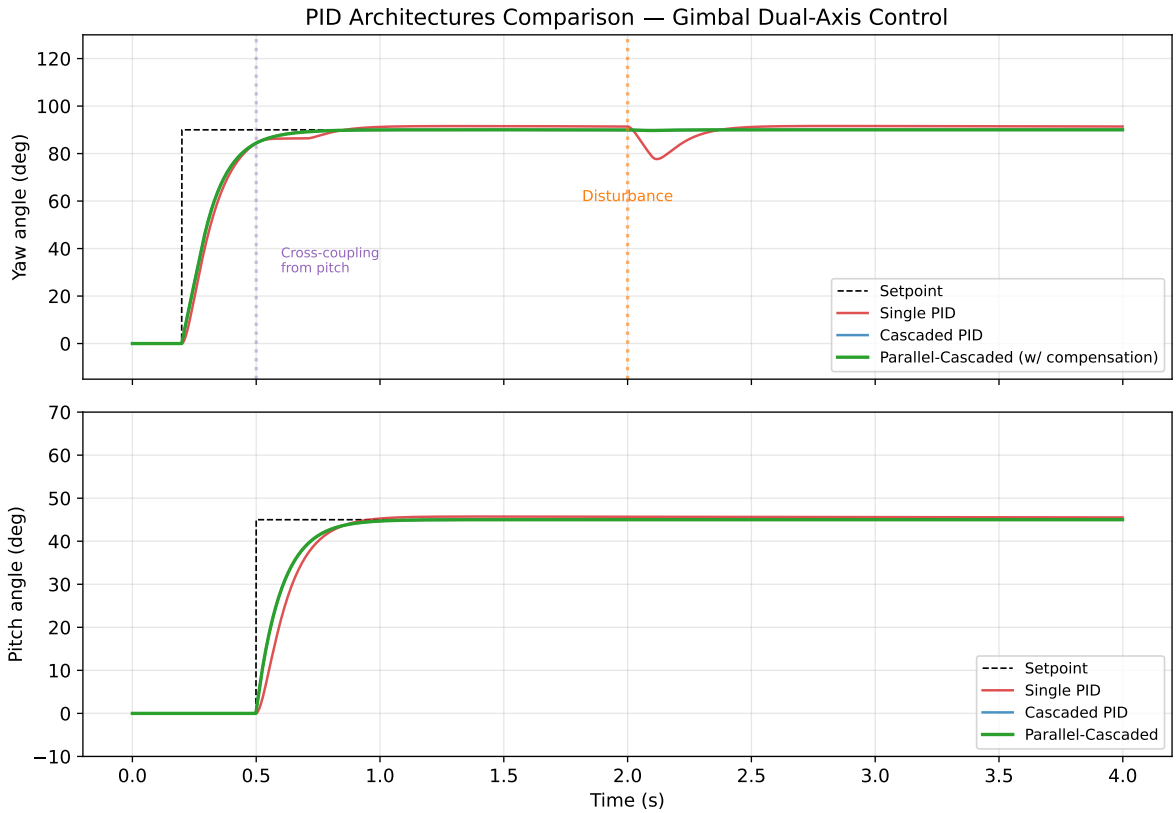

Figure 14: PID architecture comparison on a dual-axis gimbal. Top: yaw axis with disturbance at $t = 2$s and cross-coupling from pitch motion at $t = 0.5$s. Bottom: pitch axis. The parallel-cascaded architecture (green) with cross-coupling compensation provides the cleanest yaw response under both disturbance and coupling.

Practical recommendation: For any system with more than one controlled axis, use the parallel-cascaded architecture. The additional code complexity is minimal (a single struct wrapping two `CascadedPID` objects), while the performance gain from cross-coupling compensation and layered tuning is substantial.

Practical Example: Two-Wheeled Self-Balancing Scooter (Parallel PID)

A two-wheeled self-balancing robot (like a Segway) is a classic *parallel* multi-loop control problem. At least three objectives must be controlled simultaneously: balance (tilt angle), velocity, and yaw (steering). Each objective gets its own controller, and their outputs are *summed* at the motor level.

- **Balance loop (fast):** PD controller on tilt angle ϕ from the IMU. This is the most critical loop—it must run at ≥ 500 Hz and react within milliseconds. The output is a torque command to both wheels (same direction).
- **Velocity loop (medium):** PI controller on wheel velocity (encoder). The reference comes from the user joystick. The output is added to the balance torque. This loop prevents the robot from drifting while maintaining balance.
- **Yaw loop (slow):** P controller on yaw rate (gyroscope). The output is a *differential* torque—positive to the left wheel, negative to the right (or vice versa)—to turn the robot.

Motor command fusion:

$$u_{\text{left}} = \underbrace{\tau_{\text{balance}}}_{\text{PD on } \phi} + \underbrace{\tau_{\text{velocity}}}_{\text{PI on } v} + \underbrace{\tau_{\text{yaw}}}_{\text{P on } \dot{\psi}}$$

$$u_{\text{right}} = \tau_{\text{balance}} + \tau_{\text{velocity}} - \tau_{\text{yaw}}$$

The sign flip on τ_{yaw} creates differential drive for steering. Each loop is tuned independently:

balance first (the robot must stand before it can move), then velocity, then yaw.

4.9.5 Output Conflict in Parallel PID

When multiple PID controllers run in parallel and their outputs are summed into the same actuator, they can *fight each other*. This is the fundamental risk of parallel architectures.

The balancing scooter conflict. Consider the balance loop commanding "tilt forward to stay upright" while the velocity loop simultaneously commands "tilt backward to slow down." Both are correct in their own frame, but their summed output partially cancels, leaving the motor with a weak, confused command. In the worst case, the loops oscillate against each other: the velocity loop creates a tilt disturbance that the balance loop corrects, which changes velocity, which triggers the velocity loop again—an inter-loop limit cycle.

Why this happens: In a cascade, the outer loop *only* talks to the inner loop, and the inner loop *only* talks to the actuator. The hierarchy prevents conflict by construction. In a parallel architecture, all loops talk to the actuator directly, so their objectives must be *naturally compatible*. When they are not, the summed output becomes incoherent.

Mitigation strategies:

1. **Bandwidth separation.** Ensure each loop operates in a distinct frequency band: the fastest loop (balance) dominates at high frequencies, the slowest (yaw) only acts at low frequencies. Conflicting commands are then separated in time scale and do not interfere.
2. **Priority saturation.** Clamp the slower loops' outputs so they cannot override the faster, safety-critical loop. For example, limit $|\tau_{\text{velocity}}|$ so it never exceeds a fraction of $|\tau_{\text{balance}}|$.
3. **Output budget allocation.** Reserve a fixed portion of actuator range for each loop: e.g., 60% for balance, 30% for velocity, 10% for yaw. This guarantees that no loop can starve another.
4. **Convert to cascade.** If conflicts persist, restructure the controller so the velocity loop outputs a *tilt angle reference* for the balance loop, rather than a torque that competes with it. This eliminates the conflict by design—at the cost of increased coupling between loops.

Rule of thumb: parallel PID works well when the controlled variables are *physically orthogonal* (e.g., balance vs. yaw on a scooter—one is sagittal, the other is lateral). It struggles when two loops try to control variables that share the same actuator degree of freedom (e.g., both balance and velocity demand sagittal torque).

4.9.6 Food for Thought: Is Inner-Loop K_p the Same as Outer-Loop K_d?

Consider a position–velocity cascade. The outer loop outputs a velocity reference proportional to position error: $v_{\text{ref}} = K_p^{\text{outer}}(r - y)$. The inner loop then tracks this velocity. Now consider a single PID on position: the derivative term produces $K_d \cdot \dot{e} \approx -K_d \cdot \dot{y}$, which is a feedback proportional to velocity.

Are they equivalent? *Almost, but not quite.* In steady-state linear analysis, the outer K_p acting through an ideal inner velocity loop produces an effect similar to K_d on position error. Specifically, if the inner loop has unity gain and no dynamics (instantaneous velocity tracking), then:

$$u = K_p^{\text{inner}}\big(K_p^{\text{outer}}(r - y) - \dot{y}\big)$$

which expands to terms resembling $K_p \cdot e + K_d \cdot (-\dot{y})$ with $K_d \approx K_p^{\text{inner}}$.

But the cascade is superior in practice because:

1. The inner loop has its *own* integral term that eliminates velocity-level disturbances (friction, load changes) without relying on the outer loop.
2. The inner loop *bandwidth-limits* the outer command, preventing the outer loop from demanding physically impossible accelerations. A raw K_d has no such protection—it amplifies high-frequency noise and can saturate the actuator.
3. The inner loop provides a *measured* velocity signal, not a numerically differentiated one. Differentiation amplifies noise; direct measurement does not.

Bottom line: the derivative term and the inner proportional gain are conceptually related—both inject velocity-dependent damping. But the cascade architecture provides noise rejection, disturbance rejection at the velocity level, and implicit rate limiting that a bare K_d term cannot match.

5 Discretization and Implementation

You have designed a beautiful continuous-time controller on paper (or in MATLAB). Now you need to make it run on an STM32 at 1 kHz inside a timer interrupt. This section bridges the gap between theory and embedded reality.

5.1 From Continuous to Discrete

A continuous-time transfer function $C(s)$ must be converted to a discrete-time transfer function $C(z)$ before it can be implemented in code. There are several methods, each with trade-offs.

5.1.1 Zero-Order Hold (ZOH)

The ZOH method assumes the input is held constant between samples (which is exactly what a DAC does). It is the most physically accurate discretization method.

The idea: Between sampling instants, the control signal is constant. We compute exactly what the continuous system would do under this piecewise-constant input.

For a first-order system $G(s) = \frac{a}{s+a}$ with sampling period T_s:

$$G(z) = \frac{1 - e^{-aT_s}}{z - e^{-aT_s}} \tag{48}$$

When to use: When you want the discrete system to match the continuous system exactly at the sampling instants. MATLAB's `c2d(sys, Ts, 'zoh')` uses this method.

Downside: The conversion formulas can be complex for higher-order systems. Usually done with software tools, not by hand.

5.1.2 Tustin (Bilinear) Transform

The Tustin method replaces s with:

$$s \leftarrow \frac{2}{T_s} \cdot \frac{z-1}{z+1} \tag{49}$$

This is a simple algebraic substitution. You take your continuous transfer function, replace every s with the expression above, and simplify.

Example: A first-order low-pass filter $H(s) = \frac{\omega_c}{s+\omega_c}$ becomes:

$$H(z) = \frac{\omega_c T_s (z+1)}{(2 + \omega_c T_s)z + (\omega_c T_s - 2)} \tag{50}$$

Why Tustin is popular:

- Simple to compute by hand for low-order systems.
- Preserves stability: a stable continuous system maps to a stable discrete system.
- Good frequency-domain match at low frequencies.

Gotcha: At high frequencies (near the Nyquist frequency), Tustin introduces **frequency warping**—the discrete filter's cutoff frequency is shifted relative to the continuous design. Use **pre-warping** to correct this:

$$\omega_{\text{pre-warped}} = \frac{2}{T_s} \tan\left(\frac{\omega_c T_s}{2}\right) \tag{51}$$

Replace ω_c with $\omega_{\text{pre-warped}}$ in your continuous design, then apply Tustin.

5.1.3 Matched Pole-Zero Method

This method maps the poles and zeros of the continuous system to the z-domain using $z = e^{sT_s}$. It preserves the time-domain response characteristics well.

When to use: For systems where matching the transient response is more important than matching the frequency response. Less common in practice than ZOH or Tustin.

Bottom line: For most competition applications, **Tustin is the go-to method**. It is simple, stable, and accurate enough. Use ZOH when you need exact sample-instant matching. If you are using MATLAB, just use `c2d()` and let it handle the math.

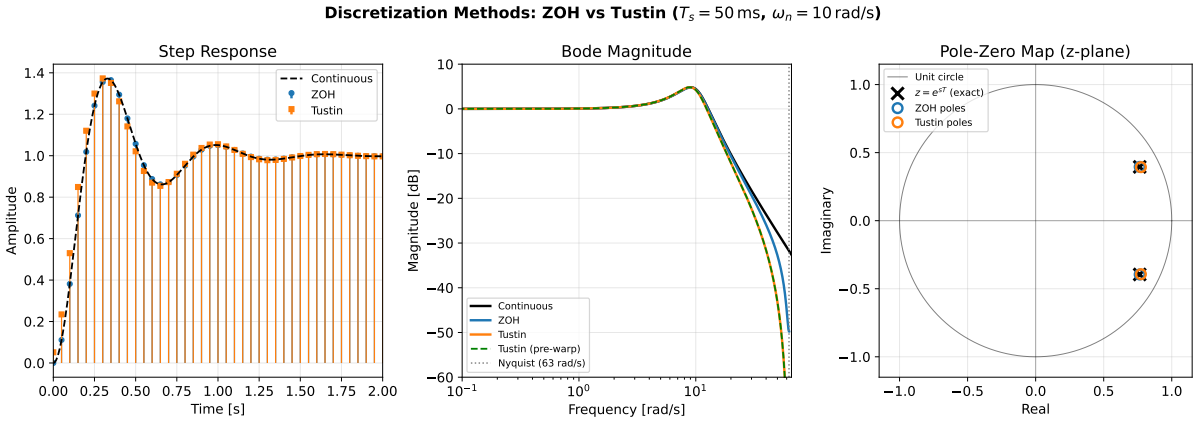

Figure 15: ZOH vs Tustin discretization of an underdamped 2nd-order system ($\omega_n = 10$ rad/s, $\zeta = 0.3$, $T_s = 50$ ms). Left: step response—ZOH matches the continuous system exactly at sample instants. Center: Bode magnitude—Tustin shows frequency warping near the Nyquist frequency; pre-warping (green) corrects this. Right: pole-zero map showing how each method maps continuous poles to the z-plane.

5.2 Fixed-Point vs Floating-Point Arithmetic

Many microcontrollers used in competitions (STM32F1, STM32F4) have hardware floating-point units (FPU). If yours does, **use floating-point**. It is easier, less error-prone, and fast enough for most control loops.

If you are stuck with a microcontroller without an FPU, or if you need maximum speed, you will need **fixed-point arithmetic**. The idea is to represent fractional numbers as scaled integers:

$$\text{fixed_value} = \text{float_value} \times 2^Q \tag{52}$$

where Q is the number of fractional bits. For example, in Q16 format, the number 3.14 is stored as $3.14 \times 65536 = 205,888$.

Pitfalls:

- Overflow: multiplication of two Q16 numbers produces a Q32 result that must be shifted back.
- Range vs. precision trade-off: more fractional bits means more precision but less range.
- Integral accumulation: the integral term in PID can overflow quickly in fixed-point.

Recommendation: Use 32-bit float unless you have a strong reason not to. STM32F4 and above can do single-precision float in one clock cycle.

Why Computation Speed Directly Determines Control Performance

Many students assume that as long as the algorithm is "correct," the control will work. This overlooks a critical fact: *every control algorithm must finish executing within the sampling period* T_s. If the computation takes longer than T_s, the controller misses its deadline, the next cycle is delayed, and the effective control frequency drops. In the worst case, the system becomes unstable.

A concrete example. Suppose your velocity loop runs at 1 kHz ($T_s = 1$ ms) on an STM32F407 at 168 MHz. How much time do different algorithms consume?

Algorithm	Complexity	Approx. time	Fits in 1 ms?
PID (3 multiplies + adds)	$O(1)$	$\sim$1 μs	Yes (1000$\times$ margin)
2nd-order IIR filter	$O(1)$	$\sim$1 μs	Yes
Kalman filter (4×4)	$O(n^3)$, $n = 4$	$\sim$20 μs	Yes (50$\times$ margin)
LQR (gain multiply, 4×1)	$O(n \cdot m)$	$\sim$2 μs	Yes
MPC ($N = 10$, $n = 4$, $m = 2$)	$O(N \cdot n^3)$	$\sim$200–500 μs	Barely
EKF with trig functions	$O(n^3) + \text{trig}$	$\sim$50–100 μs	Yes, but watch out

What happens when computation overruns T_s:

- **Missed update:** The actuator holds its previous command for an extra cycle. The system sees this as an increased time delay. Recall from the shower analogy (Section 4.1): more delay means less phase margin, which means closer to instability.
- **Inconsistent dt:** If the overrun is intermittent (computation is sometimes fast, sometimes slow), the effective dt fluctuates. The integral and derivative terms—which assume a constant dt—accumulate errors. This is *jitter*, and it is often more damaging than a constant delay because it is unpredictable.
- **Cascading failure:** In a cascaded architecture, if the inner loop overruns, the outer loop receives stale velocity data. The outer loop's response degrades, and disturbance rejection—the primary reason for using cascaded control—is compromised.

Practical diagnostics. Toggle a GPIO pin at the start and end of your ISR, and observe the pulse width on an oscilloscope. If the pulse width is close to T_s, you are in danger. If it ever exceeds T_s, you have a deadline violation that *must* be fixed before tuning.

What to do when computation is too slow:

1. **Reduce algorithm complexity.** Replace MPC with LQR if the constraints are not active. Replace EKF with a complementary filter if the nonlinearity is mild.
2. **Precompute offline.** LQR gains, lookup tables for trigonometric functions, and MPC gain sequences can all be computed in MATLAB/Python and stored as constants.
3. **Lower the control frequency.** If the system's bandwidth allows it, running at 500 Hz instead of 1 kHz doubles your computation budget. Check whether your system's time constant permits this (recall: $T_s \leq \tau/10$).
4. **Distribute across loops.** Run the expensive outer loop at a lower frequency (e.g., MPC at 100 Hz) while keeping a simple inner loop (PID at 1 kHz). The inner loop maintains control between MPC updates.

The key insight: The best algorithm is not the most sophisticated one—it is the one that reliably finishes within T_s while meeting performance requirements. A perfectly tuned PID running at 1 kHz will always outperform an MPC that overruns its deadline at 200 Hz.

5.3 Sampling Rate Selection

How fast should your control loop run? Too slow and you cannot react to fast disturbances; too fast and you waste CPU time (and the derivative term amplifies quantization noise).

Rule of thumb: The sampling rate should be 10–30$\times$ the bandwidth of the closed-loop system.

Application	Bandwidth	Sampling rate
Chassis velocity	$\sim$5 Hz	50–200 Hz
Gimbal position	$\sim$20 Hz	200–1000 Hz
Motor current loop	$\sim$500 Hz	5–20 kHz

Practical constraint: Your sampling rate is bounded by:

- Sensor update rate (no point sampling faster than your encoder/IMU updates).
- Computation time (all control math must finish within one sampling period).
- Communication latency (CAN bus, SPI, I2C can be bottlenecks).

Key insight: For cascaded control loops (e.g., position $\rightarrow$ velocity $\rightarrow$ current), the inner loop should run 5–10$\times$ faster than the outer loop. Current loop at 10 kHz, velocity loop at 1 kHz, position loop at 200 Hz is a common configuration.

5.4 Implementation on Embedded Systems

5.4.1 Timer and Interrupt Configuration

On most microcontrollers (STM32, etc.), the control loop runs inside a **timer interrupt**. The timer triggers at a fixed period (T_s), and the interrupt service routine (ISR) executes the control algorithm.

Best practices:

- Use a dedicated hardware timer for each control loop.
- Set the timer period to your desired T_s using the prescaler and auto-reload register.
- Keep the ISR **as short as possible**. Read sensor, compute output, set actuator. No printing, no UART, no memory allocation.
- Use `volatile` for variables shared between the ISR and the main loop.
- Set interrupt priorities correctly: current loop > velocity loop > position loop.

Example (STM32 HAL):

```
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {
    if (htim->Instance == TIM6) {  // 1 kHz control loop
        float measurement = read_encoder();
        float error = setpoint - measurement;
        float output = pid_compute(&pid, error);
        set_motor_pwm(output);
    }
}
```

5.4.2 Computation Latency and Jitter

Latency is the time between reading the sensor and applying the output. Lower is better. Anything above $T_s/2$ starts degrading performance significantly.

Jitter is the variation in latency from one cycle to the next. If your loop usually takes 10 μs but occasionally takes 500 μs (because another interrupt preempted it), you get inconsistent behavior. Jitter is often more harmful than a constant latency.

How to minimize jitter:

- Give your control timer the highest interrupt priority.
- Avoid disabling interrupts globally for long periods.
- Do not use blocking waits (delay, polling) in any ISR.
- Profile your ISR execution time and make sure it fits comfortably within T_s.

5.4.3 Anti-Windup in Practice

We discussed anti-windup in the PID section, but it deserves special emphasis here because it is the #1 source of "mysterious" PID misbehavior on real hardware.

When does windup happen in practice?

- Motor current limit is reached (you are commanding 20A but the ESC caps at 10A).
- PWM is saturated (duty cycle at 100%, cannot go higher).
- Mechanical hard stop (gimbal hits its limit, wheels are stalled).
- Communication failure (command sent but not received—output is effectively zero).

Implementation checklist:

1. Clamp the integral term to a reasonable range.
2. Clamp the total output to the actuator's actual limits.

3. When the output is saturated, stop accumulating the integral (conditional integration).
4. When switching controllers or modes, reset or smoothly transition the integral term.
5. Test windup recovery explicitly: command a large step, let it saturate, then reverse.

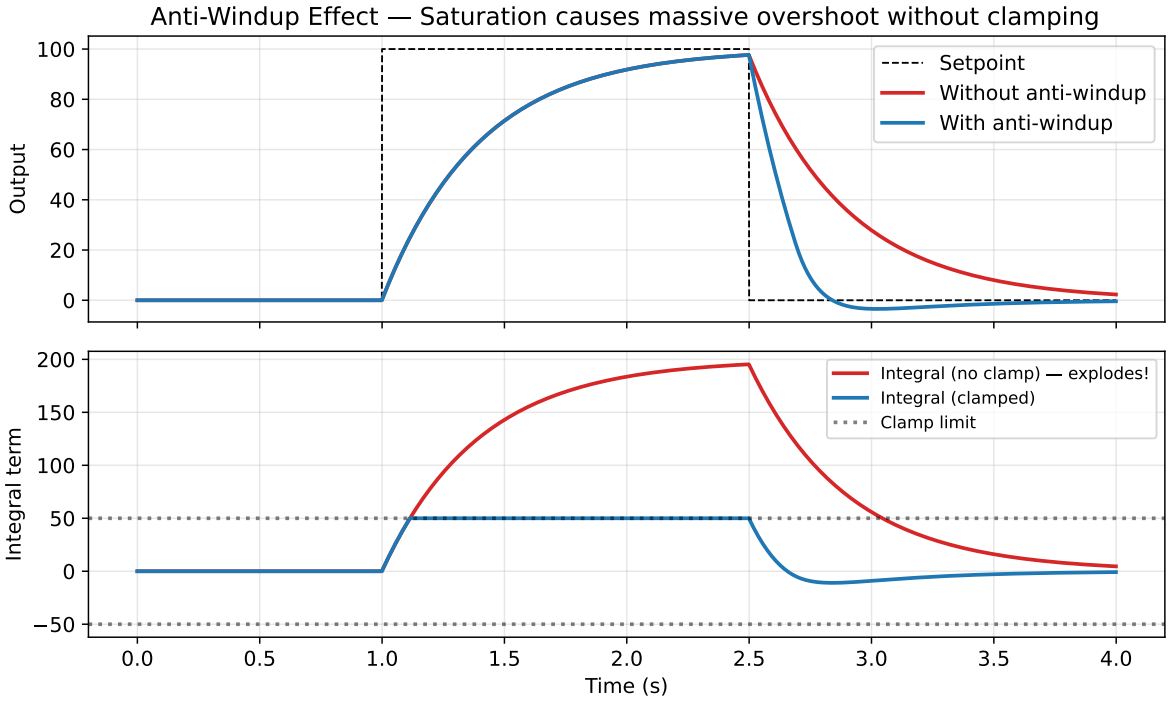

Figure 16: Without anti-windup (red), the integral term explodes during saturation, causing massive overshoot when the setpoint changes. With clamping (blue), the integral stays bounded and the response is clean.

A debugging story: Your robot's wheel motor oscillates wildly after recovering from a stall. You check K_p, K_d—they look fine. You try reducing gains—it helps but makes the system sluggish. The real cause? The integral term accumulated to a huge value during the stall and is now overdriving the motor in the opposite direction. Adding integral clamping fixes it instantly. This happens to everyone. It will happen to you. Now you know what to look for.

Putting It All Together: From Continuous PID to STM32 Code

This example walks through the complete pipeline: start with a continuous-time PID, discretize it, and implement it in an STM32 timer interrupt.

Step 1: Continuous PID. The textbook PID transfer function is:

$$C(s) = K_p + \frac{K_i}{s} + K_d s$$

With $K_p = 6$, $K_i = 0.3$, $K_d = 3$, and $T_s = 1\,\text{ms}$ (1 kHz control loop).

Step 2: Tustin discretization. Apply the bilinear transform $s \leftarrow \frac{2}{T_s}\frac{z-1}{z+1}$ to each term:

- **Integral:** $\frac{K_i}{s}$ $\rightarrow$ `integral += Ki * Ts * 0.5 * (e + e_prev)` (trapezoidal rule)
- **Derivative:** $K_d s$ $\rightarrow$ `deriv = Kd * (e - e_prev) / Ts` (backward difference on error, or better: on measurement)

Step 3: STM32 implementation.

```
// In TIM6 interrupt handler, every 1ms:
void TIM6_DAC_IRQHandler(void) {
    HAL_TIM_IRQHandler(&htim6);
```

```
    float y = read_encoder();        // measured output
    float r = get_setpoint();        // reference
    float e = r - y;

    // Tustin integral (trapezoidal)
    integral += Ki * Ts * 0.5f * (e + e_prev);
    integral = clamp(integral, -IMAX, IMAX);  // anti-windup

    // Derivative on measurement (avoids derivative kick)
    float dy = (y - y_prev) / Ts;

    // PID output
    float u = Kp * e + integral - Kd * dy;
    u = clamp(u, -UMAX, UMAX);

    set_pwm(u);  // actuator output

    e_prev = e;
    y_prev = y;
}
```

Key implementation details:

1. **Derivative on measurement** ($-K_d\dot{y}$), not on error ($K_d\dot{e}$). A step change in r creates an infinite $\dot{e}$ spike that slams the actuator. Using $\dot{y}$ avoids this "derivative kick."
2. **Integral clamping** prevents windup when the actuator saturates.
3. **Output clamping** ensures the PWM stays within hardware limits.
4. **No floating-point in critical path** on MCUs without FPU—use fixed-point Q16 or Q24 arithmetic instead.
5. **All state variables** (`integral`, `e_prev`, `y_prev`) are `static` or file-scope globals, persisting across ISR calls.

6 Modern Control Theory

Classical control (PID, frequency response, root locus) got us far. It can handle most single-input, single-output systems beautifully. But some problems demand more:

- Your holonomic chassis has four motors that must be coordinated simultaneously (MIMO).
- You want *optimal* performance, not just "good enough" (optimal control).
- You need to estimate quantities you cannot measure directly, like velocity from position data or tilt angle from a noisy IMU (state estimation).
- You want to explicitly handle constraints like maximum motor current or joint angle limits (constrained optimization).

This is where modern control theory shines. It works directly with the state-space representation and uses linear algebra as its primary tool.

6.1 Relationship Between State-Space and Transfer Functions

These are two representations of the same system, and you can convert between them.

State-space to transfer function:

$$G(s) = C(sI - A)^{-1}B + D \tag{53}$$

Transfer function to state-space: Not unique—many different state-space realizations can give the same transfer function. Common forms include controllable canonical form and observable canonical form.

The key difference: Transfer functions describe the input-output behavior. State-space describes the internal dynamics. A system can have identical transfer functions but different internal structures—the state-space representation captures this extra information.

Why it matters: Some internal modes might be unstable but invisible in the transfer function (if they are cancelled by zeros). State-space reveals them. This is why modern control prefers state-space—it sees everything.

6.2 System Performance Metrics

6.2.1 Controllability and Observability

These are two fundamental questions that you should ask before designing any controller or observer.

Controllability: "Can I drive every state to any desired value using the available inputs?"

A system (A, B) is controllable if the **controllability matrix** has full rank:

$$\mathcal{C} = \begin{bmatrix} B & AB & A^2B & \cdots & A^{n-1}B \end{bmatrix}, \quad \text{rank}(\mathcal{C}) = n \tag{54}$$

Derivation: Why the Controllability Matrix Works

Where does $\mathcal{C} = [B \;\; AB \;\; \cdots \;\; A^{n-1}B]$ come from? It is not pulled out of thin air—it follows directly from the state transition equation.

Step 1: Iterative state propagation. For the discrete-time system $\mathbf{x}[k+1] = A\mathbf{x}[k] + B\mathbf{u}[k]$, start from $\mathbf{x}[0] = \mathbf{0}$ and propagate forward:

$$\mathbf{x}[1] = B\mathbf{u}[0] \tag{55}$$

$$\mathbf{x}[2] = A\mathbf{x}[1] + B\mathbf{u}[1] = AB\mathbf{u}[0] + B\mathbf{u}[1] \tag{56}$$

$$\mathbf{x}[3] = A^2B\mathbf{u}[0] + AB\mathbf{u}[1] + B\mathbf{u}[2] \tag{57}$$

$$\vdots$$

$$\mathbf{x}[n] = A^{n-1}B\mathbf{u}[0] + A^{n-2}B\mathbf{u}[1] + \cdots + B\mathbf{u}[n-1] \tag{58}$$

Step 2: Rewrite as a matrix equation. Stack the inputs into a vector:

$$\mathbf{x}[n] = \begin{bmatrix} A^{n-1}B & A^{n-2}B & \cdots & AB & B \end{bmatrix} \begin{bmatrix} \mathbf{u}[0] \\ \mathbf{u}[1] \\ \vdots \\ \mathbf{u}[n-1] \end{bmatrix} = \mathcal{C} \cdot \mathbf{u}_{\text{seq}} \tag{59}$$

(The columns of $\mathcal{C}$ are just reordered—rank is unchanged.)

Step 3: The rank condition. We want to reach *any* target state $\mathbf{x}[n] \in \mathbb{R}^n$. This requires the equation $\mathcal{C} \cdot \mathbf{u}_{\text{seq}} = \mathbf{x}[n]$ to have a solution for every right-hand side. By linear algebra, this is possible if and only if $\text{rank}(\mathcal{C}) = n$.

Why only n columns? The Cayley–Hamilton theorem guarantees that A^n can be written as a linear combination of $I, A, A^2, \ldots, A^{n-1}$. Therefore A^nB is already in $\text{span}\{B, AB, \ldots, A^{n-1}B\}$, and higher powers add nothing new. You never need more than n columns.

Continuous-time version: For $\dot{\mathbf{x}} = A\mathbf{x} + B\mathbf{u}$, the reachable set from $\mathbf{x}(0) = \mathbf{0}$ at time t is:

$$\mathbf{x}(t) = \int_0^t e^{A(t-\tau)} B\mathbf{u}(\tau)\, d\tau$$

Expanding the matrix exponential $e^{A(t-\tau)} = \sum_{k=0}^{\infty} \frac{(t-\tau)^k}{k!} A^k$ and again applying Cayley–Hamilton to collapse powers $\geq n$, the reachable set is $\text{span}\{B, AB, \ldots, A^{n-1}B\}$—the same controllability matrix.

Derivation: Why the Observability Matrix Works

The observability matrix arises from the dual question: given outputs $\mathbf{y}[0], \mathbf{y}[1], \ldots$, can we recover the initial state $\mathbf{x}[0]$?

Step 1: Collect output equations. Assume $\mathbf{u} = \mathbf{0}$ (we can always subtract the known input contribution). The free response gives:

$$\mathbf{y}[0] = C\mathbf{x}[0] \tag{60}$$
$$\mathbf{y}[1] = C\mathbf{x}[1] = CA\mathbf{x}[0] \tag{61}$$
$$\mathbf{y}[2] = CA^2\mathbf{x}[0] \tag{62}$$
$$\vdots$$
$$\mathbf{y}[n-1] = CA^{n-1}\mathbf{x}[0] \tag{63}$$

Step 2: Stack into a matrix equation.

$$\begin{bmatrix} \mathbf{y}[0] \\ \mathbf{y}[1] \\ \vdots \\ \mathbf{y}[n-1] \end{bmatrix} = \underbrace{\begin{bmatrix} C \\ CA \\ CA^2 \\ \vdots \\ CA^{n-1} \end{bmatrix}}_{\mathcal{O}} \mathbf{x}[0] \tag{64}$$

Step 3: The rank condition. To uniquely recover $\mathbf{x}[0]$ from the output sequence, the equation $\mathcal{O} \cdot \mathbf{x}[0] = \mathbf{y}_{\text{seq}}$ must have a unique solution. This requires $\text{rank}(\mathcal{O}) = n$ (full column rank).

Why only n rows? Same reason as controllability—Cayley–Hamilton makes CA^n a linear combination of $C, CA, \ldots, CA^{n-1}$, so additional measurements add no new information.

The duality is now obvious: Controllability asks "can $\mathcal{C}$ map the input space *onto* the state space?" Observability asks "can $\mathcal{O}$ map the state space *into* distinguishable outputs?" Transpose one and you get the other—which is exactly the mathematical statement of duality: (A, B) controllable $\Leftrightarrow$ (A^T, B^T) observable (with B^T playing the role of C).

Physical interpretation: If a state is uncontrollable, no input can influence it. For example, if

your robot's chassis has a broken motor, you have lost controllability in certain directions.

Worked example: Consider a simplified 2D chassis with states $\mathbf{x} = \begin{bmatrix} v_x \\ v_y \end{bmatrix}$ (velocities in x and y) and a single forward-facing motor:

$$A = \begin{bmatrix} -1 & 0 \\ 0 & -1 \end{bmatrix}, \quad B = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

The controllability matrix: $\mathcal{C} = \begin{bmatrix} B & AB \end{bmatrix} = \begin{bmatrix} 1 & -1 \\ 0 & 0 \end{bmatrix}$. Rank $= 1$, but $n = 2$. **Not controllable!** The motor can only drive v_x; no input can change v_y. Physically obvious: a car cannot move sideways. Add a second motor facing sideways ($B = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$) and controllability is restored.

Observability: "Can I determine every state from the available outputs (measurements)?"

A system (A, C) is observable if the **observability matrix** has full rank:

$$\mathcal{O} = \begin{bmatrix} C \\ CA \\ CA^2 \\ \vdots \\ CA^{n-1} \end{bmatrix}, \quad \text{rank}(\mathcal{O}) = n \tag{65}$$

Physical interpretation: If a state is unobservable, you cannot infer it from measurements. For example, if you only measure motor position but not current, the current state might be unobservable (depending on the model).

The controllability-observability duality: A system (A, B, C) is controllable if and only if the dual system (A^T, C^T, B^T) is observable. This elegant symmetry means that techniques for controller design (pole placement) directly translate to observer design.

Bottom line: Always check controllability before designing a controller, and observability before designing an observer. If these conditions are not satisfied, no amount of mathematical sophistication will save you—you need to change the hardware (add sensors, fix actuators).

6.2.2 Closed-Loop System Stability

For the closed-loop state-space system with state feedback $\mathbf{u} = -K\mathbf{x}$:

$$\dot{\mathbf{x}} = (A - BK)\mathbf{x} \tag{66}$$

The system is stable if and only if all eigenvalues of $(A - BK)$ have negative real parts. The gain matrix K is what we design to place these eigenvalues where we want.

Lyapunov stability: A more general tool for stability analysis. A system is stable if there exists a positive definite function $V(\mathbf{x})$ (an "energy-like" function) that always decreases along trajectories ($\dot{V} < 0$). Finding such a V proves stability; failing to find one does not prove instability (it might just mean you did not look hard enough).

For linear systems, the Lyapunov equation is:

$$A^T P + PA = -Q \tag{67}$$

If $Q > 0$ and the solution $P > 0$, the system is stable. MATLAB's `lyap()` solves this directly.

6.3 Controller Design via State-Space

6.3.1 Bellman Equations and Dynamic Programming: The Engine Behind LQR and MPC

Before we dive into LQR and MPC, we need to understand the mathematical engine that powers both: **dynamic programming (DP)** and the **Bellman equation**. If you have done competitive programming (USACO, Codeforces, LeetCode), you already know DP — the control theory version is the same idea wearing a different hat.

The Bellman Principle of Optimality: *An optimal policy has the property that, regardless of the initial state and initial decision, the remaining decisions must constitute an optimal policy with regard to the state resulting from the first decision.*

In plain English: if you know the optimal strategy from every intermediate state onward, you can figure out the optimal first move by trying all options and picking the one that minimizes (first-step cost) + (optimal cost-to-go from the resulting state).

Warm-Up: Grid-World Shortest Path (DP You Already Know)

Consider a classic DP problem: you are on an $n \times n$ grid. You start at $(0, 0)$ and want to reach $(n-1, n-1)$. Each cell (i, j) has a cost $c(i, j)$. You can move right or down. Find the minimum-cost path.

State: Your position (i, j).
Decision (control): Move right or move down.
Cost-to-go: $V(i, j)$ = minimum total cost to reach the goal from (i, j).
Bellman equation:

$$V(i, j) = c(i, j) + \min\{V(i + 1, j),\ V(i, j + 1)\}$$

Boundary: $V(n-1, n-1) = c(n-1, n-1)$.
Solution: Fill the table *backward* from the goal to the start. At each cell, the optimal decision is whichever neighbor has the smaller cost-to-go. Total complexity: $O(n^2)$.

You have been doing this since your first DP problem. Now watch what happens when we change the vocabulary:

DP (programming)	Optimal control	What it means
State (i, j)	State $\mathbf{x}_k$	Where you are now
Decision (right/down)	Control input $\mathbf{u}_k$	What you do next
Transition $(i, j) \to (i + 1, j)$	$\mathbf{x}_{k+1} = f(\mathbf{x}_k, \mathbf{u}_k)$	Physics/dynamics
Step cost $c(i, j)$	Stage cost $\ell(\mathbf{x}_k, \mathbf{u}_k)$	Penalty for this step
Cost-to-go $V(i, j)$	Value function $V_k(\mathbf{x})$	Optimal future cost
Bellman recursion	Bellman equation	The DP update rule
Fill table backward	Backward Riccati recursion	How we solve it
Optimal action at (i, j)	Optimal policy $\mathbf{u}_k^* = \pi(\mathbf{x}_k)$	What to do at each state

It is *literally the same algorithm.* The only difference is that in control, the "grid" is continuous (states are real numbers, not integers), and the "decisions" are real-valued control inputs instead of discrete left/right choices.

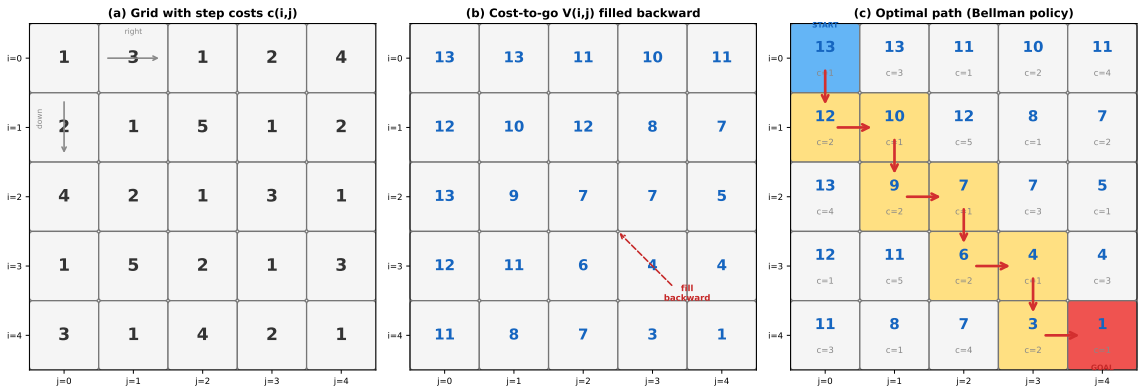

Figure 17: Grid-world DP example (5×5, moves: right or down). (a) Each cell has a step cost $c(i, j)$. (b) The cost-to-go $V(i, j)$ is filled *backward* from the goal — this is the DP table (= Riccati recursion in control). (c) The optimal path (total cost = 13) follows the Bellman policy: at each cell, go toward the neighbor with the smaller V. Replace "cell" with "state" and "right/down" with "control input" and you have LQR.

The Bellman equation for optimal control (discrete time):

Given a system $\mathbf{x}_{k+1} = f(\mathbf{x}_k, \mathbf{u}_k)$ with stage cost $\ell(\mathbf{x}_k, \mathbf{u}_k)$ and terminal cost $\ell_f(\mathbf{x}_N)$, the optimal cost-to-go (value function) satisfies:

$$V_k(\mathbf{x}) = \min_{\mathbf{u}} \left[\ell(\mathbf{x}, \mathbf{u}) + V_{k+1}\left(f(\mathbf{x}, \mathbf{u})\right)\right] \tag{68}$$

with boundary condition $V_N(\mathbf{x}) = \ell_f(\mathbf{x})$.
The optimal control at step k is:

$$\mathbf{u}_k^*(\mathbf{x}) = \arg\min_{\mathbf{u}} \left[\ell(\mathbf{x}, \mathbf{u}) + V_{k+1}\left(f(\mathbf{x}, \mathbf{u})\right)\right] \tag{69}$$

This is solved backward in time, from $k = N$ to $k = 0$, just like filling the DP table from the goal to the start.

From Grid-World to LQR: The "Continuous Grid" Problem

Now imagine the grid-world becomes continuous. Instead of cells, your "position" is a real-valued state $\mathbf{x} \in \mathbb{R}^n$. Instead of right/down, your "move" is a real-valued control $\mathbf{u} \in \mathbb{R}^m$. The transition is linear: $\mathbf{x}_{k+1} = A\mathbf{x}_k + B\mathbf{u}_k$. The step cost is quadratic: $\ell(\mathbf{x}, \mathbf{u}) = \mathbf{x}^T Q\mathbf{x} + \mathbf{u}^T R\mathbf{u}$.

Plug into the Bellman equation:

$$V_k(\mathbf{x}) = \min_{\mathbf{u}} \left[\mathbf{x}^T Q\mathbf{x} + \mathbf{u}^T R\mathbf{u} + V_{k+1}(A\mathbf{x} + B\mathbf{u})\right]$$

Here comes the magic: because the dynamics are linear and the cost is quadratic, the value function is **also quadratic**: $V_k(\mathbf{x}) = \mathbf{x}^T P_k \mathbf{x}$ for some matrix P_k. Substituting this ansatz and taking the derivative with respect to $\mathbf{u}$:

$$\frac{\partial}{\partial \mathbf{u}} \left[\mathbf{u}^T R\mathbf{u} + (A\mathbf{x} + B\mathbf{u})^T P_{k+1}(A\mathbf{x} + B\mathbf{u})\right] = 0$$

Solving gives:

$$\mathbf{u}_k^* = -\underbrace{(R + B^T P_{k+1} B)^{-1} B^T P_{k+1} A}_{K_k}\, \mathbf{x}_k \tag{70}$$

$$P_k = Q + A^T P_{k+1} A - A^T P_{k+1} B (R + B^T P_{k+1} B)^{-1} B^T P_{k+1} A \tag{71}$$

This backward recursion for P_k is the **discrete Riccati equation** — exactly the "fill the DP table backward" step. When the horizon $N \to \infty$, P_k converges to a constant P, and we get the steady-state **LQR** gain $K = (R + B^T PB)^{-1} B^T PA$.

In the grid-world analogy:

- P_k is the cost-to-go matrix — it tells you the "remaining cost" from any state, just like $V(i, j)$ in the grid.
- K_k is the optimal policy at step k — it tells you what to do at each state, just like "go right if $V(i + 1, j) < V(i, j + 1)$."
- The Riccati recursion is the DP table fill — same $O(N)$ backward pass.

Connection to MPC: MPC is the same Bellman/DP framework, but with two additions:

1. **Constraints** on states and inputs (like adding walls to the grid-world — some cells are forbidden).
2. **Receding horizon**: instead of solving once from $k = 0$ to N, re-solve at every timestep using the latest measurement as the new "start position."

The constraints make the Bellman equation harder to solve (the min is now a constrained optimization), which is why MPC needs a QP solver or the ADMM approach of TinyMPC. But the underlying principle is identical: backward recursion to find the optimal cost-to-go, forward application of the optimal policy.

Connection to reinforcement learning: If you are familiar with RL, the Bellman equation is exactly the same as the **Bellman optimality equation** in RL ($V^*(s) = \max_a[r(s, a) + \gamma V^*(s')]$). LQR is the "closed-form RL" for linear-quadratic problems. When the dynamics are unknown or nonlinear, RL methods (Q-learning, policy gradient) approximate the solution that LQR computes exactly.

6.3.2 LQR (Linear Quadratic Regulator)

With the Bellman/DP framework fresh in mind, LQR is simply the steady-state solution to the linear-quadratic Bellman equation. Instead of manually placing poles, you specify a cost function that balances performance against effort:

$$J = \int_0^\infty \left(\mathbf{x}^T Q \mathbf{x} + \mathbf{u}^T R \mathbf{u}\right) dt \tag{72}$$

- Q: penalizes state deviations (large Q = aggressive tracking, fast response).
- R: penalizes control effort (large R = gentle inputs, energy saving).

The optimal state feedback gain is:

$$K = R^{-1} B^T P \tag{73}$$

where P is the solution to the **continuous algebraic Riccati equation (CARE)**:

$$A^T P + PA - PBR^{-1}B^T P + Q = 0 \tag{74}$$

Do not solve this by hand. Use `lqr(A, B, Q, R)` in MATLAB or `scipy.linalg.solve_continuous_are()` in Python.

What is the Riccati equation actually computing? The matrix P represents the **optimal cost-to-go**: P_{ij} tells you how much "future cost" is associated with the interaction between states i and j. Think of it as a map of "how expensive is it to be in each state?" States that are hard to control (require lots of effort to bring back to zero) get large entries in P. The optimal gain $K = R^{-1}B^T P$ then says: "Apply control effort proportional to how costly each state is." Expensive states get corrected aggressively; cheap states get gentle treatment.

Tuning LQR: The art is in choosing Q and R.

- Start with $Q = I$ (identity matrix) and $R = \rho I$ where ρ is a scalar.
- Small ρ: aggressive, fast, uses lots of actuator effort.
- Large ρ: conservative, slow, gentle on actuators.
- To penalize specific states more, increase the corresponding diagonal elements of Q. For example, if you care more about position than velocity, make $Q_{11} > Q_{22}$.

Why LQR is great for competitions:

- Guaranteed stability (if the system is controllable).
- Guaranteed robustness margins (at least 6 dB gain margin and 60° phase margin for SISO systems).
- Handles MIMO systems effortlessly.
- The Q/R tuning is more intuitive than PID's three gains for multi-axis systems.

Example: Balancing a two-wheeled robot. The state is $\mathbf{x} = [\theta, \dot{\theta}, x, \dot{x}]^T$ (tilt angle, tilt rate, position, velocity). PID would need multiple nested loops; LQR handles it with a single K matrix.

Limitation: LQR requires full state feedback ($\mathbf{u} = -K\mathbf{x}$), which means you need to know all states. If you cannot measure all states, you need an observer (see Section 6.4).

6.3.3 A Brief Primer on Optimization and Convexity

MPC solves an optimization problem at every time step. Before diving in, let us establish the minimum optimization vocabulary you need.

The general optimization problem:

$$\min_{\mathbf{x}} \; f(\mathbf{x}) \quad \text{subject to} \quad g_i(\mathbf{x}) \leq 0, \;\; h_j(\mathbf{x}) = 0$$

where f is the **objective function** (cost), g_i are **inequality constraints**, and h_j are **equality constraints**. The optimizer searches for the decision variable $\mathbf{x}$ that minimizes f without violating any constraint.

What makes a problem convex? A problem is convex if:

1. The objective f is a **convex function**: for any two points $\mathbf{a}, \mathbf{b}$ and $0 \leq \lambda \leq 1$,

$$f\big(\lambda \mathbf{a} + (1-\lambda)\mathbf{b}\big) \leq \lambda f(\mathbf{a}) + (1-\lambda) f(\mathbf{b})$$

   Geometrically: the function lies below any chord. Quadratic functions $f(\mathbf{x}) = \mathbf{x}^T Q \mathbf{x}$ with $Q \succeq 0$ (positive semidefinite) are convex.
2. The **feasible set** (defined by the constraints) is convex: any line segment between two feasible points stays feasible. Linear inequalities ($A\mathbf{x} \leq \mathbf{b}$) always define convex sets.

Why convexity matters: A convex problem has a single global minimum—no local minima traps. Any solver that finds a solution has found *the* solution. Non-convex problems can have many local minima, and solvers may get stuck in a bad one.

Quadratic Programs (QP): The most common optimization problem in control. The objective is quadratic, the constraints are linear:

$$\min_{\mathbf{x}} \ \tfrac{1}{2}\mathbf{x}^T H \mathbf{x} + \mathbf{c}^T \mathbf{x} \quad \text{s.t.} \quad A\mathbf{x} \leq \mathbf{b}$$

With $H \succeq 0$, this is convex and can be solved efficiently even on embedded systems (TinyMPC uses this structure). LQR without constraints is an *unconstrained* QP whose solution is the Riccati equation. MPC with box constraints on inputs and states is a *constrained* QP.

Connection to control: Every optimal control problem we encounter in this chapter—LQR, MPC, Kalman filter (dual of LQR)—is a convex QP. This is not a coincidence: the linear dynamics and quadratic cost were chosen *precisely because* they make the problem convex and therefore tractable in real time.

6.3.4 MPC (Model Predictive Control)

MPC is the most powerful and flexible controller in the modern control toolbox. It solves an optimization problem at every time step:

"Given where I am now, what sequence of inputs over the next N steps minimizes my cost function while respecting all constraints?"

At each time step:

1. Measure (or estimate) the current state $\mathbf{x}[k]$.
2. Solve the optimization problem:

$$\min_{\mathbf{u}[k],\ldots,\mathbf{u}[k+N-1]} \sum_{i=0}^{N-1} \big(\mathbf{x}[k+i]^T Q \mathbf{x}[k+i] + \mathbf{u}[k+i]^T R \mathbf{u}[k+i]\big) + \mathbf{x}[k+N]^T P_f \mathbf{x}[k+N] \tag{75}$$

   subject to:

$$\mathbf{x}[k+i+1] = A_d \mathbf{x}[k+i] + B_d \mathbf{u}[k+i] \tag{76}$$
$$\mathbf{u}_{\min} \leq \mathbf{u}[k+i] \leq \mathbf{u}_{\max} \tag{77}$$
$$\mathbf{x}_{\min} \leq \mathbf{x}[k+i] \leq \mathbf{x}_{\max} \tag{78}$$

3. Apply only the first input $\mathbf{u}[k]$.
4. Repeat at the next time step (receding horizon).

Understanding the cost function terms:

- $\mathbf{x}^T Q \mathbf{x}$: **Stage cost on states.** Penalizes deviation from the desired state at each step. Larger Q_{ii} means "I care more about keeping state i close to zero (or the reference)."
- $\mathbf{u}^T R \mathbf{u}$: **Stage cost on inputs.** Penalizes control effort. Larger R means "go easy on the actuators." This prevents the optimizer from finding mathematically optimal but physically violent solutions.
- $\mathbf{x}[k+N]^T P_f \mathbf{x}[k+N]$: **Terminal cost.** Penalizes the state at the end of the prediction horizon. Without this, MPC can "cheat" by pushing all the error to the end of the horizon (like a student who does nothing until the last day). Setting P_f to the solution of the LQR Riccati equation is a common choice that guarantees stability.

- N (**prediction horizon**): How many steps ahead MPC looks. Longer horizon = better anticipation but heavier computation. Too short = myopic behavior (MPC cannot "see" upcoming turns or obstacles). A good starting point: $N = 10$–30 steps.

The receding horizon concept: MPC computes an *entire* optimal trajectory but only executes the first step, then replans. This sounds wasteful, but it is what makes MPC robust: at each step, it uses the latest measurement to correct for model errors and disturbances. It is like a chess player who plans 10 moves ahead but re-evaluates after each opponent's move.

Why MPC is powerful:

- **Handles constraints explicitly.** Motor current limits, joint angle limits, velocity bounds—they are all part of the optimization.
- **Handles MIMO systems naturally.** Multiple inputs and outputs are just more variables in the optimization.
- **Preview capability.** If you know the future reference trajectory (e.g., a planned path), MPC can look ahead and act proactively.
- **Intuitive tuning.** Like LQR, you tune Q and R. The prediction horizon N is the main additional parameter.

The catch: MPC requires solving an optimization problem at every time step. For linear systems with quadratic costs (linear MPC), this is a quadratic program (QP) that can be solved efficiently. For nonlinear systems (nonlinear MPC), it is much harder.

Practical considerations for competition robots:

- For simple systems (chassis velocity control), PID or LQR is usually sufficient. MPC is overkill.
- For trajectory tracking with constraints (autonomous navigation, robotic arms), MPC is worth the effort.
- Libraries like OSQP, qpOASES, or acados can solve QPs fast enough for real-time embedded use.
- If real-time computation is too expensive, consider *explicit MPC*—precomputing the solution offline as a lookup table.

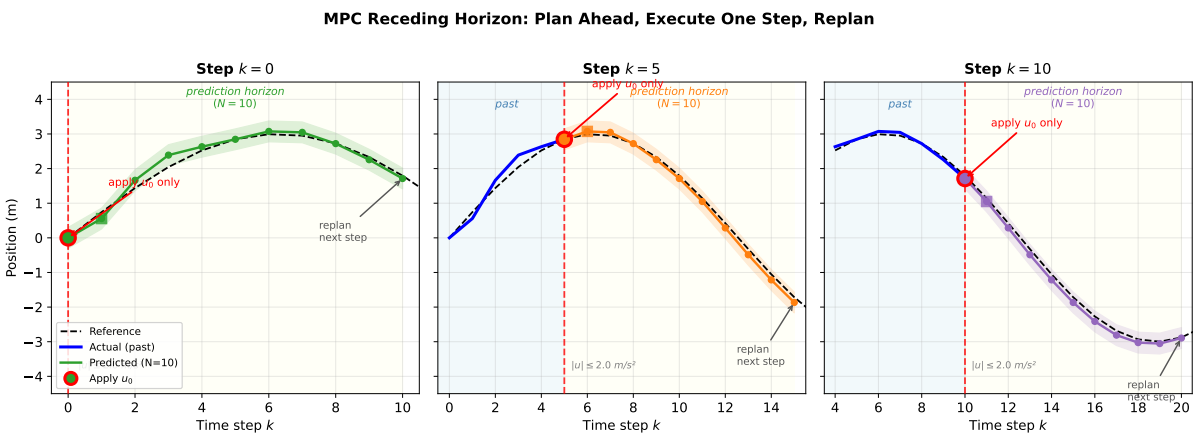

Figure 18: MPC receding horizon illustrated on a position-tracking problem. At each timestep, MPC plans an optimal trajectory over the next $N = 10$ steps (green), but only applies the **first** control action (circled). Then it re-measures, replans, and repeats. The prediction window slides forward with each step—this "plan ahead, execute one, replan" cycle is what makes MPC robust to model errors and disturbances.

6.3.5 TinyMPC: Riccati-Accelerated MPC for Embedded Systems

Standard MPC solves a full QP (quadratic program) at every timestep. For a system with n states, m inputs, and horizon N, this means solving a problem with $N(n+m)$ decision variables — feasible on a laptop but brutal on a microcontroller. **TinyMPC** (Nguyen et al., 2024) is a breakthrough that makes MPC run on resource-constrained embedded systems (Teensy, STM32, even Arduino) by exploiting the Riccati equation structure.

The key insight: For an unconstrained linear MPC, the solution is just a time-varying LQR — you can compute it via a backward Riccati recursion. TinyMPC precomputes this Riccati recursion **offline**, then handles constraints **online** with a lightweight ADMM (Alternating Direction Method of Multipliers) loop.

Algorithm structure:

Step 1 — Offline precomputation (once, or when model changes):

Run the backward Riccati recursion from the terminal cost $P_N = Q_f$ to compute the optimal feedback gains at each step:

$$S_k = R + B_d^T P_{k+1} B_d \tag{79}$$

$$K_k = S_k^{-1} B_d^T P_{k+1} A_d \tag{80}$$

$$P_k = Q + A_d^T P_{k+1} A_d - A_d^T P_{k+1} B_d K_k \tag{81}$$

This gives you N feedback gain matrices $K_0, K_1, \ldots, K_{N-1}$ and $N+1$ cost-to-go matrices $P_0, P_1, \ldots, P_N$. These are stored in memory and *never recomputed* (unless the model or cost matrices change).

Step 2 — Online solve (every control cycle):

Given the current state $\mathbf{x}_0$:

1. **Forward rollout:** Compute the unconstrained optimal trajectory using the precomputed gains: $\mathbf{u}_k = -K_k(\mathbf{x}_k - \mathbf{x}_k^{\text{ref}})$, then $\mathbf{x}_{k+1} = A_d\mathbf{x}_k + B_d\mathbf{u}_k$.
2. **ADMM projection:** If any $\mathbf{u}_k$ or $\mathbf{x}_k$ violates constraints, run a few ADMM iterations (typically 5–20) to find the nearest feasible solution. Each ADMM iteration is just:
   - A backward Riccati-like pass (using the *precomputed* K_k, so it is cheap).
   - A projection step: $z_k = \text{clamp}(u_k + \lambda_k, u_{\min}, u_{\max})$ — element-wise clamping.
   - A dual update: $\lambda_k \leftarrow \lambda_k + u_k - z_k$.
3. **Apply first input:** Send $\mathbf{u}_0$ to the actuator. Discard the rest (receding horizon).

Why this is so fast:

- The Riccati recursion (the expensive part) is done **offline**. Online work is just matrix multiplies and clamping.
- No QP solver library needed. No matrix factorization online. Just multiply-add and clamp.
- Each ADMM iteration is $O(N \cdot n^2)$ — linear in horizon, quadratic in state dimension. For a 4-state system with $N = 10$: about 1600 multiply-adds per iteration. At 10 iterations, that is 16,000 multiply-adds — an STM32F4 at 168 MHz finishes this in $< 100\,\mu$s.
- Memory usage: $N \times (n \times n + m \times n)$ floats for the precomputed gains. For 4 states, 2 inputs, $N = 10$: about 240 floats = 960 bytes.

Comparison with standard QP-based MPC:

	Standard MPC (OSQP)	**TinyMPC**
Online computation	Full QP solve	Riccati rollout + ADMM
Typical solve time (STM32)	1–10 ms	50–200 μs
Memory	KBs (sparse matrices)	$<$ 1 KB for small systems
Dependencies	QP solver library	None (just matrix ops)
Constraint handling	Exact (to tolerance)	Approximate (ADMM)
Warm starting	Possible	Natural (previous solution)

When to use TinyMPC:

- You need MPC on a microcontroller with limited RAM/CPU.
- Your model is linear (or can be linearized at each step for nonlinear systems).
- Hard constraints on inputs are important (motor current limits, joint angle limits).
- Horizon is short to moderate ($N \leq 30$).

When NOT to use TinyMPC:

- Complex nonlinear models (use acados or CasADi on a more powerful computer).
- Very long horizons ($N > 50$) with many state constraints.
- If your problem is unconstrained — just use LQR, it is even simpler.

Reference: The full TinyMPC paper and open-source implementation are at `tinympc.org`. The C implementation fits in under 1000 lines and runs on ARM Cortex-M microcontrollers. Our C++ module in the Appendix follows the same Riccati + ADMM structure.

6.4 Observer Design via State-Space

LQR (and state feedback in general) requires knowing all states. But in practice, you can only measure some of them. An **observer** (or **estimator**) reconstructs the full state vector from the available measurements.

6.4.1 Luenberger Observer

The Luenberger observer is a copy of the system model, with a correction term that uses the measurement error to drive the estimate toward the true state.

$$\dot{\hat{\mathbf{x}}} = A\hat{\mathbf{x}} + B\mathbf{u} + L(\mathbf{y} - C\hat{\mathbf{x}}) \tag{82}$$

where:
- $\hat{\mathbf{x}}$ is the estimated state.
- $\mathbf{y} = C\mathbf{x}$ is the actual measurement.
- L is the observer gain matrix (we design this).

The estimation error $\mathbf{e} = \mathbf{x} - \hat{\mathbf{x}}$ evolves as:

$$\dot{\mathbf{e}} = (A - LC)\mathbf{e} \tag{83}$$

If we choose L such that all eigenvalues of $(A - LC)$ have negative real parts, the estimation error decays to zero. The observer "catches up" to the true state.

Design rule of thumb: Place the observer poles 2–5× faster than the controller poles. The observer should converge faster than the controller acts, so the controller always has a good state estimate.

Separation principle: You can design the controller gain K and observer gain L independently. The combined controller-observer system is stable if each part is stable individually. This is a beautiful theoretical result that simplifies design enormously.

When to use Luenberger: When your model is accurate and the noise characteristics are not well known. It is simple, deterministic, and easy to debug.

6.4.2 The Deep Duality: Controller and Observer as Mirror Images

This is one of the most elegant results in all of control theory, and understanding it deeply will make you a better engineer. Let us lay out the parallel structure explicitly.

The controller problem: Given the system $\dot{\mathbf{x}} = A\mathbf{x} + B\mathbf{u}$, find a gain K such that $\mathbf{u} = -K\mathbf{x}$ makes the closed-loop system $\dot{\mathbf{x}} = (A - BK)\mathbf{x}$ stable and well-behaved.

The observer problem: Given the system $\dot{\mathbf{x}} = A\mathbf{x} + B\mathbf{u}$, $\mathbf{y} = C\mathbf{x}$, find a gain L such that the estimation error $\dot{\mathbf{e}} = (A - LC)\mathbf{e}$ decays to zero.

Now stare at the two closed-loop matrices side by side:

	Controller	Observer
Closed-loop matrix	$A - BK$	$A - LC$
Gain to design	K	L
Required condition	Controllability (A, B)	Observability (A, C)
Design goal	Place eigenvalues of $A - BK$	Place eigenvalues of $A - LC$
Information flow	Input $\to$ states	Measurements $\to$ state estimates
Riccati equation	$A^T P + PA - PBR^{-1}B^T P + Q = 0$	$AP_o + P_o A^T - P_o C^T R_n^{-1} C P_o + Q_n = 0$

The mathematical duality: If you transpose the observer Riccati equation, replacing $A \to A^T$, $C^T \to B$, $Q_n \to Q$, $R_n \to R$, you get exactly the controller Riccati equation. They are literally the same equation with transposed matrices. This means:

- Any algorithm that designs a controller can design an observer (just transpose the inputs).
- If you can place poles for a controller using `place(A, B, poles)`, you can place observer poles using `place(A', C', poles)'` — note the transposes.
- The LQR controller (K from the control Riccati) has a dual: the **Kalman filter** (L from the observer Riccati). LQR is the optimal controller; Kalman is the optimal observer. They are dual problems.

A physical analogy: Think of the controller as a *speaker* and the observer as a *microphone*. The speaker (controller) pushes energy into the system through the inputs B to shape the state trajectory. The microphone (observer) listens to the system through the outputs C to reconstruct what is happening inside. Controllability asks: "Can the speaker reach every corner of the room?" Observability asks: "Can the microphone hear every corner of the room?" They are symmetric questions about the same room (system).

Why this matters in practice: If you have already tuned a state feedback controller and it works well, you have a strong intuition for how pole placement affects performance. That *exact same intuition* applies when designing an observer. Faster poles = faster convergence but more noise sensitivity. Slower poles = smoother estimates but slower tracking. You are turning the same knob, just on the other side of the mirror.

6.4.3 Kalman Filter

The Kalman filter is the optimal observer for linear systems with Gaussian noise. It is the Luenberger observer's probabilistic cousin—instead of placing observer poles manually, it computes the optimal gain L based on the noise statistics.

System model with noise:

$$\mathbf{x}[k+1] = A_d\mathbf{x}[k] + B_d\mathbf{u}[k] + \mathbf{w}[k], \quad \mathbf{w} \sim \mathcal{N}(0, Q_n) \tag{84}$$

$$\mathbf{y}[k] = C\mathbf{x}[k] + \mathbf{v}[k], \quad \mathbf{v} \sim \mathcal{N}(0, R_n) \tag{85}$$

where $\mathbf{w}$ is process noise (model uncertainty) and $\mathbf{v}$ is measurement noise. Q_n and R_n are their covariance matrices.

The Kalman filter algorithm (discrete):

Predict:

$$\hat{\mathbf{x}}[k|k-1] = A_d\hat{\mathbf{x}}[k-1|k-1] + B_d\mathbf{u}[k-1] \tag{86}$$

$$P[k|k-1] = A_d P[k-1|k-1]A_d^T + Q_n \tag{87}$$

Update:

$$K_k = P[k|k-1]C^T(CP[k|k-1]C^T + R_n)^{-1} \tag{88}$$

$$\hat{\mathbf{x}}[k|k] = \hat{\mathbf{x}}[k|k-1] + K_k(\mathbf{y}[k] - C\hat{\mathbf{x}}[k|k-1]) \tag{89}$$

$$P[k|k] = (I - K_k C)P[k|k-1] \tag{90}$$

Intuition: The Kalman gain K_k automatically balances trust between the model prediction and the measurement:

- If process noise is large (Q_n big) relative to measurement noise (R_n small), the filter trusts the measurement more.
- If measurement noise is large relative to process noise, the filter trusts the model more.

Tuning the Kalman filter comes down to choosing Q_n and R_n:

- R_n can often be measured directly (record sensor data when the system is stationary, compute the variance).
- Q_n is harder—it represents model uncertainty. Start with small values and increase if the filter is too sluggish (not tracking fast enough).
- A common cheat: treat Q_n and R_n as tuning knobs. Increase Q_n/R_n ratio for faster tracking (more noise). Decrease for smoother estimates (more lag). Sounds familiar? It is the same trade-off as low-pass filter cutoff frequency, just expressed probabilistically.

Why the Kalman filter is everywhere in robotics:

- **Sensor fusion:** Combine IMU, encoder, and vision data into one coherent state estimate.
- **Velocity estimation:** Get smooth velocity from noisy position measurements without a pure differentiator.
- **Optimal in a well-defined sense:** It minimizes the mean-squared estimation error for linear Gaussian systems.

A practical note: The Kalman filter needs a matrix inverse at each step (the K_k computation). For small state dimensions ($n \leq 6$), this is trivial. For larger systems, use efficient implementations (e.g., Cholesky decomposition). On an STM32F4, a 6-state EKF runs comfortably at 1 kHz.

6.4.4 Extended Kalman Filter (EKF)

The standard Kalman filter assumes linear dynamics ($\mathbf{x}_{k+1} = A\mathbf{x}_k + B\mathbf{u}_k$) and linear measurements ($\mathbf{y}_k = C\mathbf{x}_k$). But real robotics systems are **nonlinear**: the relationship between Euler angles and angular rates involves trigonometric functions, the accelerometer measurement model depends on the sine/cosine of the tilt angles, and so on.

The **Extended Kalman Filter** handles nonlinearity by *linearizing* the system at each timestep around the current estimate. The idea is simple: at each step, pretend the system is locally linear, apply the standard Kalman filter, then re-linearize at the new estimate.

Nonlinear system model:

$$\mathbf{x}_{k+1} = f(\mathbf{x}_k, \mathbf{u}_k) + \mathbf{w}_k \quad \text{(nonlinear dynamics)} \tag{91}$$

$$\mathbf{y}_k = h(\mathbf{x}_k) + \mathbf{v}_k \quad \text{(nonlinear measurement)} \tag{92}$$

EKF algorithm:
Predict:

$$\hat{\mathbf{x}}_{k|k-1} = f(\hat{\mathbf{x}}_{k-1|k-1}, \mathbf{u}_{k-1}) \tag{93}$$

$$F_k = \left.\frac{\partial f}{\partial \mathbf{x}}\right|_{\hat{\mathbf{x}}_{k-1|k-1}} \quad \text{(Jacobian of dynamics)} \tag{94}$$

$$P_{k|k-1} = F_k P_{k-1|k-1} F_k^T + Q \tag{95}$$

Update:

$$H_k = \left.\frac{\partial h}{\partial \mathbf{x}}\right|_{\hat{\mathbf{x}}_{k|k-1}} \quad \text{(Jacobian of measurement)} \tag{96}$$

$$K_k = P_{k|k-1} H_k^T (H_k P_{k|k-1} H_k^T + R)^{-1} \tag{97}$$

$$\hat{\mathbf{x}}_{k|k} = \hat{\mathbf{x}}_{k|k-1} + K_k \left(\mathbf{y}_k - h(\hat{\mathbf{x}}_{k|k-1})\right) \tag{98}$$

$$P_{k|k} = (I - K_k H_k) P_{k|k-1} \tag{99}$$

The only difference from the standard Kalman filter:

1. The prediction uses the *nonlinear* function $f(\cdot)$ instead of the matrix A.
2. The measurement innovation uses the *nonlinear* function $h(\cdot)$ instead of the matrix C.
3. The covariance propagation uses the *Jacobians* F_k and H_k (evaluated at the current estimate) instead of the constant matrices A and C.

Everything else — the Kalman gain formula, the covariance update, the predict-update cycle — is *identical*.

Case Study: 6-DOF IMU Pose Estimation with EKF

This is the single most common EKF application in competition robotics: estimating the 3D orientation (roll, pitch, yaw) of your robot from a 6-axis IMU (3-axis accelerometer + 3-axis gyroscope).

State vector: $\mathbf{x} = [\phi, \theta, \psi, b_x, b_y, b_z]^T$

- ϕ: roll, θ: pitch, ψ: yaw (Euler angles in radians)
- b_x, b_y, b_z: gyroscope biases (rad/s) — the EKF estimates these automatically

Dynamics model $f(\mathbf{x}, \mathbf{u})$: The angular rates from the gyroscope (after bias subtraction) drive the Euler angles via the kinematic equations:

$$\omega_x = \text{gyro}_x - b_x, \quad \omega_y = \text{gyro}_y - b_y, \quad \omega_z = \text{gyro}_z - b_z \tag{100}$$

$$\dot{\phi} = \omega_x + \sin\phi \tan\theta \cdot \omega_y + \cos\phi \tan\theta \cdot \omega_z \tag{101}$$

$$\dot{\theta} = \cos\phi \cdot \omega_y - \sin\phi \cdot \omega_z \tag{102}$$

$$\dot{\psi} = \frac{\sin\phi}{\cos\theta} \cdot \omega_y + \frac{\cos\phi}{\cos\theta} \cdot \omega_z \tag{103}$$

$$\dot{b}_x = 0, \quad \dot{b}_y = 0, \quad \dot{b}_z = 0 \quad \text{(biases modeled as random walk)} \tag{104}$$

The Jacobian $F = \frac{\partial f}{\partial \mathbf{x}}$ is a 6×6 matrix with trigonometric entries. It must be recomputed at every timestep (this is the "extended" part).

Measurement model $h(\mathbf{x})$: The accelerometer measures gravity direction, which gives us roll and pitch (but NOT yaw):

$$h(\mathbf{x}) = \begin{bmatrix} \phi \\ \theta \end{bmatrix} = \begin{bmatrix} \arctan 2(a_y, a_z) \\ \arctan 2(-a_x, \sqrt{a_y^2 + a_z^2}) \end{bmatrix} \tag{105}$$

The Jacobian is simple: $H = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \end{bmatrix}$.

Key design decisions:

- **Yaw is unobservable** from a 6-axis IMU alone (gravity does not tell you which direction is north). Yaw relies entirely on the gyroscope. To correct yaw drift, add a magnetometer (9-DOF IMU) or external heading reference.
- Q **matrix tuning:** The diagonal entries for angles (Q_{00}, Q_{11}, Q_{22}) represent how much the gyro integration can drift per step. Larger = trust gyro less. The bias entries (Q_{33}, Q_{44}, Q_{55}) represent bias random walk rate — typically very small (10^{-4} to 10^{-6}).
- R **matrix tuning:** Represents accelerometer noise. During vibration or linear acceleration, the accelerometer readings are corrupted. Increase R during high-acceleration maneuvers (adaptive EKF) to avoid trusting bad accelerometer data.
- **Gimbal lock:** When pitch = $\pm 90°$, the $\tan\theta$ and $1/\cos\theta$ terms in the kinematic equations blow up. For robots that can tilt past $\pm 60°$, use **quaternion representation** instead of Euler angles (same EKF structure, different state parameterization).

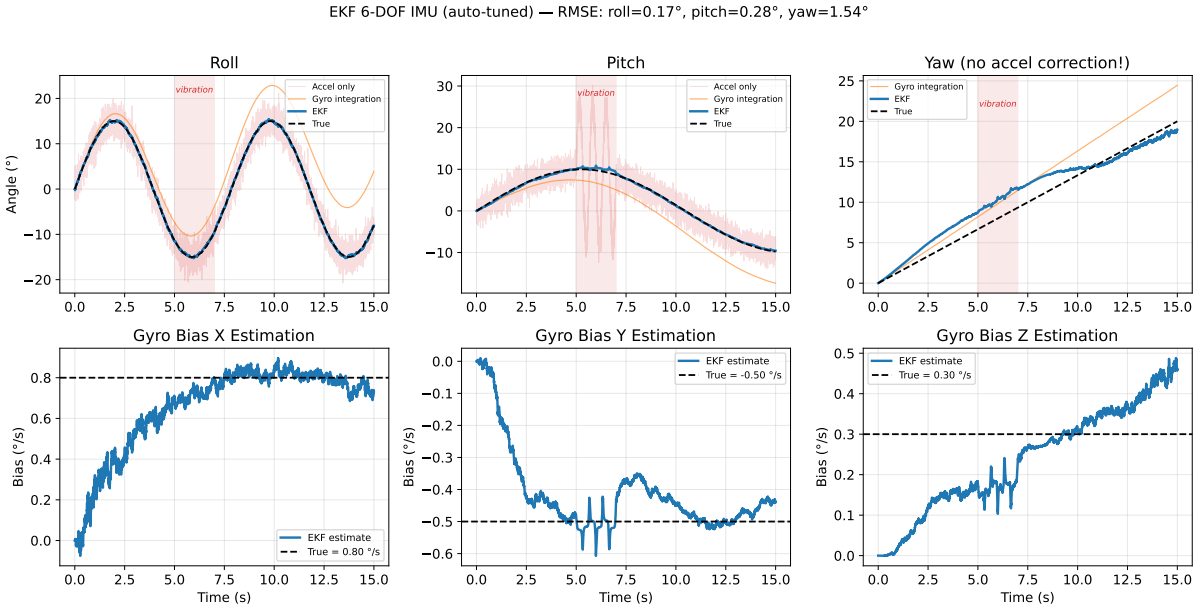

Figure 19: EKF 6-DOF IMU pose estimation with auto-tuned continuous adaptive R. Uses smooth exponential scaling $R = R_{\text{base}} \cdot (1 + \alpha\, e^{\beta \cdot \Delta a})$ with separate R_{base} for roll and pitch, plus pitch-specific x-axis accel deviation detection. Parameters found by offline grid search minimizing weighted RMSE. Top row: roll (RMSE 0.17°), pitch (0.28°), yaw (1.54°). During vibration ($t = 5$–7s, shaded), the adaptive R scales up exponentially, rejecting corrupted accelerometer data. Yaw drift is bounded by bias estimation alone (no heading reference). Bottom row: gyroscope biases converge smoothly to true values.

Beyond EKF: The EKF's linearization can be inaccurate when the state uncertainty is large or the nonlinearity is strong. Alternatives:

- **Unscented Kalman Filter (UKF):** Instead of linearizing, propagates "sigma points" (a small set of carefully chosen sample states) through the nonlinear function. More accurate than EKF for highly nonlinear systems, slightly more expensive ($2n+1$ function evaluations per step where n is the state dimension).
- **Error-State Kalman Filter (ESKF):** Estimates a *small error* around a nominal trajectory instead of the full state. Popular for IMU-based systems because the error state stays small (linearization is accurate) and quaternion singularities are avoided. This is what most production-grade IMU fusion libraries use internally.
- **Complementary filter:** For simple roll/pitch estimation where computational resources are extremely limited, the complementary filter (Section 2) remains a viable one-line alternative.

6.4.5 Quaternion-Based EKF: Avoiding Gimbal Lock

The Euler-angle EKF above works well when pitch stays within ±60°. But when pitch approaches ±90°, the kinematic equations blow up: $\tan\theta \to \infty$ and $1/\cos\theta \to \infty$. This is **gimbal lock**—a mathematical singularity, not a physical one. The robot is fine; your math is broken.

The fix is to represent orientation with a **unit quaternion** $\mathbf{q} = \begin{bmatrix} q_w & q_x & q_y & q_z \end{bmatrix}^T$ instead of Euler angles. Quaternions have no singularities, are compact (4 numbers vs 9 for a rotation matrix), and compose rotations efficiently.

State vector: $\mathbf{x} = \begin{bmatrix} \mathbf{q}^T & \boldsymbol{\beta}^T \end{bmatrix}^T \in \mathbb{R}^7$ where $\boldsymbol{\beta} = \begin{bmatrix} \beta_x & \beta_y & \beta_z \end{bmatrix}^T$ are the gyroscope biases.

Prediction step: Given gyroscope measurement $\boldsymbol{\omega}_m$, the bias-corrected angular velocity is $\boldsymbol{\omega} = \boldsymbol{\omega}_m - \boldsymbol{\beta}$. The quaternion kinematics are:

$$\dot{\mathbf{q}} = \frac{1}{2}\mathbf{q} \otimes \begin{bmatrix} 0 \\ \boldsymbol{\omega} \end{bmatrix} = \frac{1}{2}\Omega(\boldsymbol{\omega})\,\mathbf{q} \tag{106}$$

where $\otimes$ is quaternion multiplication and $\Omega(\boldsymbol{\omega})$ is the 4×4 skew matrix:

$$\Omega(\boldsymbol{\omega}) = \begin{bmatrix} 0 & -\omega_x & -\omega_y & -\omega_z \\ \omega_x & 0 & \omega_z & -\omega_y \\ \omega_y & -\omega_z & 0 & \omega_x \\ \omega_z & \omega_y & -\omega_x & 0 \end{bmatrix} \tag{107}$$

Discrete-time prediction (first-order):

$$\mathbf{q}_{k+1} = \left(I + \frac{\Delta t}{2} \Omega(\boldsymbol{\omega}_k) \right) \mathbf{q}_k, \qquad \boldsymbol{\beta}_{k+1} = \boldsymbol{\beta}_k \tag{108}$$

After prediction, **normalize q** to unit length: $\mathbf{q} \leftarrow \mathbf{q}/\|\mathbf{q}\|$.

Measurement model: The accelerometer measures gravity in the body frame. Given quaternion $\mathbf{q}$, the predicted gravity direction in body frame is:

$$\mathbf{g}_{\text{pred}} = R(\mathbf{q})^T \begin{bmatrix} 0 \\ 0 \\ g \end{bmatrix} = g \begin{bmatrix} 2(q_x q_z - q_w q_y) \\ 2(q_y q_z + q_w q_x) \\ q_w^2 - q_x^2 - q_y^2 + q_z^2 \end{bmatrix} \tag{109}$$

The measurement Jacobian $H = \partial \mathbf{g}_{\text{pred}} / \partial \mathbf{x}$ is a 3×7 matrix (3 accel components, 4 quaternion + 3 bias states). The bias columns are zero since accelerometer readings do not depend on gyro biases directly.

Update step: Standard EKF update applies. The innovation is $\boldsymbol{\nu} = \mathbf{a}_{\text{meas}} - \mathbf{g}_{\text{pred}}$, and after the update, normalize $\mathbf{q}$ again.

Euler Angles vs Quaternions: When to Use Which

- **Euler angles** are fine if your robot operates within $\pm 60°$ pitch (ground vehicles, most turrets, most drones in normal flight). The math is more intuitive and easier to debug.
- **Quaternions** are necessary for robots that can flip, tumble, or point straight up/down (acrobatic drones, underwater ROVs, robot arms near wrist singularities).
- **Tip:** You can use quaternions internally for all computation and convert to Euler angles only for display and human-readable logging: $\phi = \text{atan2}(2(q_w q_x + q_y q_z),\ 1 - 2(q_x^2 + q_y^2))$, etc.
- **The Error-State approach** (ESKF) avoids working with the 4-element quaternion directly in the EKF state. Instead, it estimates a 3-element rotation error $\delta\boldsymbol{\theta}$ and applies it to the quaternion after each update. This keeps the covariance matrix P at 6×6 instead of 7×7 and avoids the unit-norm constraint issue. Most production IMU libraries (e.g., PX4's EKF2, VINS-Mono) use ESKF internally.

6.4.6 Adaptive EKF: Auto-Tuning Q and R Online

The EKF's performance depends critically on the noise covariances Q (process noise) and R (measurement noise). In practice, these are often tuned by hand — a tedious trial-and-error process. Worse, the true noise characteristics *change during operation*: an IMU on a vibrating chassis has very different noise from the same IMU sitting still. Fixed Q and R cannot handle this.

Adaptive EKF methods estimate or adjust Q and R *online*, automatically. There are several approaches, ranging from simple to sophisticated.

Method 1: Innovation-Based Adaptive Estimation (IAE)

This is the simplest and most practical method. The idea: the **innovation sequence** $\boldsymbol{\nu}_k = \mathbf{y}_k - h(\hat{\mathbf{x}}_{k|k-1})$ (the difference between what you measured and what you predicted) contains information about whether your Q and R are correct.

The key insight: If the filter is tuned correctly, the innovation should be **zero-mean white noise** with covariance $S_k = H_k P_{k|k-1} H_k^T + R$. If the innovations are consistently larger than expected, either Q is too small (you are trusting the model too much) or R is too small (you are trusting the sensors too much).

Algorithm: Estimate the actual innovation covariance from a sliding window of N_w recent innovations:

$$\hat{S}_k = \frac{1}{N_w} \sum_{j=k-N_w+1}^{k} \boldsymbol{\nu}_j \boldsymbol{\nu}_j^T \tag{110}$$

Then update R to match:

$$\hat{R}_k = \hat{S}_k - H_k P_{k|k-1} H_k^T \tag{111}$$

If $\hat{R}_k$ has negative eigenvalues (which can happen with small N_w), clamp them to a minimum value. Typical window: $N_w = 20$–100 samples.

Code:

```
// After computing innovation nu = z - h(x_pred):
// Accumulate outer product in sliding window
innov_buffer[buf_idx] = nu;
buf_idx = (buf_idx + 1) % N_WINDOW;

if (samples_seen >= N_WINDOW) {
    // Estimate innovation covariance
    Mat<NZ,NZ> S_hat = Mat<NZ,NZ>::zeros();
    for (int j = 0; j < N_WINDOW; j++) {
        S_hat = S_hat + innov_buffer[j] * innov_buffer[j].T();
    }
    S_hat = S_hat * (1.0f / N_WINDOW);

    // Adapt R
    R = S_hat - H * P_pred * H.T();
    // Clamp diagonal to minimum (prevent negative/zero variance)
    for (int i = 0; i < NZ; i++)
        if (R(i,i) < R_min) R(i,i) = R_min;
}
```

Method 2: Residual-Based Q Adaptation

While IAE adapts R, the process noise Q can be adapted using the **state residuals** (difference between the predicted and updated state):

$$\hat{Q}_k = \frac{1}{N_w} \sum_j (K_j \boldsymbol{\nu}_j)(K_j \boldsymbol{\nu}_j)^T + P_{k|k} - F_k P_{k-1|k-1} F_k^T \tag{112}$$

This is more complex and less numerically stable than IAE. In practice, many systems only adapt R and leave Q fixed, because R varies more (sensor noise changes with environment) while Q is dominated by model uncertainty (which is more constant).

Method 3: Fading Memory (Exponential Forgetting)

The simplest "adaptive" trick: multiply the predicted covariance by a **fading factor** $\alpha > 1$:

$$P_{k|k-1} = \alpha \cdot (F_k P_{k-1|k-1} F_k^T + Q) \tag{113}$$

This artificially inflates the uncertainty at each step, making the filter "forget" old data and trust new measurements more. It is equivalent to making Q larger without explicitly changing it.

- $\alpha = 1.0$: standard EKF (no forgetting).
- $\alpha = 1.01$–1.05: mild adaptation — the filter reacts faster to sudden changes.
- $\alpha > 1.1$: very aggressive — the filter almost ignores the model and follows measurements. Only use during known disturbances.

When to use: When you expect occasional sudden model changes (e.g., a robot arm picks up an unknown load — the inertia changes instantly). The fading memory prevents the filter from "locking on" to outdated dynamics.

Code (one line added to standard EKF):

```
P_pred = F * P * F.T() + Q;
P_pred = P_pred * alpha;   // <-- fading memory, alpha = 1.01
```

Method 4: Multi-Mode Adaptive EKF (for IMU)

For competition robots, the most practical approach is a simple **mode-switching** strategy rather than continuous adaptation:

```
// Compute acceleration magnitude
float accel_mag = sqrt(ax*ax + ay*ay + az*az);
float accel_err = fabs(accel_mag - 9.81f);

if (accel_err < 0.5f) {
    // Stationary or slow motion: trust accelerometer
    ekf.R(0,0) = 0.05f;  ekf.R(1,1) = 0.05f;
} else if (accel_err < 2.0f) {
    // Moderate motion: reduce trust
    ekf.R(0,0) = 0.5f;   ekf.R(1,1) = 0.5f;
} else {
    // High acceleration / vibration: mostly ignore accelerometer
    ekf.R(0,0) = 10.0f;  ekf.R(1,1) = 10.0f;
}
```

The logic: When $|\mathbf{a}| \approx g$ (9.81 m/s^2), the robot is not accelerating and the accelerometer truly measures gravity — trust it (R small). When $|\mathbf{a}|$ deviates from g, there is linear acceleration or vibration contaminating the reading — distrust it (R large).

This is not mathematically elegant, but it is **extremely effective** in practice. Many championship-winning robots use exactly this approach. It is robust, easy to tune (just three thresholds), and computationally free.

Summary: which method to use?

Method	Adapts	Complexity	Best for
Multi-mode switching	R (discrete)	Trivial	IMU on mobile robots
Fading memory	Effective Q	1 line	Sudden model changes
IAE (innovation-based)	R (continuous)	Moderate	Varying sensor noise
Residual-based	Q and R	High	Research / offline

A word of caution: Adaptive methods can *destabilize* the filter if implemented incorrectly. Always clamp adapted Q and R to reasonable ranges, use a minimum window size ($N_w \geq 20$), and validate on recorded data before deploying on hardware.

6.4.7 Luenberger Observer vs. Kalman Filter: A Comprehensive Comparison

Now that you know both observers, let us put them side by side and understand when to use which.

Aspect	Luenberger Observer	Kalman Filter
Design input	Desired pole locations (you choose)	Noise covariances Q_n, R_n (from physics or tuning)
Gain computation	One-time pole placement	Recursive (updates P and K_k every step) or steady-state
Optimality	Not optimal — you pick "reasonable" poles	Optimal (minimizes mean-squared error for linear Gaussian systems)
Noise handling	No explicit noise model — gain is fixed	Explicitly models process and measurement noise
Adaptivity	Fixed gain L — same in all conditions	Gain K_k adapts as uncertainty (P) evolves
Computational cost	Very low (matrix multiply only)	Higher (matrix multiply + inverse each step)
Tuning difficulty	Medium (choose pole locations)	Medium (choose Q_n, R_n)
Confidence output	None	P matrix tells you how confident the estimate is
Sensor fusion	Possible but manual	Natural — just expand C matrix with more sensors

The deep connection: At steady state ($k \to \infty$), the Kalman filter gain K_k converges to a constant matrix K_∞. At that point, the Kalman filter *is* a Luenberger observer with $L = K_\infty$. The difference is

that the Kalman filter *computed* this L optimally from the noise statistics, while the Luenberger approach requires you to choose L by intuition or pole placement.

In fact, the steady-state Kalman gain K_∞ is the solution to the observer Riccati equation — the exact dual of the controller Riccati equation that LQR solves. This completes the beautiful symmetry:

	Control (input) side	**Estimation (output) side**
Manual design	Pole placement for K	Luenberger: pole placement for L
Optimal design	LQR: minimize $\int(\mathbf{x}^T Q\mathbf{x} + \mathbf{u}^T R\mathbf{u})\,dt$	Kalman: minimize $E[\|\mathbf{x} - \hat{\mathbf{x}}\|^2]$
Riccati variable	P (control cost-to-go)	P (estimation uncertainty)
Tuning knobs	Q vs. R (state penalty vs. effort)	Q_n vs. R_n (model trust vs. sensor trust)

6.4.8 LQG (Linear Quadratic Gaussian)

Now that we have both LQR (optimal controller) and the Kalman filter (optimal observer), we can combine them. **LQG (Linear Quadratic Gaussian)** is what happens when you do: LQR handles control, Kalman handles estimation, and the separation principle guarantees the combination is stable.

The LQG architecture:

$$\text{Kalman filter:} \quad \dot{\hat{\mathbf{x}}} = A\hat{\mathbf{x}} + B\mathbf{u} + L(\mathbf{y} - C\hat{\mathbf{x}}) \tag{114}$$

$$\text{LQR controller:} \quad \mathbf{u} = -K\hat{\mathbf{x}} \tag{115}$$

where K comes from the control Riccati (minimizing state cost) and L comes from the estimation Riccati (minimizing estimation error). The controller never sees the true state — it acts on the Kalman estimate.

Why LQG is superior to PID for MIMO/noisy systems:

1. **Optimal noise handling:** PID amplifies noise through the derivative term. LQG's Kalman filter provides the minimum-variance state estimate — the cleanest possible signal for the controller.

2. **MIMO natural:** PID is fundamentally SISO. For a 4-state balance bot, PID needs cascaded loops with hand-tuned interactions. LQG handles all states in one K matrix.

3. **State estimation:** PID cannot estimate unmeasured states. LQG's Kalman filter reconstructs the full state from partial measurements (e.g., estimating velocity from position-only sensors).

4. **Systematic tuning:** PID has 3 knobs per loop. LQG has Q, R (control cost) and Q_n, R_n (noise model) — physically meaningful matrices.

The LQG robustness caveat: Pure LQR has guaranteed stability margins ($\geq$ 6 dB gain, $\geq 60°$ phase). When you add the Kalman filter (making it LQG), these guarantees *vanish*. The LQG controller can have arbitrarily small stability margins. This is the famous **Doyle's counter-example** (1978) — it motivated the development of H_∞ robust control.

Practical implication: LQG works well when your model is accurate and noise is well-characterized. If there is significant model uncertainty (unknown friction, varying load), consider:

- **LQG/LTR (Loop Transfer Recovery):** Modify the Kalman filter tuning to recover LQR's robustness margins at the plant input.

- H_∞ **control:** Design for worst-case model uncertainty instead of average-case noise.

- **Practical approach:** Validate LQG in simulation with $\pm 30\%$ parameter variations. If it stays stable, it is usually fine for competition use.

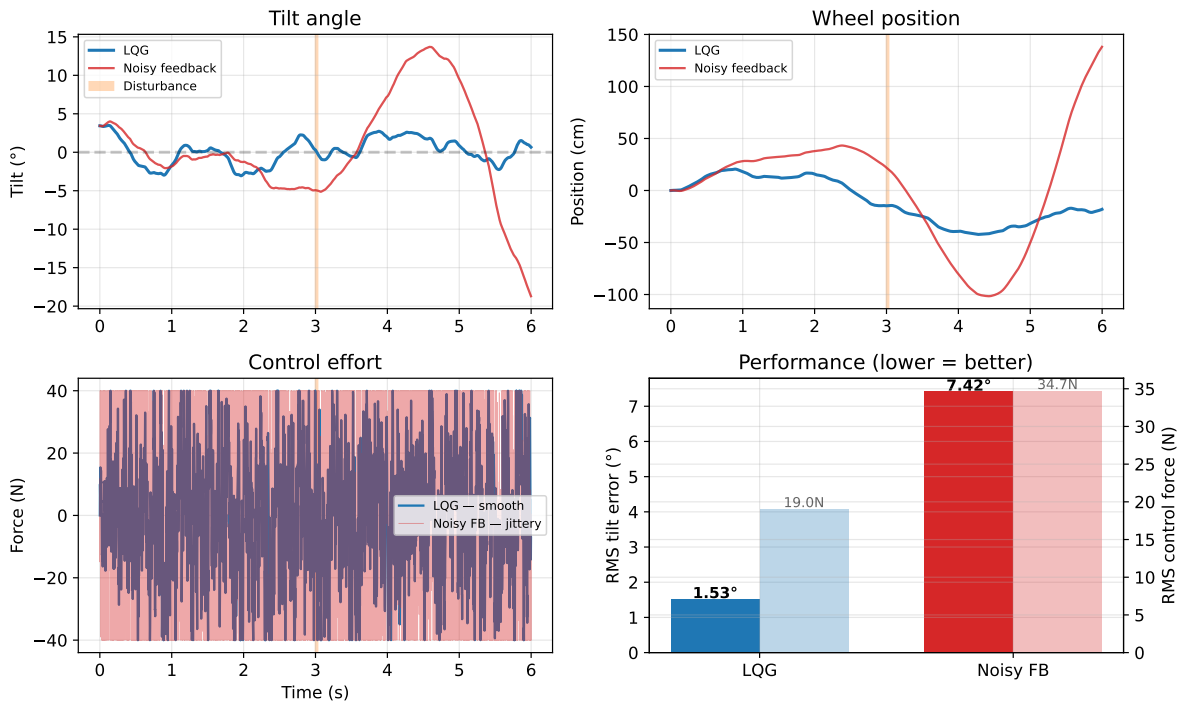

Figure 20: LQG (blue) vs direct noisy feedback (red) on a balance bot with disturbance at $t = 2$s. Both use the same LQR gains, but red uses raw noisy measurements + numerical differentiation while blue uses a Kalman filter. LQG recovers faster (top-left), maintains position (top-right), uses smoother control (bottom-left), and achieves lower RMS error (bottom-right). This demonstrates the value of the Kalman filter: same gains, same structure, dramatically different performance.

Practical decision tree:

1. **Do you need state estimation at all?** If you can measure all states directly (e.g., encoder gives position AND velocity), skip the observer entirely.
2. **Is your model accurate and noise is not a big concern?** Use Luenberger. It is simpler to implement and debug.
3. **Is sensor noise significant or do you need sensor fusion?** Use Kalman. It handles noise optimally and fuses multiple sensors naturally.
4. **Is your system nonlinear?** Use Extended Kalman Filter (EKF). It linearizes at each step, which works well for "mildly" nonlinear systems like IMU orientation estimation.
5. **Is your system highly nonlinear or does EKF diverge?** Use Unscented Kalman Filter (UKF) or consider particle filters (beyond our scope here).

Case Study: IMU Tilt Estimation — Kalman Filter vs. Complementary Filter

You are building a self-balancing robot. You need to know the tilt angle θ accurately and in real time. You have two sensors:

- **Accelerometer:** Measures gravity direction → gives you $\theta_{\text{accel}} = \arctan(a_x/a_z)$. Accurate *on average* but noisy and corrupted by any linear acceleration (when the robot moves, it "sees" phantom tilt).
- **Gyroscope:** Measures angular rate $\dot{\theta}_{\text{gyro}}$. Very clean in the short term, but *drifts* over time because you are integrating: $\theta_{\text{gyro}}(t) = \theta_0 + \int \dot{\theta}_{\text{gyro}}\, dt$. A tiny bias of 0.1°/s becomes 6° of error after one minute.

The insight: The accelerometer is good at low frequencies (static tilt) but bad at high frequen-

cies (vibration, acceleration). The gyroscope is good at high frequencies (fast rotation) but bad at low frequencies (drift). They are *complementary*.

Solution 1: Complementary filter (the quick and dirty way)

$$\theta[n] = \alpha \cdot (\theta[n-1] + \dot{\theta}_{\text{gyro}} \cdot T_s) + (1-\alpha) \cdot \theta_{\text{accel}}$$

This is literally a high-pass filter on the gyroscope and a low-pass filter on the accelerometer, with $\alpha \approx 0.98$. It works surprisingly well for a single line of code.

Solution 2: Kalman filter (the proper way)

State: $\mathbf{x} = \begin{bmatrix} \theta \\ \dot{\theta}_{\text{bias}} \end{bmatrix}$ (angle and gyro bias).

Model: the angle changes by the (bias-corrected) gyro rate:

$$\begin{bmatrix} \theta \\ \dot{\theta}_{\text{bias}} \end{bmatrix}_{k+1} = \begin{bmatrix} 1 & -T_s \\ 0 & 1 \end{bmatrix} \begin{bmatrix} \theta \\ \dot{\theta}_{\text{bias}} \end{bmatrix}_k + \begin{bmatrix} T_s \\ 0 \end{bmatrix} \dot{\theta}_{\text{gyro},k}$$

Measurement: the accelerometer gives us a noisy observation of θ:

$$y_k = \begin{bmatrix} 1 & 0 \end{bmatrix} \mathbf{x}_k + v_k$$

What the Kalman filter does that the complementary filter cannot:

- Automatically **estimates and corrects gyro bias** ($\dot{\theta}_{\text{bias}}$). This eliminates drift without manual calibration.
- Adapts the **trust ratio dynamically** based on noise statistics, rather than using a fixed α.
- Provides a **confidence estimate** (P matrix) for the state, so you know how much to trust the output.
- Generalizes easily to 3D orientation (roll, pitch, yaw) by expanding the state vector.

When is the complementary filter "good enough"? For a simple balance bot or a gimbal where the robot does not accelerate much. The complementary filter is one line of code and requires no matrix math.

When do you need the Kalman filter? When the robot undergoes significant linear acceleration (sentry robot driving fast, drone), when you need gyro bias estimation, or when fusing more than two sensors (add magnetometer for yaw, add wheel encoder for velocity).

Competition tip: Start with the complementary filter. If you see drift or poor performance during aggressive maneuvers, upgrade to a Kalman filter. The complementary filter is also a great "sanity check"—if your Kalman filter gives a wildly different answer, something is wrong with your model.

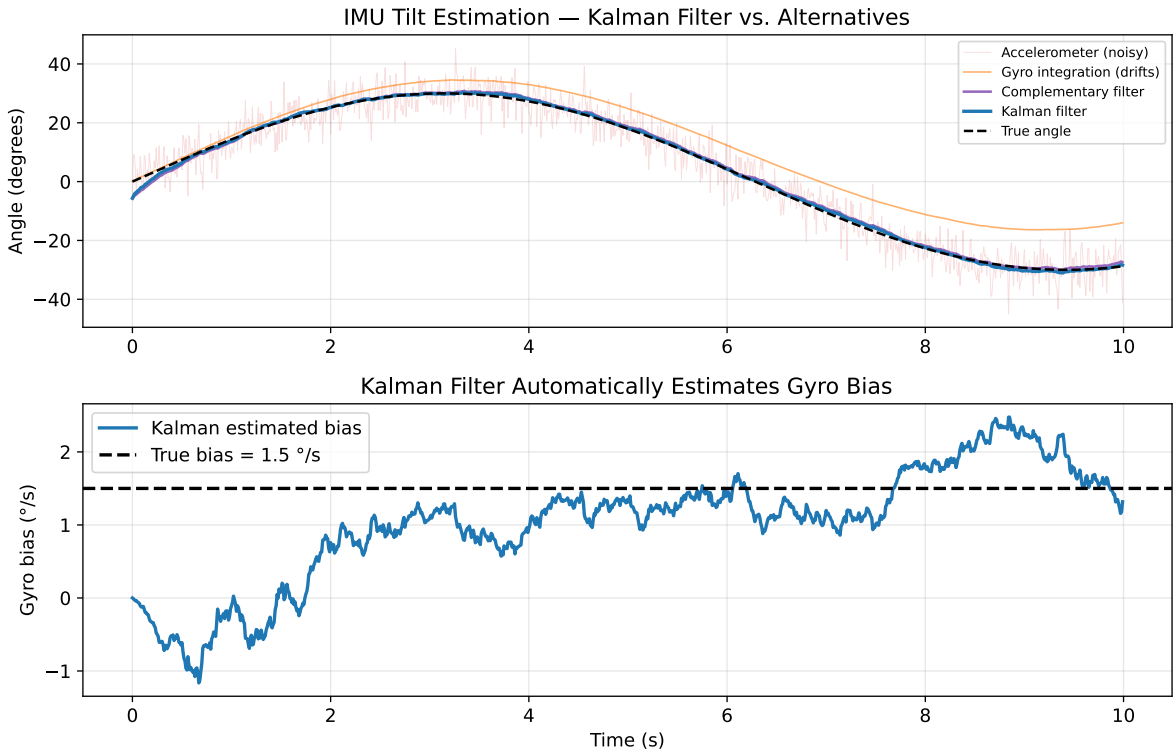

Figure 21: Top: IMU tilt estimation comparison. Pure gyro integration (orange) drifts hopelessly. The accelerometer (red, faint) is noisy. Both the complementary filter (purple) and Kalman filter (blue) track the true angle (black dashed), but Kalman is smoother. Bottom: the Kalman filter automatically estimates and converges to the true gyro bias of 1.5°/s.

Case Study: LQR Balance Bot — From Equations to Riding

Let us design the controller for a two-wheeled self-balancing robot (like a Segway). This is the "hello world" of modern control, and it beautifully showcases why LQR + Kalman is more natural than PID for MIMO systems.

The system: A two-wheeled platform with a tall body that must balance upright while also tracking a desired position.

States: $\mathbf{x} = \begin{bmatrix}\theta & \dot{\theta} & x & \dot{x}\end{bmatrix}^T$ where θ is tilt angle, x is wheel position.

The MIMO challenge: To move forward, the robot must first tilt *forward* (like a human leaning to walk). But tilting is the thing we are trying to prevent! Position control and balance are **coupled and conflicting**. A PID approach would need at least two nested loops with careful cross-coupling management. LQR handles this with a single gain matrix $K = [k_1, k_2, k_3, k_4]$:

$$u = -k_1\theta - k_2\dot{\theta} - k_3(x - x_{\text{ref}}) - k_4\dot{x}$$

Tuning via Q and R:

- Want aggressive balancing? Increase Q_{11} (penalize tilt angle).
- Want tight position tracking? Increase Q_{33} (penalize position error).
- Motors are weak? Increase R (penalize control effort).

A typical starting point: $Q = \text{diag}(100, 1, 10, 1)$, $R = 1$. This says: "I care 100× more about tilt than velocity, and 10× more about position than velocity." Run `K = lqr(A, B, Q, R)` in MATLAB, upload the four gains, and the robot balances.

The complete system:

1. **Kalman filter** estimates $[\theta, \dot{\theta}, x, \dot{x}]$ from IMU + wheel encoders.
2. **LQR controller** computes motor torque from the estimated state.
3. Both run at 500–1000 Hz on an STM32.

This Kalman+LQR architecture is the gold standard for balance bots, drone attitude control, and any system where multiple states are coupled.

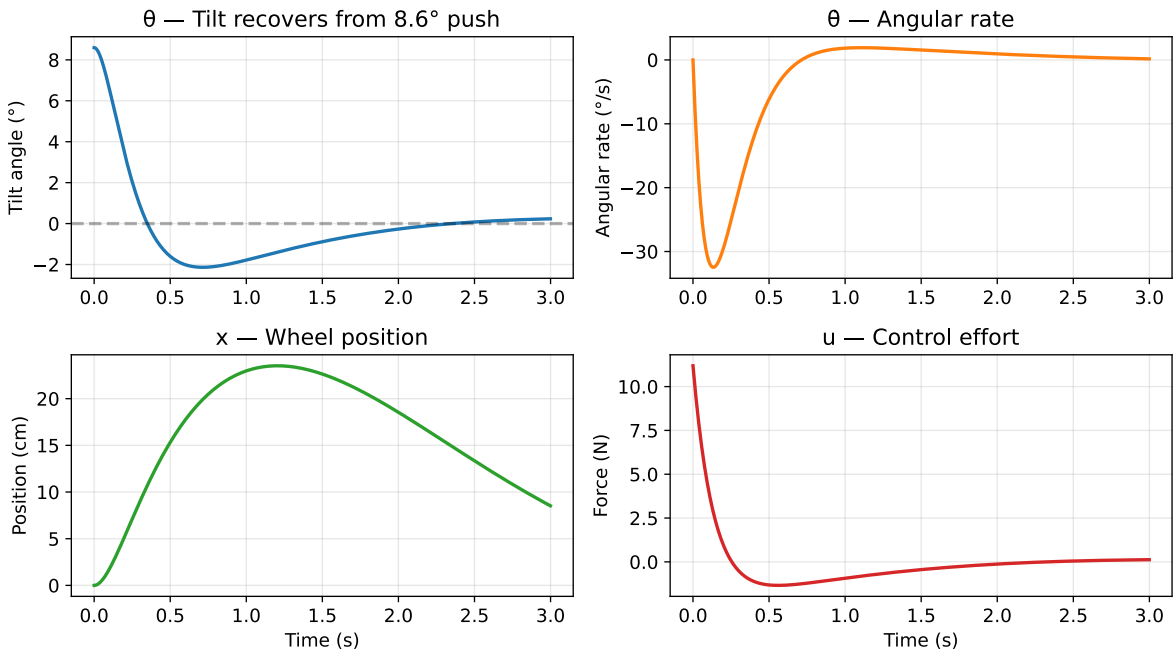

Figure 22: LQR balance bot simulation. Starting from an 8.6° tilt, the controller recovers all four states to zero. Top-left: tilt angle converges in ∼1s. Top-right: angular rate. Bottom-left: wheels move backward to catch the falling body, then return. Bottom-right: control force — note the initial burst followed by smooth decay.

6.5 Trajectory Generation: Telling Your Motor Where to Go

You cannot just command a step from 0° to 90° and expect good things to happen. The motor cannot teleport. You need a smooth **reference trajectory**—a path through position, velocity, and acceleration that respects the physical limits of your actuator. Without one, your PID saturates immediately, integral windup kicks in, and by the time the motor gets anywhere near 90° the controller is fighting itself. Trajectory generation is what separates "my turret kind of works" from "my turret tracks like a top team's."

6.5.1 Trapezoidal Velocity Profile

The trapezoidal profile is the workhorse of motion control. Given a move of distance d with maximum velocity $v_{\max}$ and maximum acceleration $a_{\max}$, it produces three phases:

1. **Accelerate** at $a_{\max}$ from rest up to $v_{\max}$
2. **Cruise** at $v_{\max}$
3. **Decelerate** at $-a_{\max}$ back to rest

Timing and distances:

$$t_{\text{accel}} = \frac{v_{\max}}{a_{\max}}, \qquad d_{\text{accel}} = \frac{1}{2} a_{\max} t_{\text{accel}}^2, \qquad d_{\text{cruise}} = d - 2\, d_{\text{accel}} \tag{116}$$

If $d_{\text{cruise}} < 0$, there is not enough distance to reach $v_{\max}$. The profile becomes **triangular**: the motor accelerates to a peak velocity $v_{\text{peak}} = \sqrt{a_{\max}\, d}$ and immediately decelerates. No cruise phase.

Reference signals at time t:

Let $t_1 = t_{\text{accel}}$, $t_2 = t_1 + d_{\text{cruise}}/v_{\max}$, $t_3 = t_2 + t_{\text{accel}}$.

$$\theta_{\text{ref}}(t) = \begin{cases} \frac{1}{2} a_{\max} t^2 & 0 \le t < t_1 \quad \text{(accelerate)} \\ d_{\text{accel}} + v_{\max}(t - t_1) & t_1 \le t < t_2 \quad \text{(cruise)} \\ d - \frac{1}{2} a_{\max}(t_3 - t)^2 & t_2 \le t \le t_3 \quad \text{(decelerate)} \end{cases} \tag{117}$$

$$\dot{\theta}_{\text{ref}}(t) = \begin{cases} a_{\max}\, t & 0 \le t < t_1 \\ v_{\max} & t_1 \le t < t_2 \\ a_{\max}(t_3 - t) & t_2 \le t \le t_3 \end{cases} \qquad \ddot{\theta}_{\text{ref}}(t) = \begin{cases} +a_{\max} & 0 \le t < t_1 \\ 0 & t_1 \le t < t_2 \\ -a_{\max} & t_2 \le t \le t_3 \end{cases} \tag{118}$$

These three signals are all you need for a fully feedforward-augmented controller (more on that in the practical box below).

6.5.2 S-Curve Profile

The trapezoidal profile has discontinuous acceleration—it jumps from 0 to $a_{\max}$ instantaneously. That sudden jerk excites mechanical resonances, stresses gears, and causes vibration. The **S-curve** (or jerk-limited) profile adds a seventh constraint: the *rate of change* of acceleration (jerk, $j_{\max}$) is bounded.

This turns the three-phase trapezoidal into a seven-phase profile:

1. Jerk up (acceleration increases from 0 to $a_{\max}$)
2. Constant acceleration at $a_{\max}$
3. Jerk down (acceleration decreases from $a_{\max}$ to 0)
4. Cruise at $v_{\max}$
5. Jerk down (acceleration goes to $-a_{\max}$)
6. Constant deceleration at $-a_{\max}$
7. Jerk up (back to zero acceleration)

The result is a position profile that is twice differentiable everywhere—smooth enough that gimbals and precision mechanisms barely feel the motion start. The math is more involved, but in practice you will rarely implement this from scratch: motion control libraries (WPILib's `TrapezoidProfile`, REV's motion magic, etc.) handle it for you.

Practical: Choosing a Profile

Trapezoidal is right for 90% of competition use cases:

- Drive wheels, flywheels, turret slew
- Simple to implement and debug—three phases, piecewise formulas

- Good enough unless you hear gear chatter or see oscillation on fast moves

S-curve when smoothness is non-negotiable:

- Gimbals and two-axis stabilization platforms
- Precision positioning mechanisms (camera sliders, encoder-based arms)
- Anywhere gear wear or vibration is a concern

The full feedforward chain for a trajectory-following turret:

At every control timestep, your trajectory planner outputs three numbers: θ_{ref}, $\dot{\theta}_{\text{ref}}$, $\ddot{\theta}_{\text{ref}}$. Use all three:

$$u = \underbrace{K_p\big(\theta_{\text{ref}} - \theta\big) + K_i \int (\cdot)\, dt + K_d \dot{e}}_{\text{outer position PID}} + \underbrace{k_v\, \dot{\theta}_{\text{ref}}}_{\text{velocity FF}} + \underbrace{k_a\, \ddot{\theta}_{\text{ref}}}_{\text{accel FF}}$$

The acceleration feedforward comes from Newton's second law for rotation: $\tau = J\alpha$, so $k_a \approx J/k_\tau$ where k_τ is the motor's torque constant. The position PID only needs to cancel small tracking errors; the feedforward does the heavy lifting.

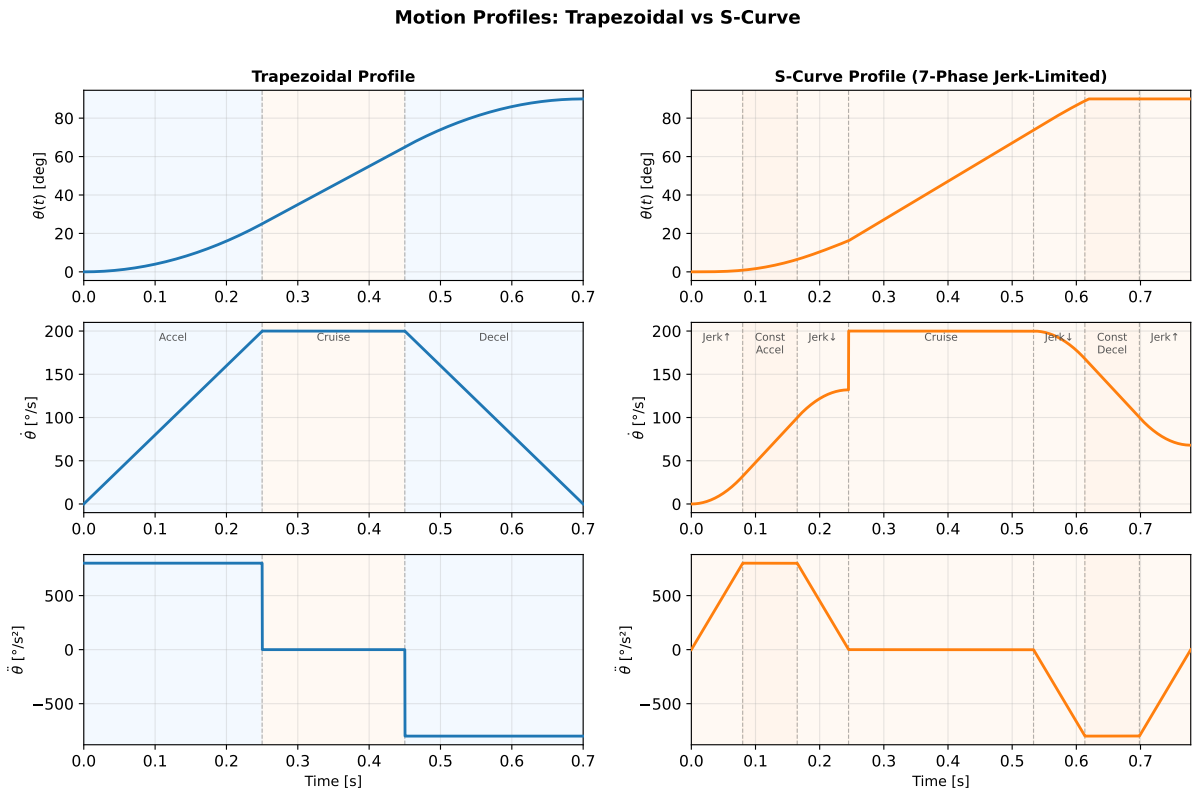

Figure 23: Trapezoidal (left) vs S-curve (right) motion profiles for a 90° move. The trapezoidal profile has discontinuous acceleration (sudden jumps), while the S-curve limits jerk for smoother motion. Both produce position, velocity, and acceleration references for feedforward control.

6.5.3 Bézier Curves and Spline Interpolation

While trapezoidal and S-curve profiles handle single point-to-point moves well, many applications require smooth trajectories through multiple waypoints or along shaped paths in 2D/3D space. **Bézier curves** and **cubic splines** are two widely used tools for this purpose.

Bézier Curves. A Bézier curve of degree n is defined by $n+1$ **control points** $\mathbf{P}_0, \mathbf{P}_1, \ldots, \mathbf{P}_n$. The curve is parameterized by $t \in [0, 1]$:

$$\mathbf{B}(t) = \sum_{i=0}^{n} \binom{n}{i} (1-t)^{n-i}\, t^i\, \mathbf{P}_i$$

The most common variant is the **cubic Bézier** ($n = 3$), which uses four control points. The curve passes through $\mathbf{P}_0$ (at $t = 0$) and $\mathbf{P}_3$ (at $t = 1$), while $\mathbf{P}_1$ and $\mathbf{P}_2$ shape the curve without lying on it. A key property is that the tangent at $\mathbf{P}_0$ points toward $\mathbf{P}_1$, and the tangent at $\mathbf{P}_3$ points from $\mathbf{P}_2$—this gives intuitive control over entry and exit directions. Bézier curves are well suited for **waypoint-based path planning** where smooth curves with controllable entry/exit directions are needed (e.g., swerve drive paths, robotic arm end-effector trajectories).

Cubic Spline Interpolation. To pass through a sequence of waypoints $\{q_0, q_1, \ldots, q_N\}$, one can use **piecewise cubic polynomials** joined at the waypoints. Each segment takes the form:

$$q_i(t) = a_i + b_i\, t + c_i\, t^2 + d_i\, t^3$$

At each junction, **continuity conditions** require that position, velocity, and acceleration match between adjacent segments, yielding C^2 continuity (continuous up to the second derivative). Two common boundary conditions are:

- **Natural spline**: the second derivative is set to zero at both endpoints.
- **Clamped spline**: the velocity (first derivative) is specified at both endpoints.

Compared to Bézier curves, cubic splines are **guaranteed to pass through all waypoints**—Bézier curves only interpolate the first and last control points. Compared to trapezoidal profiles, cubic splines provide **smooth velocity and acceleration everywhere** with no discontinuities.

When to Use What.

- **Trapezoidal / S-curve**: single point-to-point moves with explicit velocity and acceleration limits.
- **Bézier curves**: smooth 2D/3D paths with shape control (e.g., swerve drive paths, end-effector trajectories).
- **Cubic splines**: multi-waypoint trajectories where passing through every point is required (e.g., autonomous navigation sequences).

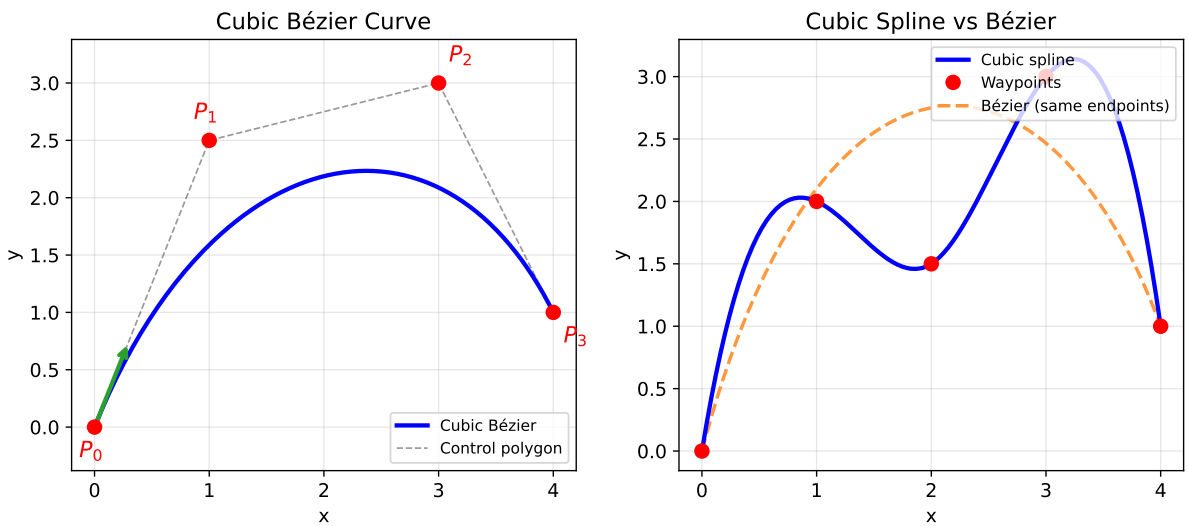

Figure 24: Left: a cubic Bézier curve defined by four control points P_0–P_3. The curve passes through P_0 and P_3; P_1 and P_2 shape the curve. Green and red arrows show the tangent directions at the endpoints. Right: cubic spline interpolation passes through all waypoints (red dots), while a Bézier curve with the same endpoints only interpolates the first and last points.

7 Appendix: Plug-and-Play C++ Modules

This appendix provides production-ready, header-only C++ implementations of every major algorithm discussed in this note. Each module is self-contained — just drop the header into your project and use it. All modules are designed for embedded systems (no dynamic allocation, no exceptions, no STL containers).

Design philosophy:

- **Header-only**: No `.cpp` files needed. Include and go.
- **No heap allocation**: Everything lives on the stack or is statically sized via templates.
- **Minimal dependencies**: Only `<cmath>` and `<cstring>`. No STL.
- **ISR-safe**: All `update()` methods are safe to call from timer interrupts.
- **Template-based**: Matrix dimensions are compile-time constants for zero-overhead abstraction.

7.1 Low-Pass Filter

First-order IIR low-pass filter. One line to construct, one line to use.

```
// lpf.hpp
#pragma once
#include <cmath>

class LowPassFilter {
public:
    LowPassFilter() : alpha_(1.0f), y_prev_(0.0f), initialized_(false) {}

    /// Configure with cutoff frequency (Hz) and sampling period (s)
    void configure(float cutoff_hz, float dt) {
        float rc = 1.0f / (2.0f * M_PI * cutoff_hz);
        alpha_ = dt / (rc + dt);
    }

    /// Configure directly with smoothing factor (0 < alpha <= 1)
    void set_alpha(float alpha) { alpha_ = alpha; }

    /// Process one sample. Call this at your fixed sample rate.
    float update(float x) {
        if (!initialized_) {
            y_prev_ = x;
            initialized_ = true;
            return x;
        }
        y_prev_ = alpha_ * x + (1.0f - alpha_) * y_prev_;
        return y_prev_;
    }

    /// Reset filter state
    void reset() { initialized_ = false; y_prev_ = 0.0f; }

    /// Get current filtered value without feeding new input
    float value() const { return y_prev_; }

private:
    float alpha_;
    float y_prev_;
    bool  initialized_;
};
```

Usage:

```
LowPassFilter gyro_filter;
gyro_filter.configure(30.0f, 0.001f);  // 30 Hz cutoff, 1 kHz loop

// In your control loop ISR:
float gyro_clean = gyro_filter.update(gyro_raw);
```

7.2 PID Controller

Full-featured PID with anti-windup (back-calculation), derivative-on-measurement, filtered derivative, and output clamping.

```
// pid.hpp
#pragma once
#include <cmath>

struct PIDConfig {
    float kp = 0.0f;
    float ki = 0.0f;
    float kd = 0.0f;
    float dt = 0.001f;         // Sampling period (s)
    float out_min = -1e6f;     // Output lower limit
    float out_max =  1e6f;     // Output upper limit
    float integral_max = 1e6f; // Integral term clamp
    float d_filter_N = 10.0f;  // Derivative filter coefficient (higher = less filtering)
};

class PID {
public:
    PID() = default;
    explicit PID(const PIDConfig& cfg) : cfg_(cfg) {}

    void configure(const PIDConfig& cfg) { cfg_ = cfg; }

    /// Compute PID output. Call once per control cycle.
    /// setpoint: desired value, measurement: actual value
    float update(float setpoint, float measurement) {
        float error = setpoint - measurement;

        // --- Proportional ---
        float p_term = cfg_.kp * error;

        // --- Integral with back-calculation anti-windup ---
        integral_ += cfg_.ki * error * cfg_.dt + anti_windup_;
        integral_ = clamp(integral_, -cfg_.integral_max, cfg_.integral_max);

        // --- Derivative on measurement (no derivative kick) ---
        //     with first-order low-pass filter
        float d_raw = -(measurement - meas_prev_) / cfg_.dt;
        float alpha_d = cfg_.dt * cfg_.d_filter_N
                      / (1.0f + cfg_.dt * cfg_.d_filter_N);
        d_filtered_ = alpha_d * d_raw + (1.0f - alpha_d) * d_filtered_;
        float d_term = cfg_.kd * d_filtered_;

        meas_prev_ = measurement;

        // --- Total output ---
        float output_raw = p_term + integral_ + d_term;
```

```
        float output = clamp(output_raw, cfg_.out_min, cfg_.out_max);

        // --- Back-calculation anti-windup ---
        if (cfg_.ki != 0.0f) {
            anti_windup_ = (output - output_raw) * (1.0f / cfg_.ki)
                         * cfg_.dt * 0.5f;
        } else {
            anti_windup_ = 0.0f;
        }

        return output;
    }

    void reset() {
        integral_ = 0.0f;
        d_filtered_ = 0.0f;
        meas_prev_ = 0.0f;
        anti_windup_ = 0.0f;
    }

    // Access internal state for debugging
    float get_integral() const { return integral_; }

private:
    PIDConfig cfg_;
    float integral_    = 0.0f;
    float d_filtered_  = 0.0f;
    float meas_prev_   = 0.0f;
    float anti_windup_ = 0.0f;

    static float clamp(float v, float lo, float hi) {
        return v < lo ? lo : (v > hi ? hi : v);
    }
};
```

Usage:

```
    PIDConfig cfg;
    cfg.kp = 6.0f;  cfg.ki = 0.3f;  cfg.kd = 3.0f;
    cfg.dt = 0.001f;
    cfg.out_min = -30000.0f;  cfg.out_max = 30000.0f;  // GM6020 range
    cfg.integral_max = 10000.0f;

    PID yaw_pid(cfg);

    // In ISR:
    float torque_cmd = yaw_pid.update(target_angle, current_angle);
```

7.3 Setpoint Ramp (LPF Soft-Start)

A dedicated setpoint smoother — wraps LPF to give ramp-limited setpoint transitions.

```
// ramp.hpp
#pragma once
#include "lpf.hpp"

class SetpointRamp {
public:
    /// ramp_time: approximate time to reach 95% of a step change (in seconds)
```

```
    void configure(float ramp_time, float dt) {
        float cutoff = 3.0f / (2.0f * M_PI * ramp_time);  // 3*tau ~ 95%
        filter_.configure(cutoff, dt);
    }

    float update(float raw_setpoint) {
        return filter_.update(raw_setpoint);
    }

    void reset() { filter_.reset(); }

private:
    LowPassFilter filter_;
};
```

7.4 Complementary Filter (IMU Tilt)

Fuses accelerometer (tilt from gravity) and gyroscope (angular rate).

```
// complementary_filter.hpp
#pragma once
#include <cmath>

class ComplementaryFilter {
public:
    /// alpha: gyro trust factor (0.95-0.99 typical)
    void configure(float alpha, float dt) {
        alpha_ = alpha;
        dt_ = dt;
    }

    /// accel_angle: tilt from accelerometer (rad)
    /// gyro_rate:   angular rate from gyro (rad/s)
    /// returns:     fused angle estimate (rad)
    float update(float accel_angle, float gyro_rate) {
        angle_ = alpha_ * (angle_ + gyro_rate * dt_)
               + (1.0f - alpha_) * accel_angle;
        return angle_;
    }

    void reset(float initial_angle = 0.0f) { angle_ = initial_angle; }
    float angle() const { return angle_; }

private:
    float alpha_ = 0.98f;
    float dt_    = 0.001f;
    float angle_ = 0.0f;
};
```

7.5 Lightweight Matrix Library

Compile-time sized matrix for embedded Kalman/LQR. No heap, no STL.

```
// mat.hpp
#pragma once
#include <cstring>
#include <cmath>

template<int ROWS, int COLS>
```

```
struct Mat {
    float data[ROWS][COLS] = {};

    float& operator()(int r, int c)       { return data[r][c]; }
    float  operator()(int r, int c) const { return data[r][c]; }

    static Mat zeros() { Mat m; return m; }

    static Mat identity() {
        static_assert(ROWS == COLS, "Identity requires square matrix");
        Mat m;
        for (int i = 0; i < ROWS; ++i) m.data[i][i] = 1.0f;
        return m;
    }

    // Matrix addition
    Mat operator+(const Mat& rhs) const {
        Mat r;
        for (int i = 0; i < ROWS; ++i)
            for (int j = 0; j < COLS; ++j)
                r.data[i][j] = data[i][j] + rhs.data[i][j];
        return r;
    }

    // Matrix subtraction
    Mat operator-(const Mat& rhs) const {
        Mat r;
        for (int i = 0; i < ROWS; ++i)
            for (int j = 0; j < COLS; ++j)
                r.data[i][j] = data[i][j] - rhs.data[i][j];
        return r;
    }

    // Matrix multiplication
    template<int COLS2>
    Mat<ROWS, COLS2> operator*(const Mat<COLS, COLS2>& rhs) const {
        Mat<ROWS, COLS2> r;
        for (int i = 0; i < ROWS; ++i)
            for (int j = 0; j < COLS2; ++j)
                for (int k = 0; k < COLS; ++k)
                    r.data[i][j] += data[i][k] * rhs.data[k][j];
        return r;
    }

    // Scalar multiplication
    Mat operator*(float s) const {
        Mat r;
        for (int i = 0; i < ROWS; ++i)
            for (int j = 0; j < COLS; ++j)
                r.data[i][j] = data[i][j] * s;
        return r;
    }

    // Transpose
    Mat<COLS, ROWS> T() const {
        Mat<COLS, ROWS> r;
        for (int i = 0; i < ROWS; ++i)
            for (int j = 0; j < COLS; ++j)
```

```
                r.data[j][i] = data[i][j];
        return r;
    }

    // 2x2 or 3x3 inverse (small matrix only, used by Kalman)
    Mat inverse() const;
};

// 1x1 inverse
template<> inline Mat<1,1> Mat<1,1>::inverse() const {
    Mat<1,1> r;
    r.data[0][0] = 1.0f / data[0][0];
    return r;
}

// 2x2 inverse
template<> inline Mat<2,2> Mat<2,2>::inverse() const {
    Mat<2,2> r;
    float det = data[0][0]*data[1][1] - data[0][1]*data[1][0];
    float inv_det = 1.0f / det;
    r.data[0][0] =  data[1][1] * inv_det;
    r.data[0][1] = -data[0][1] * inv_det;
    r.data[1][0] = -data[1][0] * inv_det;
    r.data[1][1] =  data[0][0] * inv_det;
    return r;
}

// 3x3 inverse
template<> inline Mat<3,3> Mat<3,3>::inverse() const {
    Mat<3,3> r;
    float det = data[0][0]*(data[1][1]*data[2][2]-data[1][2]*data[2][1])
              - data[0][1]*(data[1][0]*data[2][2]-data[1][2]*data[2][0])
              + data[0][2]*(data[1][0]*data[2][1]-data[1][1]*data[2][0]);
    float inv = 1.0f / det;
    r(0,0)=(data[1][1]*data[2][2]-data[1][2]*data[2][1])*inv;
    r(0,1)=(data[0][2]*data[2][1]-data[0][1]*data[2][2])*inv;
    r(0,2)=(data[0][1]*data[1][2]-data[0][2]*data[1][1])*inv;
    r(1,0)=(data[1][2]*data[2][0]-data[1][0]*data[2][2])*inv;
    r(1,1)=(data[0][0]*data[2][2]-data[0][2]*data[2][0])*inv;
    r(1,2)=(data[0][2]*data[1][0]-data[0][0]*data[1][2])*inv;
    r(2,0)=(data[1][0]*data[2][1]-data[1][1]*data[2][0])*inv;
    r(2,1)=(data[0][1]*data[2][0]-data[0][0]*data[2][1])*inv;
    r(2,2)=(data[0][0]*data[1][1]-data[0][1]*data[1][0])*inv;
    return r;
}
```

7.6 Kalman Filter

Generic discrete-time Kalman filter. Template parameters set the state and measurement dimensions at compile time.

```
// kalman.hpp
#pragma once
#include "mat.hpp"

template<int N_STATE, int N_MEAS, int N_INPUT = 1>
class KalmanFilter {
public:
    using StateVec  = Mat<N_STATE, 1>;
```

```
    using MeasVec   = Mat<N_MEAS, 1>;
    using InputVec  = Mat<N_INPUT, 1>;
    using StateMat  = Mat<N_STATE, N_STATE>;
    using InputMat  = Mat<N_STATE, N_INPUT>;
    using MeasMat   = Mat<N_MEAS, N_STATE>;
    using GainMat   = Mat<N_STATE, N_MEAS>;
    using MeasCov   = Mat<N_MEAS, N_MEAS>;

    StateMat A;   // State transition
    InputMat B;   // Input matrix
    MeasMat  C;   // Measurement matrix (called H in some texts)
    StateMat Q;   // Process noise covariance
    MeasCov  R;   // Measurement noise covariance

    StateVec x;   // State estimate
    StateMat P;   // Estimate covariance

    /// Predict step: propagate state and covariance forward
    void predict(const InputVec& u) {
        x = A * x + B * u;
        P = A * P * A.T() + Q;
    }

    /// Predict step (no input)
    void predict() {
        x = A * x;
        P = A * P * A.T() + Q;
    }

    /// Update step: incorporate measurement
    void update(const MeasVec& z) {
        // Innovation
        MeasVec y = z - C * x;
        // Innovation covariance
        MeasCov S = C * P * C.T() + R;
        // Kalman gain
        GainMat K = P * C.T() * S.inverse();
        // State update
        x = x + K * y;
        // Covariance update (Joseph form for numerical stability)
        StateMat I_KC = StateMat::identity() - K * C;
        P = I_KC * P * I_KC.T() + K * R * K.T();
    }

    /// Combined predict + update
    void step(const InputVec& u, const MeasVec& z) {
        predict(u);
        update(z);
    }
};
```

Usage — IMU tilt estimation (2-state Kalman from the case study):

```
    KalmanFilter<2, 1, 1> kf;
    float dt = 0.001f;

    // State: [angle, gyro_bias]
    kf.A(0,0) = 1.0f;  kf.A(0,1) = -dt;
    kf.A(1,0) = 0.0f;  kf.A(1,1) = 1.0f;
```

```
    kf.B(0,0) = dt;
    kf.B(1,0) = 0.0f;

    kf.C(0,0) = 1.0f;  kf.C(0,1) = 0.0f;

    // Tune these
    kf.Q(0,0) = 0.001f;  kf.Q(1,1) = 0.003f;
    kf.R(0,0) = 0.03f;

    kf.P = Mat<2,2>::identity() * 1.0f;

    // In ISR:
    Mat<1,1> u;  u(0,0) = gyro_rate;
    Mat<1,1> z;  z(0,0) = accel_angle;
    kf.step(u, z);
    float estimated_angle = kf.x(0, 0);
    float estimated_bias  = kf.x(1, 0);
```

7.7 LQR Gain (Offline Computation Helper)

LQR gain computation requires solving the Riccati equation, which is typically done *offline* in MATLAB/Python. This module applies the precomputed gain K at runtime.

```
// lqr.hpp
#pragma once
#include "mat.hpp"

template<int N_STATE, int N_INPUT>
class LQRController {
public:
    using StateVec = Mat<N_STATE, 1>;
    using InputVec = Mat<N_INPUT, 1>;
    using GainMat  = Mat<N_INPUT, N_STATE>;

    GainMat K;  // Set this from MATLAB: K = lqr(A, B, Q, R)

    /// Compute optimal input: u = -K * (x - x_ref)
    InputVec compute(const StateVec& x, const StateVec& x_ref) const {
        StateVec error = x - x_ref;
        return (K * error) * (-1.0f);
    }

    /// Compute optimal input for regulation (x_ref = 0)
    InputVec compute(const StateVec& x) const {
        return (K * x) * (-1.0f);
    }
};
```

Usage — balance bot:

```
    // Gain computed offline: K = lqr(A, B, diag([100,1,10,1]), 1)
    LQRController<4, 1> lqr;
    lqr.K(0,0) = -31.62f;  // theta gain
    lqr.K(0,1) = -5.27f;   // theta_dot gain
    lqr.K(0,2) = -3.16f;   // x gain
    lqr.K(0,3) = -4.12f;   // x_dot gain

    // In ISR (state from Kalman filter):
```

```
    Mat<4,1> state;
    state(0,0) = kf.x(0,0);  // theta
    state(1,0) = kf.x(1,0);  // theta_dot
    state(2,0) = encoder_pos;
    state(3,0) = encoder_vel;

    auto u = lqr.compute(state);
    set_motor_torque(u(0,0));
```

7.8 Cascaded PID

Two-loop cascaded controller (outer position + inner velocity) as described in the gimbal case study.

```
// cascaded_pid.hpp
#pragma once
#include "pid.hpp"

class CascadedPID {
public:
    PID outer;   // Position loop (slower)
    PID inner;   // Velocity loop (faster)

    struct Config {
        PIDConfig outer_cfg;
        PIDConfig inner_cfg;
        float ff_gain = 0.0f;  // Velocity feedforward gain
    };

    void configure(const Config& cfg) {
        outer.configure(cfg.outer_cfg);
        inner.configure(cfg.inner_cfg);
        ff_gain_ = cfg.ff_gain;
    }

    /// Full cascaded update
    /// pos_setpoint: desired position
    /// pos_measured: actual position
    /// vel_measured: actual velocity
    /// vel_feedforward: reference velocity for feedforward (optional)
    float update(float pos_setpoint, float pos_measured,
                 float vel_measured, float vel_feedforward = 0.0f) {
        // Outer loop: position -> velocity command
        float vel_cmd = outer.update(pos_setpoint, pos_measured)
                      + ff_gain_ * vel_feedforward;

        // Inner loop: velocity -> torque/current command
        float output = inner.update(vel_cmd, vel_measured);
        return output;
    }

    void reset() { outer.reset(); inner.reset(); }

private:
    float ff_gain_ = 0.0f;
};
```

Usage — gimbal yaw:

```
    CascadedPID::Config cfg;
```

```
    // Outer loop (200 Hz)
    cfg.outer_cfg = {.kp=8.0f, .ki=0.0f, .kd=0.0f, .dt=0.005f,
                     .out_min=-300.0f, .out_max=300.0f};
    // Inner loop (1 kHz)
    cfg.inner_cfg = {.kp=20.0f, .ki=1.0f, .kd=0.0f, .dt=0.001f,
                     .out_min=-30000.0f, .out_max=30000.0f,
                     .integral_max=10000.0f};
    cfg.ff_gain = 1.0f;

    CascadedPID gimbal;
    gimbal.configure(cfg);

    // Inner loop ISR (1 kHz) - only update inner
    float torque = gimbal.inner.update(vel_cmd_cached, gyro_yaw);

    // Outer loop ISR (200 Hz) - update both
    float torque = gimbal.update(target_angle, encoder_yaw,
                                 gyro_yaw, ref_velocity);
```

7.9 Advanced PID (Integral Separation + PI-D + 2-DOF)

Extends the base PID with all improvements from Section 4.5.

```
// pid_advanced.hpp
#pragma once
#include <cmath>

struct AdvancedPIDConfig {
    float kp = 0.0f, ki = 0.0f, kd = 0.0f;
    float dt = 0.001f;
    float out_min = -1e6f, out_max = 1e6f;
    float integral_max = 1e6f;
    float d_filter_N = 10.0f;     // Derivative LPF coefficient
    float setpoint_weight_b = 1.0f; // P setpoint weight (0-1)
    float setpoint_weight_c = 0.0f; // D setpoint weight (0=PI-D, 1=PID)
    float integral_sep_eps = 0.0f;  // 0 = disabled, >0 = threshold
};

class AdvancedPID {
public:
    AdvancedPID() = default;
    explicit AdvancedPID(const AdvancedPIDConfig& c) : cfg_(c) {}
    void configure(const AdvancedPIDConfig& c) { cfg_ = c; }

    float update(float setpoint, float measurement) {
        float error = setpoint - measurement;

        // --- P with setpoint weighting (2-DOF) ---
        float p_term = cfg_.kp * (cfg_.setpoint_weight_b * setpoint
                                  - measurement);

        // --- I with integral separation ---
        if (cfg_.integral_sep_eps <= 0.0f
            || fabsf(error) < cfg_.integral_sep_eps) {
            integral_ += cfg_.ki * error * cfg_.dt;
        }
        // Anti-windup: back-calculation
        integral_ += anti_windup_;
        integral_ = clamp(integral_, -cfg_.integral_max,
```

```
                        cfg_.integral_max);

        // --- D with setpoint weighting + filtered derivative ---
        float d_input = cfg_.setpoint_weight_c * setpoint - measurement;
        float d_raw = -(d_input - d_prev_) / cfg_.dt;
        float alpha = cfg_.dt * cfg_.d_filter_N
                    / (1.0f + cfg_.dt * cfg_.d_filter_N);
        d_filtered_ = alpha * d_raw + (1.0f - alpha) * d_filtered_;
        float d_term = cfg_.kd * d_filtered_;
        d_prev_ = d_input;

        // --- Output ---
        float raw = p_term + integral_ + d_term;
        float out = clamp(raw, cfg_.out_min, cfg_.out_max);

        // Back-calculation anti-windup
        anti_windup_ = (cfg_.ki != 0.0f)
            ? (out - raw) / cfg_.ki * cfg_.dt * 0.5f
            : 0.0f;
        return out;
    }

    void reset() {
        integral_ = 0; d_filtered_ = 0; d_prev_ = 0; anti_windup_ = 0;
    }

private:
    AdvancedPIDConfig cfg_;
    float integral_ = 0, d_filtered_ = 0, d_prev_ = 0, anti_windup_ = 0;
    static float clamp(float v, float lo, float hi) {
        return v < lo ? lo : (v > hi ? hi : v);
    }
};
```

Usage examples:

```
// Standard PID (same as pid.hpp)
AdvancedPIDConfig cfg;
cfg.kp=6; cfg.ki=0.3; cfg.kd=3; cfg.dt=0.001f;

// PI-D (derivative on measurement, no kick)
cfg.setpoint_weight_c = 0.0f;  // D on measurement only

// I-PD (P and D both on measurement)
cfg.setpoint_weight_b = 0.0f;  // P on measurement only
cfg.setpoint_weight_c = 0.0f;  // D on measurement only

// 2-DOF: soften setpoint response, keep disturbance rejection
cfg.setpoint_weight_b = 0.7f;  // reduce P overshoot on setpoint
cfg.setpoint_weight_c = 0.0f;  // no derivative kick

// Integral separation: only integrate when close to target
cfg.integral_sep_eps = 5.0f;   // degrees threshold
```

7.10 EKF for 6-DOF IMU (Pose Estimation)

A ready-to-use EKF that estimates roll, pitch, yaw and gyro biases from a 6-axis IMU.

```
// ekf_imu.hpp - 6-DOF IMU Pose Estimation via EKF
```

```
// State: [roll, pitch, yaw, bias_gx, bias_gy, bias_gz]
// Measurement: accelerometer -> roll, pitch
// Part of: A Practical Guide to Control Theory
// License: MIT
#pragma once
#include "mat.hpp"
#include <cmath>

class EKF_IMU {
public:
    static constexpr int NX = 6;  // states
    static constexpr int NZ = 2;  // measurements (roll, pitch from accel)

    using State  = Mat<NX, 1>;
    using Cov    = Mat<NX, NX>;
    using MeasV  = Mat<NZ, 1>;
    using MeasC  = Mat<NZ, NZ>;
    using KGain  = Mat<NX, NZ>;
    using Hmtx   = Mat<NZ, NX>;

    State x;   // [roll, pitch, yaw, bx, by, bz]
    Cov   P;

    Cov   Q;   // Process noise covariance
    MeasC R;   // Measurement noise covariance

    float dt = 0.001f;  // sampling period

    void init() {
        x = State::zeros();
        P = Cov::identity() * 0.1f;
        // Defaults (tune for your IMU)
        Q = Cov::zeros();
        Q(0,0)=0.001f; Q(1,1)=0.001f; Q(2,2)=0.001f;
        Q(3,3)=0.0001f; Q(4,4)=0.0001f; Q(5,5)=0.0001f;
        R(0,0) = 0.15f; R(1,1) = 0.15f;
    }

    /// Main update. Call at IMU sample rate.
    /// gx,gy,gz: gyro readings (rad/s)
    /// ax,ay,az: accel readings (m/s^2)
    void update(float gx, float gy, float gz,
                float ax, float ay, float az) {
        // === PREDICT ===
        float phi = x(0,0), theta = x(1,0);
        float wx = gx - x(3,0), wy = gy - x(4,0), wz = gz - x(5,0);

        float sp = sinf(phi), cp = cosf(phi);
        float tt = tanf(theta), ct = cosf(theta);

        // Euler kinematics
        float phi_dot   = wx + sp*tt*wy + cp*tt*wz;
        float theta_dot = cp*wy - sp*wz;
        float psi_dot   = (sp/ct)*wy + (cp/ct)*wz;

        // Predict state
        State xp = x;
        xp(0,0) += phi_dot * dt;
```

```
        xp(1,0) += theta_dot * dt;
        xp(2,0) += psi_dot * dt;

        // Jacobian F (6x6)
        Cov F = Cov::identity();
        F(0,0) += (cp*tt*wy - sp*tt*wz)*dt;
        F(0,1)  = (sp/(ct*ct)*wy + cp/(ct*ct)*wz)*dt;
        F(0,3) = -dt;  F(0,4) = -sp*tt*dt;  F(0,5) = -cp*tt*dt;
        F(1,0)  = (-sp*wy - cp*wz)*dt;
        F(1,4) = -cp*dt;  F(1,5) = sp*dt;
        F(2,0)  = (cp/ct*wy - sp/ct*wz)*dt;
        float st = sinf(theta);
        F(2,1)  = (sp*st/(ct*ct)*wy + cp*st/(ct*ct)*wz)*dt;
        F(2,4) = -sp/ct*dt;  F(2,5) = -cp/ct*dt;

        Cov Pp = F * P * F.T() + Q;

        // === UPDATE (accel -> roll, pitch) ===
        float z_roll  = atan2f(ay, az);
        float z_pitch = atan2f(-ax, sqrtf(ay*ay + az*az));

        MeasV z;  z(0,0) = z_roll;  z(1,0) = z_pitch;
        MeasV h;  h(0,0) = xp(0,0); h(1,0) = xp(1,0);

        MeasV innov;
        innov(0,0) = wrap_pi(z(0,0) - h(0,0));
        innov(1,0) = wrap_pi(z(1,0) - h(1,0));

        // H = [1 0 0 0 0 0; 0 1 0 0 0 0]
        Hmtx H = Hmtx::zeros();
        H(0,0) = 1.0f; H(1,1) = 1.0f;

        MeasC S = H * Pp * H.T() + R;
        KGain Kk = Pp * H.T() * S.inverse();

        x = xp + Kk * innov;
        P = (Cov::identity() - Kk * H) * Pp;
    }

    float roll()  const { return x(0,0); }
    float pitch() const { return x(1,0); }
    float yaw()   const { return x(2,0); }
    float bias_x() const { return x(3,0); }
    float bias_y() const { return x(4,0); }
    float bias_z() const { return x(5,0); }

private:
    static float wrap_pi(float a) {
        while (a >  3.14159265f) a -= 6.28318530f;
        while (a < -3.14159265f) a += 6.28318530f;
        return a;
    }
};
```

Usage:

```
    EKF_IMU ekf;
    ekf.dt = 0.002f;  // 500 Hz IMU
    ekf.init();
```

```
    // Tune for your IMU (MPU6050 example):
    ekf.Q(0,0) = 0.001f;  ekf.Q(1,1) = 0.001f;  ekf.Q(2,2) = 0.001f;
    ekf.Q(3,3) = 1e-5f;   ekf.Q(4,4) = 1e-5f;   ekf.Q(5,5) = 1e-5f;
    ekf.R(0,0) = 0.1f;    ekf.R(1,1) = 0.1f;

    // In IMU interrupt (500 Hz):
    ekf.update(gyro_x, gyro_y, gyro_z, accel_x, accel_y, accel_z);

    float roll_deg  = ekf.roll()  * 180.0f / M_PI;
    float pitch_deg = ekf.pitch() * 180.0f / M_PI;
    float yaw_deg   = ekf.yaw()   * 180.0f / M_PI;
```

7.11 TinyMPC Solver

A minimal embedded MPC solver using the Riccati-based approach from TinyMPC (see Section 6 theory). Solves box-constrained LQR problems in real time on microcontrollers.

```
// tiny_mpc.hpp - Riccati-based MPC for embedded systems
// Based on: TinyMPC (Nguyen et al., 2024)
// Precomputes Riccati recursion offline, solves ADMM online.
#pragma once
#include "mat.hpp"

template<int NX, int NU, int N_HORIZON>
class TinyMPC {
public:
    using StateVec = Mat<NX, 1>;
    using InputVec = Mat<NU, 1>;
    using StateMat = Mat<NX, NX>;
    using InputMat = Mat<NX, NU>;
    using GainMat  = Mat<NU, NX>;

    // System model (discrete)
    StateMat Ad;
    InputMat Bd;

    // Cost matrices
    StateMat Q;          // State cost
    Mat<NU,NU> R;        // Input cost
    StateMat Q_terminal; // Terminal cost (set to DARE solution)

    // Constraints
    InputVec u_min, u_max;
    StateVec x_min, x_max;

    // ADMM parameters
    float rho = 1.0f;        // Penalty parameter
    int   max_iter = 10;      // ADMM iterations (5-20 typical)

    // Precomputed Riccati gains (call precompute() once offline)
    GainMat  K_gains[N_HORIZON];    // Feedback gains
    StateMat P_mats[N_HORIZON + 1]; // Cost-to-go matrices

    /// Precompute Riccati recursion (call once, or when model changes)
    void precompute() {
        P_mats[N_HORIZON] = Q_terminal;
        for (int k = N_HORIZON - 1; k >= 0; --k) {
            // P = Q + A'*P_next*A - A'*P_next*B*(R+B'*P_next*B)^-1
```

```
        //     * B'*P_next*A
        auto P_next = P_mats[k + 1];
        auto BtP = Bd.T() * P_next;
        auto S = R + BtP * Bd;  // NU x NU
        auto S_inv = S.inverse();
        K_gains[k] = S_inv * BtP * Ad;  // NU x NX
        auto AtP = Ad.T() * P_next;
        P_mats[k] = Q + AtP * Ad - AtP * Bd * K_gains[k];
    }
}

/// Solve MPC for current state. Returns optimal first input.
InputVec solve(const StateVec& x0,
               const StateVec x_ref[N_HORIZON]) {
    // Initialize with unconstrained LQR rollout
    StateVec x_traj[N_HORIZON + 1];
    InputVec u_traj[N_HORIZON];
    x_traj[0] = x0;

    for (int k = 0; k < N_HORIZON; ++k) {
        StateVec dx = x_traj[k] - x_ref[k];
        u_traj[k] = (K_gains[k] * dx) * (-1.0f);
        // Clamp (projection step)
        for (int i = 0; i < NU; ++i) {
            float v = u_traj[k](i, 0);
            v = v < u_min(i,0) ? u_min(i,0) :
                (v > u_max(i,0) ? u_max(i,0) : v);
            u_traj[k](i, 0) = v;
        }
        x_traj[k + 1] = Ad * x_traj[k] + Bd * u_traj[k];
    }

    // ADMM refinement for constraint satisfaction
    InputVec z[N_HORIZON];   // Slack variables
    InputVec lambda[N_HORIZON]; // Dual variables
    for (int k = 0; k < N_HORIZON; ++k) {
        z[k] = u_traj[k];
        lambda[k] = InputVec::zeros();
    }

    for (int iter = 0; iter < max_iter; ++iter) {
        // Forward pass: solve unconstrained with penalty
        x_traj[0] = x0;
        for (int k = 0; k < N_HORIZON; ++k) {
            StateVec dx = x_traj[k] - x_ref[k];
            // Modified Riccati includes ADMM penalty
            InputVec u_unc = (K_gains[k] * dx) * (-1.0f);
            // ADMM correction
            u_traj[k] = u_unc + (z[k] - lambda[k]) * (rho * 0.5f);
            x_traj[k+1] = Ad * x_traj[k] + Bd * u_traj[k];
        }
        // z-update: projection onto constraints
        for (int k = 0; k < N_HORIZON; ++k) {
            for (int i = 0; i < NU; ++i) {
                float v = u_traj[k](i,0) + lambda[k](i,0);
                v = v < u_min(i,0) ? u_min(i,0) :
                    (v > u_max(i,0) ? u_max(i,0) : v);
                z[k](i, 0) = v;
```

```
                }
            }
            // Dual update
            for (int k = 0; k < N_HORIZON; ++k) {
                lambda[k] = lambda[k] + u_traj[k] - z[k];
            }
        }
        return u_traj[0];
    }
};
```

Usage — chassis trajectory tracking:

```
// 2D position control: state=[x,y,vx,vy], input=[ax,ay]
TinyMPC<4, 2, 10> mpc;  // 4 states, 2 inputs, horizon=10
float dt = 0.01f;

// Discrete double-integrator model
mpc.Ad = Mat<4,4>::identity();
mpc.Ad(0,2) = dt; mpc.Ad(1,3) = dt;
mpc.Bd(2,0) = dt; mpc.Bd(3,1) = dt;

// Costs
mpc.Q = Mat<4,4>::identity() * 10.0f;
mpc.R = Mat<2,2>::identity() * 1.0f;
mpc.Q_terminal = Mat<4,4>::identity() * 100.0f;

// Constraints: max acceleration 5 m/s^2
mpc.u_min(0,0) = -5; mpc.u_min(1,0) = -5;
mpc.u_max(0,0) =  5; mpc.u_max(1,0) =  5;

mpc.precompute();  // Once at startup

// In control loop:
Mat<4,1> x_ref[10];  // Fill from path planner
auto u = mpc.solve(current_state, x_ref);
set_accel(u(0,0), u(1,0));
```

7.12 Cubic Bézier Trajectory

Generates smooth 2D paths through four control points. Useful for swerve drive paths and end-effector trajectories.

```
// bezier.hpp -- Cubic Bezier curve for 2D trajectory generation
#pragma once

struct Vec2 { float x, y; };

class CubicBezier {
public:
    Vec2 p0, p1, p2, p3;  // Control points

    /// Evaluate position at parameter t in [0, 1]
    Vec2 position(float t) const {
        float u = 1.0f - t;
        float u2 = u * u, t2 = t * t;
        float u3 = u2 * u, t3 = t2 * t;
        return {
            u3*p0.x + 3*u2*t*p1.x + 3*u*t2*p2.x + t3*p3.x,
```

```
            u3*p0.y + 3*u2*t*p1.y + 3*u*t2*p2.y + t3*p3.y
        };
    }

    /// Evaluate first derivative (tangent) at parameter t
    Vec2 tangent(float t) const {
        float u = 1.0f - t;
        return {
            3*(u*u*(p1.x-p0.x) + 2*u*t*(p2.x-p1.x) + t*t*(p3.x-p2.x)),
            3*(u*u*(p1.y-p0.y) + 2*u*t*(p2.y-p1.y) + t*t*(p3.y-p2.y))
        };
    }

    /// Evaluate second derivative (curvature direction)
    Vec2 acceleration(float t) const {
        float u = 1.0f - t;
        return {
            6*(u*(p2.x - 2*p1.x + p0.x) + t*(p3.x - 2*p2.x + p1.x)),
            6*(u*(p2.y - 2*p1.y + p0.y) + t*(p3.y - 2*p2.y + p1.y))
        };
    }

    /// Approximate arc length by summing N linear segments
    float arc_length(int N = 100) const {
        float len = 0;
        Vec2 prev = p0;
        for (int i = 1; i <= N; ++i) {
            Vec2 cur = position(static_cast<float>(i) / N);
            float dx = cur.x - prev.x, dy = cur.y - prev.y;
            len += sqrtf(dx*dx + dy*dy);
            prev = cur;
        }
        return len;
    }
};
```

Usage — swerve drive path:

```
CubicBezier path;
path.p0 = {0, 0};        // Start
path.p1 = {1.0f, 0};     // Exit direction: forward
path.p2 = {2.0f, 1.5f};  // Entry direction at end
path.p3 = {3.0f, 1.5f};  // End point

// Track with 50 samples
for (int i = 0; i <= 50; ++i) {
    float t = static_cast<float>(i) / 50;
    Vec2 pos = path.position(t);
    Vec2 vel = path.tangent(t);  // For feedforward heading
    // Send (pos.x, pos.y) to chassis controller
}
```

7.13 Module Dependency Map

Module	File	Depends on
Low-pass filter	`lpf.hpp`	(none)
PID controller	`pid.hpp`	(none)
Advanced PID	`pid_advanced.hpp`	(none)
Setpoint ramp	`ramp.hpp`	`lpf.hpp`
Complementary filter	`complementary_filter.hpp`	(none)
Matrix library	`mat.hpp`	(none)
Kalman filter	`kalman.hpp`	`mat.hpp`
EKF IMU (6-DOF)	`ekf_imu.hpp`	`mat.hpp`
LQR controller	`lqr.hpp`	`mat.hpp`
TinyMPC solver	`tiny_mpc.hpp`	`mat.hpp`
Cascaded PID	`cascaded_pid.hpp`	`pid.hpp`
Cubic Bézier	`bezier.hpp`	(none)

All modules are MIT licensed. Copy into your project's `include/control/` directory, `#include`, and go.

Closing Thoughts

If you have made it this far, congratulations—you now have a solid theoretical foundation for designing control systems on competition robots. Here is a cheat sheet for what to reach for in different situations:

Situation	**Tool**
Noisy sensor data	Low-pass filter (Section 2)
What does my motor actually do?	Motor modeling (Section 3)
Single motor speed/position control	PID (Section 4)
Gimbal tracking	PID + feedforward (Section 4)
Smooth point-to-point moves	Trajectory generation (Section 6)
Getting PID from paper to MCU	Tustin discretization (Section 5)
Balance bot / MIMO system	LQR (Section 6)
Trajectory tracking with constraints	MPC (Section 6)
State estimation from noisy sensors	Kalman filter (Section 6)
IMU + encoder fusion	Extended Kalman filter (Section 6)
Robot can flip/tumble past $\pm 90°$	Quaternion EKF (Section 6)

Remember: theory is a tool, not an end goal. The best controller is the one that **works on the field**, not the one that looks most elegant on paper. Start simple (PID), understand your system, add complexity only when needed, and always, always test on real hardware.

Good luck at your next competition. Go build something awesome.